



SYSTEM DESIGN IN GO

System Design in Go

నిజమైన systems ని Go lo – production patterns,
observability, deployment.

ఇది Go-నిర్దిష్ట అమలు. Language-agnostic design ఆలోచనకి

[HLD_Design_Problems_Telugu.pdf](#) చూడండి.

విషయ సూచిక (Table of Contents)

Part 1 – Framework (Go lens)

1. System Design interview framework (7 steps) + Go ఎండుకు implementations
2. Back-of-envelope estimation refresher (QPS, storage, bandwidth) + numbers table
3. Core Go concurrency primitives you'll reuse in every design (worker pool, channel hub, semaphore, consistent hashing helper) - reusable toolkit

Part 2 – Case Studies (each full, with Go code for the key component)

4. URL Shortener (TinyURL) - base62, counter/snowflake ID, read-heavy caching
5. Distributed Rate Limiter - token bucket & sliding window, Redis vs in-memory
6. Chat System (WhatsApp) - WebSockets, presence, fan-out, message ordering
7. News Feed (Twitter/Instagram) - fan-out on write vs read, celebrity problem, ranking
8. Notification System - multi-channel, queue, retries, dedup, template
9. Typeahead / Autocomplete - trie, top-k, prefix
10. Video Streaming (YouTube/Netflix) - upload/transcode pipeline, CDN, ABR
11. Ride-sharing / Geo (Uber) - geohash/quadtrees, matching, location updates
12. Distributed Job Scheduler (cron at scale) - leader election, sharding, at-least-once
13. Distributed Key-Value Store - consistent hashing, replication, quorum (R/W), gossip
14. Distributed Cache (memcached-style) - sharding, LRU eviction, cache-aside
15. Payment/Ledger System - idempotency, double-entry, exactly-once, saga
16. Collaborative Editing (Google Docs) - OT vs CRDT, real-time sync, presence/cursors, versioning
17. Web Crawler - URL frontier, bloom-filter dedup, politeness/robots.txt, worker pool, BFS vs DFS
18. Distributed Message Queue (Kafka) - topics/partitions, offsets, consumer groups, ISR replication
19. Distributed File Storage (Dropbox/S3) - chunking, content-addressed dedup, metadata, delta sync

Part 3 – Reference

20. Common patterns across designs (idempotency, dedup, backpressure, sharding, caching) recap
 21. System-design-in-Go interview checklist + Memory Tips + Common Mistakes + numbers/latency cheat sheet
-

Part 1 – Framework (Go lens)

1. System Design interview framework (7 steps) + Go ఎందుకు

SYSTEM DESIGN FRAMEWORK • 7 అడుగులు

7-STEP FRAMEWORK

1 · Clarify	ప్రశ్నలు · scope కుదించడం
2 · Requirements	functional + non-functional
3 · Estimation	QPS, storage, bandwidth
4 · API	endpoints, contracts
5 · Data model	access pattern → DB choice
6 · Architecture	components + ఎందుకు
7 · Deep dive	ఒక్క భాగాన్ని లోతుగా

Go-నిర్దిష్ట అంశం

Step 6 lo – "ఈ component ని Go lo ఎలా రాస్తారు?" అని అడుగుతారు.
Worker pool, channel, context – ఇవి మే జవాబులో సహజంగా రావాలి.

వివరణ

"Design Twitter" లాంటి open-ended ప్రశ్న వచ్చినప్పుడు నేరుగా boxes గీయడం మొదలుపెట్టకూడదు. ఒక **క్రమం (framework)** follow చేయాలి. Interview అంతా ఈ 7 అడుగుల మీద నడుస్తుంది - మనం ప్రతి case study ని ఇదే క్రమంలో చేస్తాం.

7 అడుగుల Framework

1. REQUIREMENTS (5 min) – Functional + Non-functional. Scope narrow చేయి.
2. ESTIMATION (5 min) – QPS, storage, bandwidth. Design ని numbers drive చేయాలి.
3. API DESIGN (5 min) – ముఖ్య endpoints (REST/gRPC). Contract ముందు fix చేయి.
4. DATA MODEL (5 min) – Entities, SQL vs NoSQL, schema, access patterns.
5. HIGH-LEVEL (5 min) – Boxes: client → LB → services → cache → DB → queue.
6. DEEP DIVE (15 min) – 1-2 hardest components: scaling, sharding, bottlenecks.
7. WRAP UP (5 min) – Failure modes, trade-offs, monitoring, "ఇంకా scale చేయాలి".

Real-life Analogy

ఇల్లు కట్టడం లాంటిది: ముందు ఎన్ని గదులు కావాలి (requirements), ఎంత stametrial/budget (estimation), తలుపులు/కిటికీలు ఎక్కడ (API - బయటివాళ్ళు ఎలా interact చేస్తారు), foundation & plan (data model + architecture), తర్వాత ఒక్కో గది deep గా (deep dive). Foundation తప్పితే మొత్తం కూలుతుంది - అలాగే requirements/estimation తప్పితే design తప్పు.

ప్రతి అడుగులో ముఖ్య ప్రశ్నలు

అడుగు	అడగాల్సిన ప్రశ్నలు
Requirements	Users ఎంత? Read-heavy vs write-heavy? ఏ features MVP? Consistency ఎంత ముఖ్యం?
Estimation	DAU? Peak QPS? Storage/year? Bandwidth? Cache size?
API	REST vs gRPC? Pagination ఎలా? Idempotency key ఉందా?
Data Model	Relations ఎక్కువా (SQL)? Scale/flexibility (NoSQL)? Access pattern ఏది?
Deep Dive	Bottleneck ఎక్కడ? Single point of failure ఏది? Hot key ఏది?
Wrap Up	Node down అయితే? Region down అయితే? Retry storm ఏమవుతుంది?

Go ఎందుకు ఈ implementations కి? (ఇది interview లో చెప్పగలిగే reasoning)

System design లో backend services build చేయడానికి Go ఒక **default choice** అవడానికి కారణాలు - ప్రతి కారణం మనం ఏదో ఒక case study లో వాడతాం:

Go feature	System design లో ఎక్కడ ఉపయోగం
Goroutines (cheap threads)	ఒక్కో connection/request కి ఒక goroutine - Chat, WebSocket hub లో లక్షల connections
Channels (CSP)	Fan-out, pipelines, worker pools - News feed, Notification pipeline
sync package	Sharded LRU, token bucket - lock-per-shard, atomic counters
Static binary, fast start	Container/k8s లో instant scale-out, low memory footprint
net/http + context	Timeouts, cancellation, deadlines - resilience patterns
GC + low latency	Sub-ms tail latency కి tune చేయగలం (redirect service లాంటివి)
database/sql , gRPC, kafka-go	ఎక్కడైనా production-grade libraries

Interview లో ఒక్కో వాక్యం: "నేను Go వాడతాను ఎందుకంటే ఒక్కో connection కి goroutine cheap (few KB stack), channels తో fan-out/back-pressure clean గా వస్తుంది, static binary వల్ల k8s లో fast horizontal scaling - ఇవన్నీ high-concurrency backend కి perfect."

Go concurrency mental model (గుర్తుంచుకో)

"Don't communicate by sharing memory; share memory by communicating."

– Go proverb

- Goroutine = ఒక పని (function) ని background లో నడపడం. `go f()`.
- Channel = goroutines మధ్య data pipe. `ch <- v` (send), `v := <-ch` (recv).
- select = అనేక channels మీద wait (ఏది ready అయితే అది).
- sync.Mutex = shared memory ని lock చేయడం (channel అతిగా అయినప్పుడు).
- context = cancellation/deadline ని goroutine tree అంతా propagate చేయడం.

Key Points

- 7 అడుగులు **వరుసగా** follow చేయి - requirements skip చేస్తే మిగతా అంతా తప్పు direction.
- ప్రతి box కి "**ఎందుకు?**" answer ఉండాలి - అదే seniority signal.
- Go = high-concurrency backend కి default; **goroutine + channel + sync + context** అనే 4 primitives తో దాదాపు అన్ని designs వస్తాయి.
- **Think aloud** - silence negative signal.

2. Back-of-envelope Estimation refresher (QPS, storage, bandwidth)

ESTIMATION · గుర్తుంచుకోవాల్సిన సంఖ్యలు

ESTIMATION – గుర్తుంచుకోవాల్సిన సంఖ్యలు

1 రోజు	~86,400 సెకన్లు $\approx 10^5$
1 M/day	~12 QPS
1 B/day	~12,000 QPS
Peak factor	avg $\times 3$ సుంచి $\times 10$
1 KB $\times$ 1 M	1 GB
L1 cache	~1 ns · RAM ~100 ns · SSD ~100 μ s · network ~1 ms

ఎలా వేగంగా లెక్కించాలి

సెకన్లని 10^5 గా అనుకోండి – 86,400 కాదు. లెక్క చాలా సులభమవుతుంది.
Order of magnitude చాలు – interviewer ఖచ్చితత్వం కోసం చూడడం.
ప్రతి assumption ని బయటికి చెప్పడం ముఖ్యం: "నేను 20% DAU అనుకుంటున్నాను".

వివరణ

Design decisions ని **numbers** తో justify చేయాలి. "ఒక server సరిపోతుందా? ఎన్ని కావాలి? ఎంత storage? Cache ఎంత?" - వీటికి rough లెక్కలు. Interviewer precision కాదు, **reasoning** చూస్తాడు. ఒక్కో case study లో ఇవే లెక్కలు మళ్ళీ వస్తాయి కాబట్టి ఇక్కడ ఒకసారి పట్టుకో.

Real-life Analogy

వంట చేయడానికి సరుకులు కొనడం లాంటిది: 100 మందికి భోజనం అంటే బియ్యం ఎంత, కూరగాయలు ఎంత - రౌండ్ లెక్క వేసుకుంటావు. ఒక్క గింజ కూడా లెక్కపెట్టవు. System estimation కూడా అంతే - "సుమారు" చాలు, కానీ **order of magnitude** తప్పకూడదు (100 vs 100,000 తేడా ముఖ్యం).

గుర్తుంచుకోవాల్సిన Numbers (interview cheat)

అంశం	విలువ (గుర్తుంచుకో)
1 day	$\sim 86,400 \text{ sec} \approx 10^5 \text{ sec}$
1 million/day	$\sim 12 \text{ per sec}$ ($1\text{M} \div 86400$)
1 billion/day	$\sim 12,000 \text{ per sec}$
Peak QPS	Average $\times 2$ నుండి 3
1 char (ASCII / UTF-8)	1 byte (2-4 for Unicode)
KB $\rightarrow$ MB $\rightarrow$ GB $\rightarrow$ TB	ప్రతిదీ $\times 1000$ (rough)
1M users $\times$ 1KB	1 GB
1B users $\times$ 1KB	1 TB
Cache hot set (80-20 rule)	$\sim 20\%$ of data serves $\sim 80\%$ traffic

Latency numbers (SSE లో అడుగుతారు)

Operation	Latency (rough)
L1/L2 cache reference	$\sim 1\text{-}10 \text{ ns}$
Main memory (RAM) reference	$\sim 100 \text{ ns}$
Mutex lock/unlock	$\sim 25 \text{ ns}$
SSD random read	$\sim 100 \mu\text{s}$
Network round-trip (same DC)	$\sim 0.5 \text{ ms}$
Disk (HDD) seek	$\sim 10 \text{ ms}$
Network round-trip (cross-region/US $\leftrightarrow$ EU)	$\sim 50\text{-}150 \text{ ms}$

గుర్తు: Memory RAM = disk కంటే $\sim 1000\times$ fast, same-DC network = cross-region కంటే $\sim 100\times$ fast. అందుకే cache RAM లో, replicas same region లో.

Estimation Steps (ఒక క్రమం)

1. DAU (Daily Active Users) – assume చేయి (ఉదా: 100M)
2. QPS = (DAU × actions/user) / 86400
3. Peak QPS = Average QPS × 2 to 3
4. Storage = records/day × size/record × retention(years) × 365
5. Bandwidth = QPS × payload size
6. Cache = 20% of hot data (80-20 rule) → RAM లో fit అవ్వాలి
7. Servers = Peak QPS ÷ (per-server QPS capacity)

ఉదాహరణ - ఒక read-heavy system (Go లో calculator)

Interview లో మనసులో చేసే లెక్కను Go program గా. Estimation కి Go అవసరం లేదు కానీ numbers ఎలా flow అవుతాయో చూడటానికి:

```
package main

import "fmt"

func main() {
    const (
        dau           = 100_000_000 // 100M daily active users
        actionsPerUser = 5           // సగటున requests/user/day
        readWriteRatio  = 100         // 100 reads : 1 write (read-heavy)
        secondsPerDay   = 86_400
        avgPayloadBytes = 500 // ఒక్కో response ~500 bytes
        retentionYears  = 5
        bytesPerRecord  = 1_000 // 1 KB/record on disk
    )

    totalReqPerDay := dau * actionsPerUser
    avgQPS := totalReqPerDay / secondsPerDay
    peakQPS := avgQPS * 3

    writeQPS := avgQPS / readWriteRatio
    readQPS := avgQPS - writeQPS

    // Storage: writes/day × size × retention
    recordsPerDay := writeQPS * secondsPerDay
    totalRecords := recordsPerDay * 365 * retentionYears
    storageGB := (totalRecords * bytesPerRecord) / (1 << 30)

    // Read bandwidth at peak
    readBandwidthMBps := (peakQPS * avgPayloadBytes) / (1 << 20)

    fmt.Printf("Avg QPS      : %d\n", avgQPS)           // ~5,787
    fmt.Printf("Peak QPS      : %d\n", peakQPS)          // ~17,361
    fmt.Printf("Read QPS       : %d\n", readQPS)              // ~5,729
    fmt.Printf("Write QPS      : %d\n", writeQPS)             // ~57
    fmt.Printf("Storage (5yr) : %d GB\n", storageGB)         // capacity plan
    fmt.Printf("Read BW peak  : %d MB/s\n", readBandwidthMBps)
}
```

ఎందుకు ఇది ముఖ్యం: ఈ numbers design ని *drive* చేస్తాయి. Read QPS >> write QPS ⇒ **cache + read replicas**. Storage పెద్దది ⇒ **sharding**. Peak >> average ⇒ **auto-scaling + queue for buffering**.

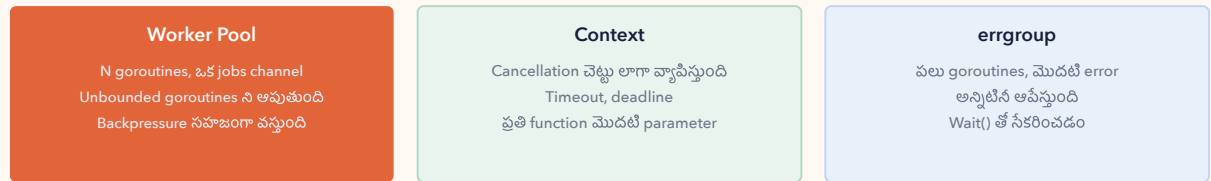
Key Points

- 1M/day ≈ 12/sec , 1B/day ≈ 12K/sec , 1 day ≈ 10⁵ sec - ఈ మూడు తో దాదాపు అన్ని QPS లెక్కలు.
- **Peak = avg × 2-3**. Design ని peak కి చేయి, average కి కాదు.
- Read >> write ⇒ cache; storage పెద్దది ⇒ shard; payload పెద్దది ⇒ CDN/blob store.
- Order of magnitude సరిపోతుంది - "సుమారు 17K QPS" అని చెప్పు, "17,361" కాదు.

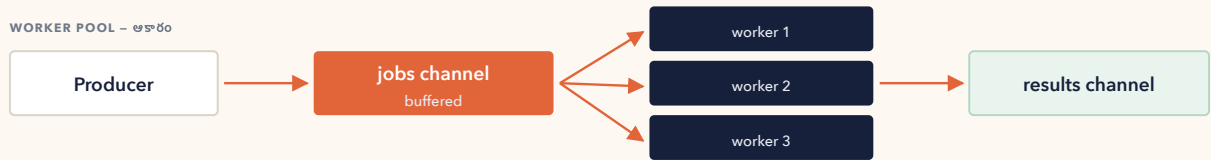
3. Core Go concurrency primitives - reusable toolkit

GO CONCURRENCY TOOLKIT · WORKER POOL, CONTEXT, ERRGROUP

GO CONCURRENCY TOOLKIT - ఈ మూడూ దాదాపు ప్రతి design lo



WORKER POOL - ఆకారం

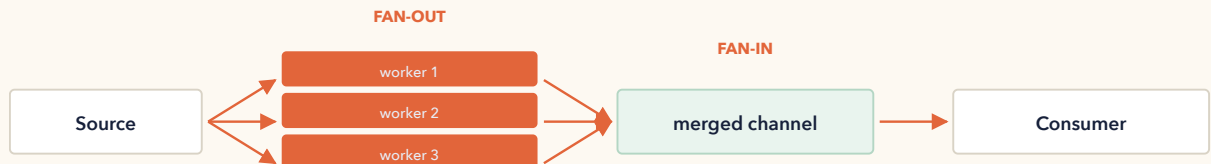


ఎందుకు ఇది ముఖ్యం

"ప్రతి request కి ఒక goroutine" - Go lo చౌక, కానీ **అపరిమితం కాదు**.
10 లక్షల jobs వస్తే - 10 లక్షల goroutines memory ని తీసేస్తాయి, DB pool ని ముంచేస్తాయి.
Worker pool = సమాంతరతకి ఒక పద్ధతి. అదే production Go code lo అత్యంత సాధారణమైన pattern.

FAN-OUT / FAN-IN · GO CONCURRENCY PATTERN

FAN-OUT / FAN-IN - Go concurrency మొక్క గుండ్



Fan-in lo ముఖ్యమైన detail

Merged channel ని **ఎప్పుడు మూయాలి?** - అన్ని workers ముగిసినప్పుడు.
sync.WaitGroup తో లెక్కించి, ఒక goroutine lo wg.Wait() తర్వాత close(out).
ముందే మూస్తే - "send on closed channel" panic. మూయకపోతే - consumer శాశ్వతంగా వేచి ఉంటాడు.

వివరణ

ప్రతి case study లో మనం ఈ 4 building blocks లో ఏదో ఒకటి వాడతాం. వీటిని ఇక్కడ **ఒకసారి** idiomatic Go లో వ్రాసి, తర్వాత "toolkit నుండి worker pool వాడతాం" అని reference చేస్తాం. ఇవి interview లో whiteboard మీద గీయగలగాలి.

Real-life Analogy

ఒక restaurant kitchen:

- **Worker pool** = నిర్ణీత సంఖ్య cooks (workers), orders ఒక queue (channel) నుండి తీసుకుంటారు. Cooks 5 మంది ఉంటే ఒకసారి 5 orders. మిగతావి wait.
- **Channel hub** = ఒక waiter (hub goroutine) అన్ని tables (clients) ని coordinate చేస్తాడు - orders తీసుకుని, food deliver చేసి. ఒక్కడే state manage చేస్తాడు కాబట్టి గందరగోళం లేదు (no locks).
- **Semaphore** = kitchen లో 3 ovens మాత్రమే - ఒకసారి 3 dishes మాత్రమే bake. 4వది oven ఖాళీ అయ్యేదాకా wait.
- **Consistent hashing** = ఏ cook ఏ cuisine (key) చూస్తాడో నిర్ణయించే chart - కొత్త cook వస్తే/పోతే అందరూ మొత్తం మారిపోకుండా, కొంచెం మాత్రమే redistribute.

Toolkit 1 – Worker Pool (bounded concurrency)

సమస్య: 1 million tasks ఉన్నాయి, కానీ ఒకసారి 1 million goroutines పెంచితే memory/DB crash.

Solution: N workers, ఒక jobs channel, results channel.

```
package pool

import "sync"

// Job = ఒక పని; Result = దాని ఫలితం.
type Job[T any] func() T

// Run – jobs ని N workers తో process చేసి results return చేస్తుంది.
// bounded concurrency: ఏ క్షణంలోనైనా గరిష్టంగా workers goroutines మాత్రమే.
func Run[T any](workers int, jobs []Job[T]) []T {
    jobCh := make(chan int, len(jobs)) // index పంపుతాం (result slot కోసం)
    results := make([]T, len(jobs))
    var wg sync.WaitGroup

    // N workers ని start చేయి
    for w := 0; w < workers; w++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            for i := range jobCh { // channel close అయ్యేదాకా jobs తీసుకో
                results[i] = jobs[i]() // ఒక్కో job వేరే index → lock
            }
        }()
    }

    // jobs ని feed చేయి, తర్వాత channel close చేయి
```

```

    for i := range jobs {
        jobCh <- i
    }
    close(jobCh) // ఇక jobs లేవు → workers exit అవుతారు

    wg.Wait() // అన్ని workers అయ్యేదాకా wait
    return results
}

```

కీలకం: `close(jobCh)` = "ఇక pani లేదు" signal. Workers `for i := range jobCh` loop నుండి బయటికి వస్తారు. `wg.Wait()` అన్నీ finish అయ్యేదాకా block. ఒక్కో result వేరే index కాబట్టి **mutex అవసరం లేదు** - ఇది idiomatic Go.

Toolkit 2 – Channel Hub (single-owner state, no locks)

సమస్య: అనేక clients ఒకే shared state (registered clients map) ని modify చేస్తున్నాయి. Locks తో deadlock/contention. **Solution:** ఒకే goroutine ("hub") state ని own చేస్తుంది; మిగతావి channels ద్వారా requests పంపుతాయి. ఇది Chat System §6 లో గుండె.

```

package hub

// ఇది pattern skeleton – §6 లో నిజమైన WebSocket hub గా విస్తరిస్తాం.
type Hub[C comparable] struct {
    clients    map[C]bool
    register   chan C
    unregister chan C
    broadcast  chan string
}

func New[C comparable]() *Hub[C] {
    return &Hub[C]{
        clients:    make(map[C]bool),
        register:   make(chan C),
        unregister: make(chan C),
        broadcast:  make(chan string),
    }
}

// Run – ఒకే goroutine. clients map ని ఇది మాత్రమే touch చేస్తుంది → NO mutex.
func (h *Hub[C]) Run() {
    for {
        select {
        case c := <-h.register:
            h.clients[c] = true
        case c := <-h.unregister:
            delete(h.clients, c)
        case msg := <-h.broadcast:
            for c := range h.clients {
                _ = c
                _ = msg // ఒక్కో client కి send (§6 లో deliver చేస్తాం)
            }
        }
    }
}

```

ఎందుకు locks లేవు: `clients` map ని **ఒకే goroutine** (Run) మాత్రమే చదువుతుంది/ వ్రాస్తుంది. మిగతా goroutines దాన్ని నేరుగా touch చేయవు - channels ద్వారా requests పంపుతాయి. ఇది Go proverb "share memory by communicating" యొక్క నిజమైన రూపం.

Toolkit 3 – Semaphore (limit concurrent access to a resource)

సమస్య: DB కి ఒకేసారి 10,000 connections వస్తే crash. గరిష్టంగా N concurrent మాత్రమే కావాలి.

Solution: buffered channel = counting semaphore.

```
package sem

// Semaphore – buffered channel తో. capacity = గరిష్ట concurrent holders.
type Semaphore chan struct{}

func New(n int) Semaphore { return make(Semaphore, n) }

// Acquire – slot దొరికేదాకా block (channel full అయితే wait).
func (s Semaphore) Acquire() { s <- struct{}{} }

// Release – slot ఖాళీ చేయి.
func (s Semaphore) Release() { <-s }

// ఉదాహరణ వాడకం:
// sem := New(10)           // గరిష్టంగా 10 concurrent DB calls
// sem.Acquire()
// defer sem.Release()
// db.Query(...)           // ఏ క్షణంలోనైనా max 10 goroutines ఇక్కడ
```

ఎందుకు channel: buffered channel capacity = N. `s <- struct{}{} channel full` అయితే block అవుతుంది (11వ goroutine wait). `struct{}{} = zero bytes` (memory waste లేదు, కేవలం counting). ఇది `context` తో కలిపి timeout కూడా పెట్టవచ్చు: golang.org/x/sync/semaphore package దీన్నే weighted గా ఇస్తుంది.

Toolkit 4 – Consistent Hashing ring helper

సమస్య: N cache/DB nodes; ఒక key ఏ node కి వెళ్తుంది? `hash(key) % N` వాడితే - ఒక node add/remove అయితే **దాదాపు అన్ని keys** remap అవుతాయి (cache stampede). **Solution:** consistent hashing - node add/remove అయితే **~1/N keys** మాత్రమే మారతాయి. Virtual nodes తో load balance. ఇది KV store §13 లో గుండ్డె.

```
package hashring

import (
    "hash/crc32"
    "sort"
    "strconv"
)

// Ring – consistent hashing ring with virtual nodes.
type Ring struct {
```

```

replicas int          // ఒక్క physical node కి ఎన్ని virtual nodes
keys      []uint32     // sorted hash positions (ring)
hashMap   map[uint32]string // hash → physical node name
}

func New(replicas int) *Ring {
    return &Ring{replicas: replicas, hashMap: make(map[uint32]string)}
}

func (r *Ring) hash(s string) uint32 { return crc32.ChecksumIEEE([]byte(s)) }

// Add – physical node ని ring కి చేర్చు (replicas virtual points న్న).
func (r *Ring) Add(nodes ...string) {
    for _, node := range nodes {
        for i := 0; i < r.replicas; i++ {
            h := r.hash(node + "#" + strconv.Itoa(i)) // vnode
            r.keys = append(r.keys, h)
            r.hashMap[h] = node
        }
    }
    sort.Slice(r.keys, func(i, j int) bool { return r.keys[i] < r.keys[j] })
}

// Get – ఈ key ఏ physical node కి? (ring మీద clockwise next node)
func (r *Ring) Get(key string) string {
    if len(r.keys) == 0 {
        return ""
    }
    h := r.hash(key)
    // binary search: h కంటే >= మొదటి position
    idx := sort.Search(len(r.keys), func(i int) bool { return r.keys[i] >= h })
    if idx == len(r.keys) {
        idx = 0 // wrap around (ring)
    }
    return r.hashMap[r.keys[idx]]
}

```

ఎందుకు virtual nodes: physical node ఒక్కటే point గా ఉంటే ring మీద uneven distribution (కొన్ని nodes కి ఎక్కువ keys). ఒక్క physical node ని replicas (ఉదా: 150) virtual points గా చెల్లాచెదురు చేస్తే **smooth balance**. Node down అయితే దాని keys ring మీద పక్క nodes కి పంచుకుంటాయి - మొత్తం కాదు.

Bonus – sync.Mutex vs channel: ఎప్పుడు ఏది?

పరిస్థితి	వాడు
Simple shared counter/map, short critical section	sync.Mutex / sync.RWMutex
అనేక goroutines coordinate, ownership transfer	channel
Read చాలా ఎక్కువ, write తక్కువ	sync.RWMutex
Once-only init (singleton)	sync.Once

పరిస్థితి	వాడు
Atomic counter (lock-free)	sync/atomic
Bounded concurrency	buffered channel (semaphore)

Rule of thumb: "state ఒకే చోట ఉండి, దాన్ని protect చేయాలంటే **Mutex**. State ని pass చేయాలంటే/coordinate చేయాలంటే **channel**." అతిగా channels వాడకు - simple counter కి mutex clean.

Key Points

- ఈ 4 toolkit pieces (worker pool, channel hub, semaphore, hash ring) 15 case studies అంతటా తిరిగి వస్తాయి - వీటిని whiteboard మీద గీయగలగాలి.
- **Worker pool** = bounded concurrency (channel + WaitGroup).
- **Channel hub** = single-owner state, lock లేకుండా (Chat §6).
- **Semaphore** = buffered channel, resource limit.
- **Consistent hashing** = node add/remove కి $\sim 1/N$ keys మాత్రమే మారతాయి; virtual nodes తో balance (KV §13, Cache §14).
- Mutex vs channel: state protect $\Rightarrow$ Mutex; state coordinate $\Rightarrow$ channel.

Part 2 – Case Studies

4. URL Shortener (TinyURL / bit.ly)

Real-life Analogy: పొడవైన ఇంటి address (long URL) కి ఒక చిన్న **nickname** (short code) ఇవ్వడం లాంటిది. "3వ లైన్, 4వ క్రాస్, జయనగర్..." అనే బదులు "మా ఇల్లు" అంటే చాలు - అందరికీ దారి తెలుసు. Short code = ఆ nickname, redirect = దారి చూపడం.

Requirements (Functional + Non-functional)

Functional:

- పొడవైన URL → చిన్న URL generate చేయడం
- చిన్న URL → అసలు URL కి redirect (301/302)
- (optional) custom alias (`bit.ly/my-brand`), expiry, click analytics

Non-functional:

- **High availability** - redirect ఎప్పుడూ down కాకూడదు (down అయితే ప్రతి link విరిగిపోతుంది)
- **Low latency** - redirect < 50ms (users wait చేయరు)
- **Read-heavy** - reads:writes ≈ **100:1** (ఒకసారి create, వందల సార్లు click)
- Short codes **unique**, guessable కాకూడదు (కొంతవరకు)

Estimation (QPS, storage)

```
కొత్త URLs : 100M/day → 100M / 86400 ≈ 1,160 writes/sec
Reads (100:1): 100 × 1,160 ≈ 116,000 reads/sec (peak ~350K)
Storage/record: ~500 bytes (shortCode + longUrl + metadata)
5 years : 100M × 365 × 5 ≈ 182 billion URLs
Storage : 182B × 500B ≈ 91 TB → sharding తప్పనిసరి
Short code : 7 base62 chars → 62^7 ≈ 3.5 trillion (182B కి చాలు)
Cache (hot 20%): కొన్ని GB hot URLs → Redis లో fit
```

Design driver: reads >> writes ⇒ **aggressive caching + read replicas**. Storage పెద్దది ⇒ **sharded NoSQL**. Redirect latency-critical ⇒ **Go service, cache-first**.

API Design

```
POST /api/v1/shorten
Body: { "longUrl": "https://...", "customAlias": "?", "expiryDays": 365 }
→ 201 { "shortUrl": "https://sho.rt/aB3xK9z" }
```

```
GET /{shortCode}
→ 301 Moved Permanently, Location: <longUrl> (browser caches → fast)
→ 404 if not found / expired

GET /api/v1/analytics/{shortCode}
→ 200 { "clicks": 12903, "topReferrers": [...] }
```

301 vs 302: 301 (permanent) = browser/CDN caches ⇒ మళ్ళీ మన server hit అవదు ⇒ **fast, కానీ analytics లేవు**. 302 (temporary) = ప్రతిసారి server hit ⇒ **analytics వస్తాయి కానీ load ఎక్కువ**. Analytics కావాలంటే 302, లేదా 301 + async pixel/beacon.

Data Model

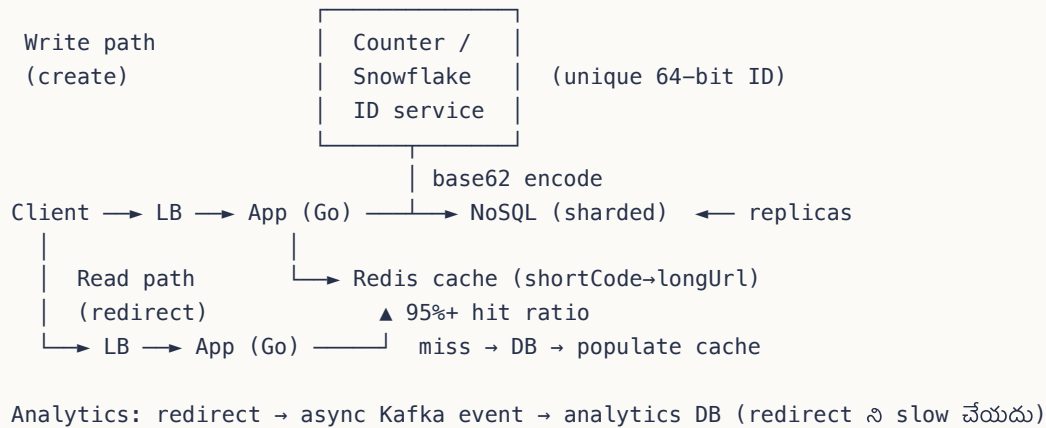
NoSQL (key-value; massive scale, no joins అవసరం లేదు):

Table: urls (partition key = shortCode)

shortCode	longUrl	createdAt	expiry	userId
aB3xK9z	https://example.com/...	169..	170..	u_4412

Table: counters (ID generation, ఒక్కో shard కి range block)
Cache: Redis shortCode → longUrl (TTL, LRU eviction)

High-level Architecture



Deep Dive – Short code ఎలా generate చేయాలి? (hardest part)

మూడు options:

పద్ధతి	Pros	Cons
Counter + Base62	No collision, short, sequential	Global counter bottleneck; guessable
Hash (MD5/SHA) → 7 chars	Stateless	Collision possible → check + retry

పద్ధతి	Pros	Cons
Snowflake (distributed ID)	No central counter, time-ordered	64-bit → base62 ~11 chars (కొంచెం పొడవు)

Best hybrid: range-based counter (counter bottleneck solve చేస్తుంది) → base62. Counter service ఒక్కో app instance కి **1000 IDs block** (range) ఇస్తుంది. Instance ఆ block లోపల local గా increment - మళ్ళీ counter service ని అడగదు (1000 సార్లకి ఒకసారి మాత్రమే). ఇది single point bottleneck ని 1000× తగ్గిస్తుంది.

Collision: counter/snowflake వాడితే collision **అసలు ఉండదు** (unique ID guaranteed). Hash వాడితే మాత్రమే collision - అప్పుడు DB `INSERT ... IF NOT EXISTS` (conditional write) తో check.

Go Implementation – ID generator (range counter + base62) + redirect handler

ఇది system యొక్క గుండె: unique short code generate చేయడం, thread-safe గా, counter service ని అరుదుగా మాత్రమే hit చేస్తూ.

```
package shortener

import (
    "errors"
    "net/http"
    "sync"
)

const base62 = "0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ"

// EncodeBase62 – unique 64-bit ID → short string (collision లేదు).
func EncodeBase62(num uint64) string {
    if num == 0 {
        return string(base62[0])
    }
    buf := make([]byte, 0, 11)
    for num > 0 {
        buf = append(buf, base62[num%62])
        num /= 62
    }
    // reverse (మనం least-significant digit ముందు append చేశాం)
    for i, j := 0, len(buf)-1; i < j; i, j = i+1, j-1 {
        buf[i], buf[j] = buf[j], buf[i]
    }
    return string(buf)
}

// IDGenerator – range-based. Counter service నుండి block తీసుకుని
// local గా serve చేస్తుంది. Counter service ని 'blockSize' కి ఒకసారి hit.
type IDGenerator struct {
    mu      sync.Mutex
    next    uint64 // ప్రస్తుత block లో తర్వాత ID
    max     uint64 // ఈ block ముగింపు (exclusive)
    blockSize uint64
    allocate func(size uint64) (start uint64, err error) // counter service call
}
```



```

}

func NewIDGenerator(blockSize uint64, allocate func(uint64) (uint64, error))
*IDGenerator {
    return &IDGenerator{blockSize: blockSize, allocate: allocate}
}

// Next – thread-safe. చాలావరకు lock + increment (fast path).
// Block అయిపోతేనే counter service ని hit (slow path, blockSize కి ఒకసారి).
func (g *IDGenerator) Next() (uint64, error) {
    g.mu.Lock()
    defer g.mu.Unlock()

    if g.next >= g.max { // block exhausted → refill
        start, err := g.allocate(g.blockSize)
        if err != nil {
            return 0, err
        }
        g.next = start
        g.max = start + g.blockSize
    }
    id := g.next
    g.next++
    return id, nil
}

// ---- HTTP handlers ----

type Store interface {
    Save(shortCode, longURL string) error
    Load(shortCode string) (string, bool) // cache-first ఇక్కడ implement
}

type Service struct {
    ids *IDGenerator
    store Store
}

// Shorten – write path: unique ID → base62 → store.
func (s *Service) Shorten(w http.ResponseWriter, r *http.Request) {
    longURL := r.FormValue("longUrl")
    if longURL == "" {
        http.Error(w, "longUrl required", http.StatusBadRequest)
        return
    }
    id, err := s.ids.Next()
    if err != nil {
        http.Error(w, "id allocation failed", http.StatusServiceUnavailable)
        return
    }
    code := EncodeBase62(id)
    if err := s.store.Save(code, longURL); err != nil {
        http.Error(w, "save failed", http.StatusInternalServerError)
        return
    }
    w.WriteHeader(http.StatusCreated)
    _, _ = w.Write([]byte("https://sho.rt/" + code))
}

```

```
// Redirect - read path: cache-first lookup → 301. ఇది hot path,
// చాలావరకు Redis నుండి వస్తుంది (DB touch అవదు).
func (s *Service) Redirect(w http.ResponseWriter, r *http.Request) {
    code := r.PathValue("code") // Go 1.22+ path wildcard
    longURL, ok := s.store.Load(code)
    if !ok {
        http.NotFound(w, r)
        return
    }
    // async analytics event ఇక్కడ Kafka కి పంపవచ్చు (redirect ని block చేయకుండా)
    http.Redirect(w, r, longURL, http.StatusMovedPermanently) // 301
}

var ErrExhausted = errors.New("id space exhausted")
```

Go ఎందుకు perfect ఇక్కడ: redirect handler దాదాపు cache lookup + 301 మాత్రమే - Go net/http ఒక్కో request కి goroutine, sub-ms latency. IDGenerator లో mutex fast path (local increment) వల్ల 116K QPS ని ఒక్క instance handle చేయగలదు; counter service ని blockSize కి ఒకసారి మాత్రమే hit ⇒ central bottleneck 1000× తగ్గింది.

Bottlenecks & Trade-offs

- **Counter single point** → range allocation (block per instance) తో solve; counter service down అయినా current block అయిపోయేదాకా serve.
- **Read hot spots** → viral link ఒకటే lakhs QPS. Redis + **local in-process cache** (Go `sync.Map` / LRU) hybrid; CDN edge caching.
- **301 vs 302 trade-off** → 301 fast కానీ analytics లేవు; 302 analytics కానీ ప్రతిసారి origin hit. మేము 301 + async beacon తో balance.
- **Storage 91 TB** → shardKey = `shortCode` hash; hot shard సమస్యకి consistent hashing (§3 toolkit).
- **Guessable codes** → sequential IDs enumerable. అవసరమైతే snowflake లేదా encrypt-then-encode.

Key Points

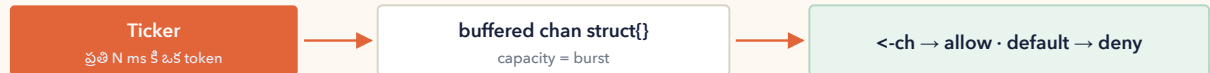
- **Range-based counter + base62** = collision-free, short, scalable ID. Counter bottleneck ని block allocation తో పగలగొట్టు.
- Read-heavy ⇒ **cache-first redirect** (95%+ hit), read replicas.
- $62^7 \approx 3.5 \text{ trillion}$ codes - 7 chars చాలు.
- 301 = fast/no-analytics, 302 = analytics/origin-hit.
- Go: redirect = goroutine + cache lookup + 301, ultra-low latency.

5. Distributed Rate Limiter

Real-life Analogy: నీటి ట్యాంక్ + కొళాయి. ట్యాంక్ నిండా tokens (నీళ్ళు) ఉంటాయి, పైనుండి నిర్ణీత rate తో నిండుతుంది (refill). ప్రతి request ఒక token తీసుకుంటుంది (కొళాయి తిప్పడం). ట్యాంక్ ఖాళీ అయితే - wait లేదా reject (429). Bursts allow అవుతాయి (ట్యాంక్ నిండి ఉంటే), కానీ steady state లో refill rate దాటదు.

RATE LIMITER IN GO · CHANNEL-BASED TOKEN BUCKET

RATE LIMITER in Go - channel based token bucket



ఎందుకు ఇది idiomatic Go

Lock లేదు, mutex లేదు - channel యొక్క buffer అదే token bucket.
select తో default case - non-blocking check (ఖాళీగా ఉంటే వెంటనే deny).
ఇంకా సులభం: golang.org/x/time/rate - production కి అదే వాడాలి.

కానీ ఇది single-process మాత్రమే

పలు pods ఉంటే - ప్రతి pod కి తన సొంత bucket. నిజమైన limit = pods × limit.
Distributed కావాలంటే Redis + Lua (HLD book Problem 02 చూడండి).

Requirements (Functional + Non-functional)

Functional:

- User / IP / API-key కి నిర్ణీత window లో గరిష్టంగా **N requests**
- దాటితే **429 Too Many Requests** + `Retry-After` header
- వేర్వేరు tiers (free: 100/min, paid: 10000/min)

Non-functional:

- **Distributed** - అనేక app servers మధ్య **shared limit** (ఒక్కో server local counter అయితే user $N \times \text{servers}$ దాటగలడు)
- **Low latency** - ప్రతి request ముందు check $\Rightarrow$ overhead $< 1\text{ms}$
- **Accurate** but **fault-tolerant** - limiter down అయితే మొత్తం API down కాకూడదు

Estimation (QPS, storage)

1M users, ప్రతి active user ~ 10 req/min $\rightarrow$ check QPS $\approx$ requests QPS
Peak API QPS: 100K $\rightarrow$ 100K rate-limit checks/sec
State/user : ~ 50 bytes (counter + timestamp)
1M users : ~ 50 MB $\rightarrow$ Redis single node లో సులభంగా fit
Sliding log : window లో ప్రతి request timestamp $\rightarrow$ memory ఎక్కువ (trade-off)

API Design

Rate limiter middleware గా (dedicated service కాదు, in-process filter):

ప్రతి incoming request → Allow(key) bool
key = "user:123" లేదా "ip:1.2.3.4" లేదా "apikey:xyz"

Response headers (rate-limit contract):
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 42
X-RateLimit-Reset: 1699999999 (epoch when window resets)
Retry-After: 12 (429 అయినప్పుడు, seconds)

Data Model

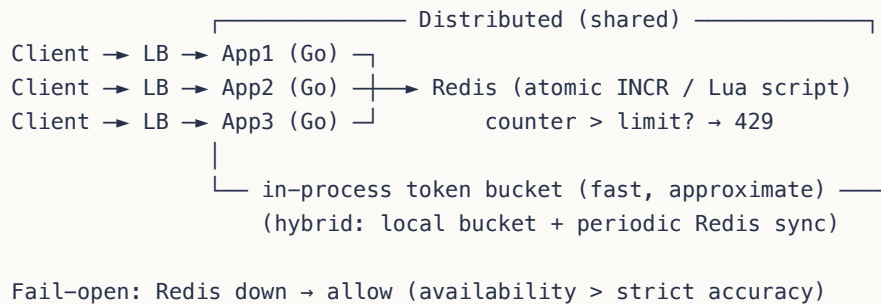
Token bucket (per key):
{ tokens: float64, lastRefill: int64(nanos) }

Sliding window log (per key):
sorted list of request timestamps within window (కచ్చితం కానీ memory)

Sliding window counter (per key):
{ currWindowCount, prevWindowCount, currWindowStart } (approx, తక్కువ memory)

Storage: in-memory (single instance) OR Redis (distributed, shared)

High-level Architecture



Deep Dive – Algorithm ఎంపిక + distributed accuracy

Algorithm	లక్షణం	Memory	Accuracy
Token Bucket	Bursts allow (bucket నిండితే), smooth refill	తక్కువ	మంచిది
Leaky Bucket	స్థిర output rate (queue), bursts smooth అవుతాయి	తక్కువ	మంచిది
Fixed Window	Simple counter/window; boundary spike సమస్య	అతి తక్కువ	తక్కువ
Sliding Window Log	ప్రతి request timestamp; కచ్చితం	ఎక్కువ	ఉత్తమం
Sliding Window Counter	Fixed + weighted prev window; boundary fix	తక్కువ	మంచిది

Fixed window boundary సమస్య: limit = 100/min. User 0:59 కి 100, 1:00 కి 100 - రెండు windows కానీ 1 సెకనులో 200 requests! Sliding window దీన్ని fix చేస్తుంది.

Distributed accuracy సమస్య: 10 app servers, ఒక్కో server local counter అయితే user 10×limit దాటగలడు.

- **Solution A (accurate):** Redis centralized. అన్ని servers Redis ని atomic గా update (Lua script - race condition లేదు). Network hop overhead.
- **Solution B (fast):** local approximate bucket + periodic sync to Redis. Slightly over-limit possible కానీ zero-latency common case.
- **Hybrid (production):** local bucket కి "share of global limit" (limit/N) allocate చేసి, periodically Redis తో rebalance.

Go Implementation – concurrency-safe token bucket + sliding window log

రెండు algorithms - production లో వాడేవి. Token bucket lazy refill (goroutine-free, on-demand compute) వాడుతుంది - ఇది idiomatic, background timer అవసరం లేదు.

```
package ratelimit

import (
    "sync"
    "time"
)

// ----- Token Bucket (lazy refill) -----
//
// Background goroutine లేదు. ప్రతి Allow() లో "ఇప్పటిదాకా ఎంత refill
// అయ్యిందో" అని lazily compute చేస్తాం. ఇది millions of keys కి scale అవుతుంది.
type TokenBucket struct {
    mu      sync.Mutex
    tokens  float64 // ప్రస్తుత tokens
    capacity float64 // గరిష్ట (burst size)
    refillRate float64 // tokens per second
    last    time.Time // చివరి refill/access time
}

func NewTokenBucket(capacity, refillPerSec float64) *TokenBucket {
    return &TokenBucket{
        tokens:    capacity, // నుండి start
        capacity:  capacity,
        refillRate: refillPerSec,
        last:      time.Now(),
    }
}

// Allow - 1 token తీసుకోగలిగితే true. Thread-safe, lazy refill.
func (b *TokenBucket) Allow() bool {
    return b.AllowN(1)
}

func (b *TokenBucket) AllowN(n float64) bool {
    b.mu.Lock()
    defer b.mu.Unlock()
```

```

    now := time.Now()
    elapsed := now.Sub(b.last).Seconds()
    b.last = now

    // gap లో ఎన్ని tokens చేరాలో add చేయి (capacity దాటకుండా)
    b.tokens += elapsed * b.refillRate
    if b.tokens > b.capacity {
        b.tokens = b.capacity
    }

    if b.tokens >= n {
        b.tokens -= n
        return true // allowed
    }
    return false // rate limited → caller 429 పంపాలి
}

// ----- Sliding Window Log (కచ్చితం) -----
//
// Window లోని ప్రతి request timestamp ని ఉంచుతాం. కచ్చితం కానీ memory ఎక్కువ.
type SlidingWindowLog struct {
    mu      sync.Mutex
    window time.Duration
    limit   int
    events []int64 // sorted request times (unix nanos)
}

func NewSlidingWindowLog(limit int, window time.Duration) *SlidingWindowLog {
    return &SlidingWindowLog{window: window, limit: limit}
}

func (s *SlidingWindowLog) Allow() bool {
    s.mu.Lock()
    defer s.mu.Unlock()

    now := time.Now().UnixNano()
    cutoff := now - s.window.Nanoseconds()

    // window దాటిన పాత events తొలగించు (events sorted కాబట్టి front నుండి)
    i := 0
    for i < len(s.events) && s.events[i] <= cutoff {
        i++
    }
    s.events = s.events[i:]

    if len(s.events) < s.limit {
        s.events = append(s.events, now)
        return true
    }
    return false // window లో limit నిండింది
}

// ----- Per-key registry (millions of users) -----
//
// ఒక్కో user కి ఒక bucket. sync.Map = read-heavy concurrent map కి మంచిది.
type Limiter struct {
    buckets sync.Map // key(string) → *TokenBucket
    capacity float64
}

```

```

    rate    float64
}

func NewLimiter(capacity, ratePerSec float64) *Limiter {
    return &Limiter{capacity: capacity, rate: ratePerSec}
}

func (l *Limiter) Allow(key string) bool {
    b, ok := l.buckets.Load(key)
    if !ok {
        // LoadOrStore: race లో ఇద్దరు create చేసినా ఒక్కటే గెలుస్తుంది
        b, _ = l.buckets.LoadOrStore(key, NewTokenBucket(l.capacity, l.rate))
    }
    return b.(*TokenBucket).Allow()
}

```

Distributed version (Redis + Lua, atomic): single-node bucket ని ఇలా Redis కి తీసుకెళ్తాం - Lua script atomic గా read-modify-write చేస్తుంది (race condition లేదు):

```

// KEYS[1]=bucketKey, ARGV[1]=capacity ARGV[2]=refillRate ARGV[3]=now
// ARGV[4]=requested
// EVALSHA తో ఒక్క network round-trip లో atomic గా జరుగుతుంది.
const tokenBucketLua = `
local b = redis.call('HMGET', KEYS[1], 'tokens', 'ts')
local tokens = tonumber(b[1]) or tonumber(ARGV[1])
local ts     = tonumber(b[2]) or tonumber(ARGV[3])
local delta  = math.max(0, tonumber(ARGV[3]) - ts) / 1000.0
tokens = math.min(tonumber(ARGV[1]), tokens + delta * tonumber(ARGV[2]))
local allowed = 0
if tokens >= tonumber(ARGV[4]) then
    tokens = tokens - tonumber(ARGV[4]); allowed = 1
end
redis.call('HMSET', KEYS[1], 'tokens', tokens, 'ts', ARGV[3])
redis.call('PEXPIRE', KEYS[1], 60000) -- idle keys ని 60s తర్వాత తొలగించు
return allowed
`

```

Go ఎందుకు perfect: millions of concurrent users $\Rightarrow$ `sync.Map` (read-heavy) + per-bucket `sync.Mutex` (fine-grained lock, ఒక user మరో user ని block చేయడు). **Lazy refill** వల్ల background goroutine/timer అవసరం లేదు - ఒక్క bucket కి goroutine అయితే millions goroutines waste. `LoadOrStore` race-safe lazy init.

Bottlenecks & Trade-offs

- **Accuracy vs latency:** centralized Redis = accurate కానీ network hop; local = fast కానీ approximate. Hybrid తో balance.
- **Redis single point** $\rightarrow$ Redis Cluster + replication; hot key కి client-side sharding.
- **Fail-open vs fail-closed:** Redis down అయితే **fail-open** (allow అన్నీ - availability) most APIs కి; payment/security కి **fail-closed** (block).

- **Memory:** sliding window log కచ్చితం కాన high-traffic key కి memory పెరుగుతుంది → sliding window counter (approx, తక్కువ memory).
- **Idle keys:** millions of keys కూడబడతాయి → TTL/eviction (Redis `PEXPIRE`, in-memory cleanup goroutine).

Key Points

- **Token bucket** = bursts allow + smooth rate; **lazy refill** (goroutine లేదు) = production idiomatic Go.
- **Fixed window boundary spike** ⇒ sliding window (log = accurate, counter = cheap approx).
- Distributed accuracy: **Redis + Lua (atomic)** for strict; local bucket for speed; hybrid balances.
- **Fail-open** (availability) vs **fail-closed** (safety) - system బట్టి ఎంచుకో.
- Go: `sync.Map` + per-bucket mutex + `LoadOrStore` = millions of users, zero background goroutines.

6. Chat System (WhatsApp / Messenger)

Real-life Analogy: పోస్ట్ ఆఫీస్ + ఎప్పుడూ తెరిచిన ఫోన్ లైన్. ప్రతి user కి ఒక persistent line (WebSocket) postman (server) దగ్గర ఉంటుంది. Message వస్తే postman వెంటనే recipient line కి deliver చేస్తాడు (online అయితే). Recipient offline అయితే postman message ని inbox (queue/DB) లో పెట్టి, online అయ్యాక ఇస్తాడు. "Delivered ✓✓" = postman confirmation.

Requirements (Functional + Non-functional)

Functional:

- 1:1 messaging + group chats
- **Delivery + read receipts** (sent ✓, delivered ✓✓, read ✓✓ blue)
- **Presence** (online/last seen/typing...)
- Offline delivery (recipient online అయ్యాక)
- **Message ordering** guarantee (per conversation)

Non-functional:

- **Low latency** delivery (< 500ms feel-instant)
- **Massive concurrent connections** - 100M+ persistent WebSockets
- **Reliability** - message ఎప్పుడూ lost కాకూడదు (at-least-once)
- **Ordering** consistency per conversation

Estimation (QPS, storage)

Users : 500M DAU, ఒక్కో user 40 messages/day
Messages/day : $500M \times 40 = 20B/day \rightarrow 20B/86400 \approx 231K \text{ messages/sec}$ (peak ~700K)
Connections : 100M concurrent WebSockets (persistent)
→ ఒక్కో server ~100K-1M connections (Go goroutines cheap)
→ 100M / 500K per box = 200 chat servers
Storage/msg : ~300 bytes $\rightarrow 20B \times 300B = 6 \text{ TB/day} \rightarrow$ sharded, TTL/archival

Design driver: 100M **persistent connections** $\Rightarrow$ Go goroutines (few KB stack each).
Fan-out (group) + presence $\Rightarrow$ channels/hub. Ordering $\Rightarrow$ per-conversation sequence.

API Design

WebSocket: `wss://chat.app/ws?token=<jwt>`
Client $\rightarrow$ Server frames:
{ "type": "send", "to": "u_2", "convId": "c_9", "clientMsgId": "uuid", "body": "..."}
{ "type": "read", "convId": "c_9", "upto": 4412 }
{ "type": "typing", "convId": "c_9" }
Server $\rightarrow$ Client frames:
{ "type": "message", "convId": "c_9", "seq": 4413, "from": "u_1", "body": "..."}
{ "type": "ack", "clientMsgId": "uuid", "seq": 4413 } (sent ✓)
{ "type": "receipt", "convId": "c_9", "seq": 4413, "state": "delivered|read" }
{ "type": "presence", "userId": "u_2", "state": "online|offline", "lastSeen": ... }
REST (history): GET /conversations/{id}/messages?before=<seq>&limit=50

Data Model

messages (shard key = convId; per-conversation ordering)

convId	seq	senderId	body	sentAt	clientMsgId
(seq = per-conv monotonic)					

PK = (convId, seq) $\rightarrow$ ఒకే conversation పంక్తులు కలిసి, sorted

conversations: convId, type(1:1/group), memberIds[], lastSeq

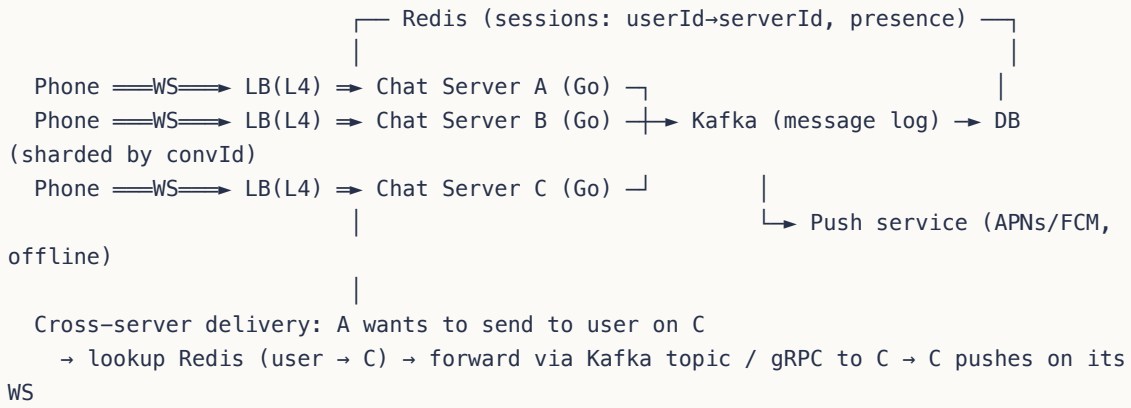
receipts: convId, userId, lastDeliveredSeq, lastReadSeq

sessions: userId $\rightarrow$ connectedServerId (ఏ chat server లో ఉన్నాడు) [Redis]

outbox (offline): userId $\rightarrow$ pending message ids [queue/DB]

Ordering trick: ఒక్కో conversation కి **monotonic seq** (DB sequence / Redis INCR convId). Clients seq తో sort చేస్తారు $\Rightarrow$ network out-of-order వచ్చినా correct order. Global ordering అవసరం లేదు (per-conversation చాలు).

High-level Architecture



శీలకం: recipient వేరే chat server లో connect అయ్యింటే? Sender's server Redis లో "ఏ server?" చూసి, ఆ server కి message forward చేస్తుంది (Kafka topic per server, లేదా direct gRPC). ఆ server తన local WS hub ద్వారా push చేస్తుంది.

Deep Dive – WebSocket hub (the hardest, Go-perfect component)

ఒక్కో chat server కి లక్షల persistent connections. వీటిని ఎలా manage చేయాలి?

- ఒక్కో connection కి **2 goroutines**: `readPump` (client→server) + `writePump` (server→client). WebSocket concurrent write allowed కాదు కాబట్టి **ఒకే writer goroutine** ఒక్కో connection కి, buffered `send` channel ద్వారా.
- **Hub goroutine** (single owner) `clients` map ని manage - register/unregister/broadcast. **No mutex** (§3 channel hub pattern).
- **Slow consumer సమస్య**: client నెమ్మదిగా read చేస్తే server memory పెరుగుతుంది. Solution: buffered `send` channel; full అయితే **connection ని drop** (slow client మొత్తం server ని కూలదీయకూడదు = backpressure).

Go Implementation – WebSocket hub (register / unregister / broadcast)

ఇది system గుండె. `gorilla/websocket` వాడుతున్నట్లు. Hub single-goroutine state owner - lock లేదు.

```
package chat

import (
    "encoding/json"
    "time"

    "github.com/gorilla/websocket"
)

// Client – ఒక persistent WebSocket connection.
type Client struct {
    userID string
    conn   *websocket.Conn
    send   chan []byte // buffered outbound queue (writePump ఇక్కడనుండి తీస్తుంది)
    hub    *Hub
}
```

```

}

// Hub - ఒక chat server లోని అన్ని connections ని own చేసే single goroutine.
// clients map ని Hub.run() మాత్రమే touch చేస్తుంది → NO mutex.
type Hub struct {
    clients    map[string]*Client // userID → client (ఒకే device అనుకుందాం)
    register   chan *Client
    unregister chan *Client
    deliver    chan Envelope // ఈ server లోని user కి message deliver చేయాలి
}

type Envelope struct {
    ToUser string
    Data   []byte
}

func NewHub() *Hub {
    return &Hub{
        clients:    make(map[string]*Client),
        register:   make(chan *Client),
        unregister: make(chan *Client),
        deliver:    make(chan Envelope, 1024),
    }
}

// run - THE single owner goroutine. అన్ని state mutations ఇక్కడే → race లేదు.
func (h *Hub) run() {
    for {
        select {
        case c := <-h.register:
            h.clients[c.userID] = c
            // presence: Redis లో userID → thisServer set చేయవచ్చు ఇక్కడ

        case c := <-h.unregister:
            if cur, ok := h.clients[c.userID]; ok && cur == c {
                delete(h.clients, c.userID)
                close(c.send) // writePump ని ఆపు
            }

        case env := <-h.deliver:
            if c, ok := h.clients[env.ToUser]; ok {
                select {
                case c.send <- env.Data: // fast path: queue లో పెట్టు
                default:
                    // send buffer FULL = slow consumer → drop
                    connection (backpressure)
                    close(c.send)
                    delete(h.clients, c.userID)
                }
            }
            // user ఈ server లో లేకపోతే: caller ముందే Redis lookup చేసి
            // సరైన server కి forward చేసుండాలి (cross-server delivery).
        }
    }
}

// readPump - client నుండి frames చదివి process చేస్తుంది. ఒక్కో conn కి ఒకటి.
func (c *Client) readPump() {
    defer func() {

```

```

        c.hub.unregister <- c
        _ = c.conn.Close()
    }()
    c.conn.SetReadLimit(4096)
    for {
        _, raw, err := c.conn.ReadMessage()
        if err != nil {
            return // connection closed/error → cleanup (defer)
        }
        var in struct {
            Type, To, ConvID, Body, ClientMsgID string
        }
        if json.Unmarshal(raw, &in) != nil {
            continue
        }
        if in.Type == "send" {
            // 1) Kafka కి persist (at-least-once, per-conv seq assign)
            // 2) recipient ఈ server లో ఉంటే direct deliver; లేకపోతే Redis
            lookup → forward
            c.hub.deliver <- Envelope{ToUser: in.To, Data: raw}
        }
    }
}

// writePump – send channel నుండి తీసి WS కి వ్రాస్తుంది. ఒక్క conn కి ఒకటే
// writer (WebSocket concurrent write allowed కాదు). ping తో liveness check.
func (c *Client) writePump() {
    ticker := time.NewTicker(30 * time.Second) // heartbeat ping
    defer func() { ticker.Stop(); _ = c.conn.Close() }()
    for {
        select {
        case msg, ok := <-c.send:
            if !ok { // hub channel ని close చేసింది → connection ముగింపు
                _ = c.conn.WriteMessage(websocket.CloseMessage, nil)
                return
            }
            if c.conn.WriteMessage(websocket.TextMessage, msg) != nil {
                return
            }
        case <-ticker.C:
            if c.conn.WriteMessage(websocket.PingMessage, nil) != nil {
                return // pong రాలేదు/write fail → dead connection
            }
        }
    }
}

// ServeWS – HTTP upgrade → 2 goroutines start (read + write).
func (h *Hub) ServeWS(conn *websocket.Conn, userID string) {
    c := &Client{userID: userID, conn: conn, send: make(chan []byte, 256), hub: h}
    h.register <- c
    go c.writePump() // server → client
    go c.readPump() // client → server (ఈ goroutine లో run అవుతుంది)
}

```

Go ఎందుకు ఇక్కడ అజేయం: 1M connections = 2M goroutines, ఒక్కోటి ~4KB stack ⇒ ~8GB RAM - ఒక్క machine లో సాధ్యం (Java threads అయితే అసాధ్యం). **Hub single-goroutine** ⇒ `clients` map కి lock అవసరం లేదు, race లేదు. **Buffered send + default drop** = backpressure (slow client మొత్తం server ని కూలదీయదు). `select` తో ping/heartbeat cleanly integrate.

Bottlenecks & Trade-offs

- **Connection storm on restart:** server restart అయితే లక్షల reconnects ఒకసారి. Solution: **jittered backoff** on client, gradual rollout, connection draining.
- **Cross-server fan-out:** group chat, members వేర్వేరు servers లో. Kafka topic per server, లేదా pub/sub (Redis/NATS) ద్వారా route. Global routing table (Redis) ని fresh గా ఉంచాలి.
- **Ordering vs availability:** strict per-conv order ⇒ single writer per conv (Kafka partition by convId). Trade-off: hot conversation (huge group) = hot partition.
- **Offline delivery:** recipient offline ⇒ persist to outbox + **push notification** (APNs/FCM). Online అయ్యాక sync.
- **Presence at scale:** ప్రతి presence change ని friends అందరికీ broadcast = fan-out storm. Solution: **pull on demand** + throttle, "last seen" eventual.

Key Points

- **2 goroutines/connection** (read + write pump); **single writer** per WS (concurrent write illegal).
- **Hub = single-owner goroutine** ⇒ clients map కి no mutex (§3 channel hub).
- **Backpressure:** buffered `send` + `default: drop` ⇒ slow client isolated.
- **Ordering:** per-conversation `seq` (Kafka partition by convId), global order అవసరం లేదు.
- **Cross-server delivery:** Redis session table (user→server) + Kafka/pub-sub forwarding.
- Go goroutines ⇒ millions of persistent connections per fleet - ఇదే Go killer use-case.

7. News Feed (Twitter / Instagram)

Real-life Analogy: వార్తాపత్రిక delivery. రెండు పద్ధతులు: (1) **Fan-out on write** = ప్రతి writer వార్త రాయగానే అందరి subscribers ఇళ్ళకు వెంటనే copy వేయడం (feed ముందే ready, చదవడం fast, కానీ writer కి పని ఎక్కువ). (2) **Fan-out on read** = subscriber అడిగినప్పుడే అన్ని writers నుండి తాజా వార్తలు collect చేయడం (write తేలిక, కానీ read నెమ్మది). Celebrity (10M followers) కి పద్ధతి-1 అయితే ఒక్క tweet కి 10M copies - అందుకే hybrid.

Requirements (Functional + Non-functional)

Functional:

- User posts (tweet/photo) create చేయడం

- Followers timeline (home feed) - followed users posts, reverse-chron లేదా ranked
- Like/comment/retweet
- **Ranking** (pure chronological కాదు, relevance)

Non-functional:

- **Low read latency** - feed instant గా load అవ్వాలి (< 200ms)
- **Read-heavy** - feed reads >> posts (100:1+)
- Eventual consistency OK (కొన్ని seconds delay tolerable)
- **Celebrity problem** handle చేయాలి

Estimation (QPS, storage)

```

Users          : 300M DAU
Posts          : ఒక్కో user 0.2 posts/day → 60M posts/day → ~700 posts/sec
Feed reads     : ఒక్కో user 10 feed loads/day → 3B reads/day → ~35K reads/sec (peak ~100K)
Avg followers: ~200 (median తక్కువ, celebrities millions)
Fan-out write: 700 posts/sec × 200 followers = 140K feed writes/sec (avg)
               → celebrity ఒక్కో post = millions writes → hybrid తప్పనిసరి
Feed cache     : ఒక్కో user ~800 recent post-ids × 8 bytes = 6.4KB → 300M users = ~2 TB
               (Redis cluster)
  
```

API Design

```

POST /api/v1/posts          { "body": "...", "mediaUrl": "?" } → 201 { postId }
GET  /api/v1/feed?cursor=<c>&limit=20 → { posts: [...], nextCursor }
POST /api/v1/follow/{userId}
POST /api/v1/posts/{id}/like

Feed response = hydrated posts (author, body, media, counts) - post-ids feed cache
నుండి,
               content batch-fetch from post store.
  
```

Data Model

```

posts:      postId, authorId, body, mediaUrl, createdAt, likeCount
follows:    followerId, followeeId          (who follows whom; 2 indexes)
feed cache: userId → [postId, postId, ...] (Redis sorted set, score =
timestamp/rank)
           → precomputed timeline (fan-out on write పద్ధతిలో)

celebrity_posts: authorId → recent postIds (fan-out on read, celebrities కి)
  
```

High-level Architecture

```

Post write:
  Client → Post Service (Go) → Post DB (sharded by postId)
                                   |
                                   └→ Fan-out Service (Go worker pool)
  
```

```

└─ normal author: follower list → ప్రతి follower feed cache
    $ push
        └─ celebrity (>threshold): SKIP fan-out (mark; read-time
            merge)

    Feed read:
        Client → Feed Service (Go) → Redis feed cache (precomputed post-ids) [fan-
        out-on-write part]
        |
        | + celebrity posts (fetched live, merged) [fan-
        out-on-read part]
        └─ hydrate (batch fetch post content) → rank → return
  
```

Deep Dive – Fan-out on write vs read + celebrity problem

అంశం	Fan-out on WRITE (push)	Fan-out on READ (pull)
Feed compute	Post time (ముందే)	Read time (అప్పుడే)
Read latency	Fast (precomputed)	Slow (live merge)
Write cost	High (N followers = N writes)	Low (1 write)
Celebrity	సమస్య (10M writes/post)	మంచిది (1 write)
Best for	సాధారణ users (few followers)	Celebrities (millions followers)

Hybrid (production - Twitter approach):

- **సాధారణ author** post ⇒ fan-out on write (followers feed caches \$ push). Read fast.
- **Celebrity** (followers > threshold, ఉదా 100K) post ⇒ **fan-out skip**. Read time లో follower feed = (precomputed cache) **merge** (celebrity posts fetched live). Celebrity post 10M writes కాదు.
- **Inactive followers:** dormant users \$ fan-out skip (వాళ్ళు రారు) - "fan-out to active only".

Ranking: pure reverse-chron కాకుండా score = f(recency, affinity, engagement, media). ML model లేదా heuristic. Feed cache లో sorted set score = rank.

Go Implementation – fan-out worker pool

Post వచ్చినప్పుడు followers అందరి feed caches ని update చేయాలి - millions కావచ్చు. Bounded worker pool (§3 toolkit) తో, celebrity skip logic తో.

```

package feed

import (
    "context"
    "sync"
    "time"
)

type FeedCache interface {
    // PushToTimeline – user timeline (sorted set) లో postID ని score తో చేర్చు.
    PushToTimeline(ctx context.Context, userID, postID string, score int64) error
}
  
```

```

type FollowGraph interface {
    // ActiveFollowers - batched n active followers ని stream చేస్తుంది.
    ActiveFollowers(ctx context.Context, authorID string) (<-chan []string,
error)
    FollowerCount(ctx context.Context, authorID string) (int, error)
}

type FanoutService struct {
    cache          FeedCache
    graph          FollowGraph
    workers        int
    celebThreshold int // ఈ threshold దాటితే fan-out skip (read-time merge)
}

func NewFanoutService(c FeedCache, g FollowGraph) *FanoutService {
    return &FanoutService{cache: c, graph: g, workers: 100, celebThreshold:
100_000}
}

// Fanout - post వచ్చినప్పుడు call అవుతుంది. Celebrity అయితే skip.
func (s *FanoutService) Fanout(ctx context.Context, authorID, postID string) error {
    count, err := s.graph.FollowerCount(ctx, authorID)
    if err != nil {
        return err
    }
    if count > s.celebThreshold {
        // CELEBRITY: fan-out skip. Read time లో ఈ author posts merge
అవుతాయి.
        // (author's own "celebrity timeline" లో మాత్రమే వ్రాస్తాం - 0(1))
        return s.cache.PushToTimeline(ctx, "celeb:"+authorID, postID,
time.Now().Unix())
    }

    // NORMAL author: bounded worker pool తో followers అందరి feed కి push.
    batches, err := s.graph.ActiveFollowers(ctx, authorID)
    if err != nil {
        return err
    }

    jobs := make(chan string, s.workers*2) // follower userIDs
    var wg sync.WaitGroup
    score := time.Now().Unix()

    // N workers start
    for w := 0; w < s.workers; w++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            for followerID := range jobs {
                // per-follower failure మొత్తం fan-out ని అపకూడదు (log
+ continue)
                _ = s.cache.PushToTimeline(ctx, followerID, postID,
score)
            }
        }()
    }

    // follower batches ని jobs channel లోకి feed చేయి

```



```

        for batch := range batches {
            for _, followerID := range batch {
                select {
                    case jobs <- followerID:
                    case <-ctx.Done():
                        close(jobs)
                        wg.Wait()
                        return ctx.Err()
                }
            }
        }
        close(jobs) // ఇక followers లేరు → workers exit
        wg.Wait()
        return nil
    }

    // ---- Read path: merge precomputed feed + celebrity posts ----

    type FeedReader struct {
        cache FeedCache
        graph FollowGraph
        // timeline reader, celebrity list reader ఇక్కడ inject అవుతాయి
    }

    // BuildFeed – precomputed timeline + followed celebrities' recent posts merge.
    // ఇది fan-out-on-write (fast) + fan-out-on-read (celebrities) hybrid.
    func (r *FeedReader) BuildFeed(ctx context.Context, userID string, limit int)
    []string {
        // 1) precomputed timeline (fan-out-on-write భాగం) – O(limit), Redis ZREVRANGE
        timeline := r.readTimeline(ctx, userID, limit)

        // 2) followed celebrities recent posts (fan-out-on-read భాగం) – live fetch +
        merge
        celebs := r.followedCelebrities(ctx, userID)
        for _, celebID := range celebs {
            timeline = append(timeline, r.readTimeline(ctx, "celeb:"+celebID,
            limit)...)
        }

        // 3) rank/merge by score, top-`limit` (heap లేదా sort)
        return mergeTopK(timeline, limit)
    }

    // (helpers – skeleton)
    func (r *FeedReader) readTimeline(ctx context.Context, key string, n int) []string {
        return nil
    }
    func (r *FeedReader) followedCelebrities(ctx context.Context, userID string) []string {
        return nil
    }
    func mergeTopK(ids []string, k int) []string {
        return ids
    }

```

Go ఎందుకు perfect: fan-out = embarrassingly parallel (N independent cache writes) ⇒ **worker pool** (§3) ఖచ్చితంగా సరిపోతుంది. Bounded workers ⇒ Redis/DB ని overwhelm చేయవు (100 concurrent writes, million goroutines కాదు). context cancellation ⇒ timeout/shutdown అయితే fan-out clean గా ఆగుతుంది. Per-follower failure isolated (ఒక్కటి fail అయినా మిగతా followers కి delivery ఆగదు).

Bottlenecks & Trade-offs

- **Celebrity write amplification:** ఒక్క tweet = 10M feed writes. Hybrid (skip fan-out, read-time merge) తో solve. Trade-off: celebrity followers కి read కొంచెం slow.
- **Fan-out lag:** post చేసాక followers కి కనిపించడానికి కొన్ని seconds. Eventual consistency accept (feed కి fine).
- **Feed cache memory:** 300M users × timeline = TB scale. Solution: **active users కి మాత్రమే** cache, inactive కి on-demand rebuild; TTL.
- **Thundering herd on hot post:** viral post hydration ⇒ post store hammered. Post content ని aggressive cache + CDN for media.
- **Ranking cost:** ML ranking read path లో ఖరీదు. Pre-rank at write, re-rank top-N at read.

Key Points

- **Fan-out on write** = fast read, costly write; **fan-out on read** = cheap write, slow read.
- **Hybrid** = normal users push, **celebrities pull (read-time merge)** ⇒ celebrity problem solved.
- Fan-out to **active followers only**; inactive on-demand.
- Go **worker pool** (bounded) = ideal for fan-out; context for cancellation, per-follower failure isolation.
- Feed cache = Redis sorted set (score = rank/timestamp); eventual consistency accepted.

8. Notification System (Push / SMS / Email)

Real-life Analogy: పెళ్ళి పిలుపులు పంపే system. ఒక్క అతిథికి ఏ మాధ్యమం నచ్చుతుందో అలా పంపుతావు - కొందరికి ఫోన్ (push), కొందరికి SMS, కొందరికి పోస్ట్ (email). ఒకే అతిథికి రెండుసార్లు పంపకూడదు (dedup). పంపలేకపోతే మళ్ళీ ప్రయత్నించు (retry), కానీ 100 సార్లు కాదు (limit). Template = ఒకే invitation design, పేరు మాత్రం మారుస్తావు.

Requirements (Functional + Non-functional)

Functional:

- Multi-channel: **push** (APNs/FCM), **SMS** (Twilio), **email** (SES/SendGrid), in-app
- **Templates** (personalization: name, order id...)
- **User preferences** (channel opt-in/out, quiet hours, frequency caps)

- **Scheduling** (send later, timezone-aware)

Non-functional:

- **Reliability** - at-least-once (notification lost కాకూడదు)
- **Dedup** - same event $\Rightarrow$ ఒక్కసారి (retries duplicate పంపకూడదు)
- **High throughput** - millions/min (flash sale, breaking news)
- **Rate limits** per provider (Twilio/APNs limits గౌరవించాలి)

Estimation (QPS, storage)

Notifications: 500M/day $\rightarrow$ ~5,800/sec average, peak (broadcast) ~500K/sec (burst)
 Channels split: push 70%, email 20%, SMS 10%
 Providers rate: APNs ~fast, Twilio ~100/sec/number (bottleneck!) $\rightarrow$ SMS ని smooth చేయాలి
 Dedup store : event dedup keys, TTL 24h $\rightarrow$ millions keys $\rightarrow$ Redis
 Retry queue : failed $\times$ avg 2 retries $\rightarrow$ queue depth planning

API Design

```
POST /api/v1/notify
{ "userId":"u_1", "templateId":"order_shipped",
  "data":{"orderId":"0123","eta":"Tue"}, "channels":["push","email"],
  "idempotencyKey":"evt_9f3a" }      ← dedup కీలకం
→ 202 Accepted { notificationId }
```

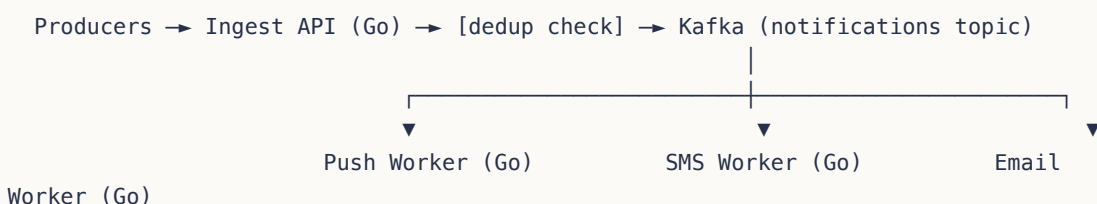
GET /api/v1/notifications/{id} $\rightarrow$ status per channel (queued/sent/delivered/failed)

Idempotency key = producer supplied unique event id. అదే key మళ్ళీ వస్తే (retry, duplicate publish) $\Rightarrow$ system re-send చేయదు. ఇది exactly-once effect ఇస్తుంది (delivery at-least-once అయినా).

Data Model

notifications: id, userId, templateId, channel, status, attempts, idempotencyKey, createdAt
 templates: templateId, channel, subject, bodyTemplate (mustache/text-template)
 preferences: userId, channel, enabled, quietHours, freqCap
 dedup: idempotencyKey $\rightarrow$ notificationId (Redis, TTL 24-48h)
 DLQ: permanently-failed notifications (manual/alert)

High-level Architecture





Deep Dive – Pipeline: dedup + preferences + retry + provider rate limits

Stages (pipeline):

1. **Dedup** - idempotencyKey Redis `SET NX` . Already ఉంటే drop (duplicate).
2. **Preferences** - user opt-out? quiet hours? freq cap దాటిందా? ⇒ suppress.
3. **Template render** - `text/template` తో personalize.
4. **Channel dispatch** - provider కి పంపు.
5. **Retry** - transient failure (5xx, timeout) ⇒ exponential backoff + jitter, max attempts.
Permanent (invalid number) ⇒ DLQ నేరుగా.
6. **Provider rate limit** - Twilio 100/sec ⇒ token bucket (§5) తో smooth చేయి.

Retry classification (కీలకం): అన్ని failures retry చేయకూడదు.

- **Retryable:** timeout, 429, 5xx, network. → backoff retry.
- **Non-retryable:** 400 (bad request), invalid token/number, unsubscribed. → DLQ, retry వృథా.

Exactly-once effect: delivery at-least-once (retries). కానీ dedup + provider idempotency key ⇒ user ఒక్కసారే చూస్తాడు.

Go Implementation – pipeline with retry + idempotency

Consumer worker: dedup → preferences → render → send with backoff retry. `context` deadline, classified retry.

```

package notify

import (
    "context"
    "errors"
    "math"
    "math/rand"
    "text/template"
    "time"
)

type Notification struct {
    ID            string
    UserID        string
    TemplateID    string
    Channel       string
    Data          map[string]string
    IdempotencyKey string
    Attempt       int
}
  
```

```

}

// sentinel errors – retry classification
var (
    ErrRetryable    = errors.New("retryable")      // 5xx/timeout/429
    ErrPermanent    = errors.New("permanent")      // 400/invalid → DLQ
    ErrDuplicate     = errors.New("duplicate")      // dedup hit
    ErrSuppressed    = errors.New("suppressed")     // prefs/quiet hours
)

type DedupStore interface {
    // FirstSeen – key మొదటిసారి అయితే true (SET NX). Duplicate అయితే false.
    FirstSeen(ctx context.Context, key string, ttl time.Duration) (bool, error)
}

type Preferences interface {
    Allowed(ctx context.Context, userID, channel string) (bool, error)
}

type Provider interface {
    Send(ctx context.Context, to, subject, body string) error //
    ErrRetryable/ErrPermanent తిరిగి ఇస్తుంది
}

type Worker struct {
    dedup    DedupStore
    prefs    Preferences
    providers map[string]Provider          // channel → provider
    templates map[string]*template.Template // templateId → parsed template
    limiter   func(channel string) bool           // $5 token bucket per provider
    maxAttempt int
    deadLetter func(context.Context, Notification, error)
}

// Process – ఒక్క notification ని pipeline లో నడుపు.
func (w *Worker) Process(ctx context.Context, n Notification) error {
    // STAGE 1: dedup (idempotency) – ఇదే exactly-once effect ఇచ్చేది
    first, err := w.dedup.FirstSeen(ctx, n.IdempotencyKey, 48*time.Hour)
    if err != nil {
        return err // dedup store down → retry చేయనివ్వ (safer: skip send)
    }
    if !first {
        return ErrDuplicate // ఇదివరకే process అయింది → silently drop
    }

    // STAGE 2: preferences (opt-out, quiet hours, freq cap)
    ok, err := w.prefs.Allowed(ctx, n.UserID, n.Channel)
    if err != nil {
        return err
    }
    if !ok {
        return ErrSuppressed
    }

    // STAGE 3: template render
    tmpl, ok := w.templates[n.TemplateID]
    if !ok {
        return ErrPermanent // template లేదు → retry వృథా
    }
    body := renderTemplate(tmpl, n.Data)

```

```

// STAGE 4+5: dispatch with classified retry + provider rate limit
return w.sendWithRetry(ctx, n, body)
}

func (w *Worker) sendWithRetry(ctx context.Context, n Notification, body string)
error {
    prov := w.providers[n.Channel]
    for attempt := 0; attempt < w.maxAttempt; attempt++ {
        // provider rate limit (Twilio 100/sec మొ.) – smooth చేయ
        for !w.limiter(n.Channel) {
            select {
            case <-time.After(20 * time.Millisecond):
            case <-ctx.Done():
                return ctx.Err()
            }
        }

        err := prov.Send(ctx, n.UserID, "", body)
        switch {
        case err == nil:
            return nil // ✓ delivered
        case errors.Is(err, ErrPermanent):
            w.deadLetter(ctx, n, err) // retry వృథా → DLQ నేరుగా
            return ErrPermanent
        case errors.Is(err, ErrRetryable):
            // exponential backoff + jitter (retry storm ని ఆపడానికి jitter
            కీలకం)
            backoff := time.Duration(math.Pow(2, float64(attempt))) * 100
            * time.Millisecond

            jitter := time.Duration(rand.Int63n(int64(backoff / 2)))
            select {
            case <-time.After(backoff + jitter):
            case <-ctx.Done():
                return ctx.Err()
            }

            default:
                w.deadLetter(ctx, n, err)
                return err
            }
        }

        // max attempts దాటింది → DLQ (alert)
        err := errors.New("max attempts exhausted")
        w.deadLetter(ctx, n, err)
        return err
    }

    func renderTemplate(t *template.Template, data map[string]string) string {
        // (skeleton) t.Execute(buf, data) → personalized body
        return ""
    }
}

```

Go ఎందుకు perfect: ఒక్క channel = వేరే Kafka consumer group + goroutine worker pool ⇒ push/SMS/email independent గా scale. **context** deadline ⇒ hung provider call ని cancel. **errors.Is** sentinel classification ⇒ retryable vs permanent clean గా. **Jitter** (rand) retry storm ని విడగొడుతుంది. **text/template** native personalization.

Bottlenecks & Trade-offs

- **Provider rate limits:** SMS provider 100/sec ⇒ ముందు fast produce అయినా slow drain. Token bucket smoothing + per-provider queue depth monitoring.
- **Retry storm:** provider down అయితే అందరూ ఒకేసారి retry ⇒ **jitter + backoff + circuit breaker** (provider down అయితే fast-fail, queue పేరుకోకుండా).
- **Dedup store dependency:** Redis down అయితే? Fail-closed (send ఆపు, duplicate risk avoid) - ఇక్కడ safety > availability.
- **At-least-once ⇒ duplicates possible:** dedup + provider idempotency key తో user-visible exactly-once.
- **Broadcast spike:** breaking news 500K/sec ⇒ Kafka buffers spike, workers steady drain (queue = shock absorber).

Key Points

- **Pipeline:** dedup → preferences → render → dispatch → classified retry → DLQ.
- **Idempotency key + dedup store** ⇒ at-least-once delivery, exactly-once effect.
- **Retry classification:** retryable (5xx/timeout) vs permanent (400/invalid) - permanent ⇒ DLQ నేరుగా.
- **Backoff + jitter** = retry storm నివారణ; **circuit breaker** for down providers.
- **Queue (Kafka) = shock absorber;** per-channel workers + token bucket for provider limits.
- Go: per-channel consumer groups, `context` deadlines, `errors.Is` classification.

9. Typeahead / Autocomplete (Search Suggestions)

Real-life Analogy: డిక్షనరీలో పదం వెతకడం. "cat" అని రాయడం మొదలుపెడితే, "ca" దగ్గరే డిక్షనరీ ఆ page తెరిచి "cat, car, care, cattle..." అని చూపిస్తుంది. Trie = ఆ డిక్షనరీ structure - ప్రతి అక్షరం ఒక మెట్టు, prefix ఒకటే ఉన్న పదాలు కలిసి ఉంటాయి. Top-k = అందులో అత్యంత popular వాటిని ముందు చూపడం (అందరూ "cat" ఎక్కువ వెతుకుతారు కాబట్టి అది పైన).

Requirements (Functional + Non-functional)

Functional:

- Prefix ఇస్తే **top-k suggestions** (popularity ranked)
- ప్రతి keystroke కి fast response
- Fresh terms (trending) కొన్ని hours లో కనిపించాలి

Non-functional:

- **Ultra low latency** - < 50ms (ప్రతి keystroke; feel-instant)
- **High QPS** - ప్రతి user typing = అనేక requests
- Read-heavy (queries >> updates)
- Fault tolerant, approximate ranking OK

Estimation (QPS, storage)

Searches : 5B searches/day → typing = ప్రతి search కి ~5 keystrokes → 25B prefix QPS/day
→ ~290K prefix-QPS/sec (peak ~1M) → caching + in-memory trie తప్పనిసరి
Vocabulary : ~100M unique terms/phrases
Trie memory : 100M terms × ~30 bytes avg → కొన్ని GB → in-memory (sharded by prefix)
Top-k precompute: ప్రతి node కి top-k (ఉదా k=10) cached → node స్వేదం = $O(1)$
Updates : batch (query logs → hourly aggregate → trie rebuild/merge)

API Design

```
GET /api/v1/suggest?q=ca&limit=10
→ { "suggestions": [
    {"term": "cat videos", "score": 98213},
    {"term": "career", "score": 45120}, ... ] }
```

Internal: query logs → aggregation (Spark/Flink) → weighted terms → trie build → serving nodes

Data Model

Trie node:
children: map[rune]*Node
isWord: bool
topK: []Suggestion ← ఈ node కింద top-k terms PRECOMPUTED (query fast)

Suggestion: { term string, score int }

Serving: prefix మొదటి 1-2 chars బట్టి sharded tries (memory distribute)

Source: aggregated (term → frequency) offline pipeline నుండి

కీలక optimization: ప్రతి trie node లో **top-k ముందే compute చేసి cache** చేయి. అప్పుడు query = prefix దాకా walk ($O(\text{len})$) + ఆ node topK return ($O(1)$). Query time లో subtree అంతా scan చేయకూడదు.

High-level Architecture

OFFLINE (build):
Query logs → Aggregator (Flink/Spark: term → weighted count) → Trie Builder →
→ immutable trie snapshots → push to serving nodes (versioned)

ONLINE (serve):
Client → LB → Suggest Service (Go, in-memory trie) → top-k ($O(1)$ per node)
| cache layer (hot prefixes) + CDN edge for popular
↳ shard by prefix[0:1] across serving nodes

Build/serve separation: trie ని offline build చేసి **immutable snapshot** నా serving nodes కి push. Serving = read-only trie (lock-free reads). కొత్త snapshot atomic pointer swap.

Deep Dive – Concurrent trie + top-k precompute

సవాళ్ళు:

1. **Read concurrency:** లక్షల reads/sec ఒకే trie మీద. Read-only immutable trie ⇒ **lock లేకుండా** concurrent reads (Go).
2. **Top-k at each node:** node కింద subtree scan అయితే slow. Build time లో ప్రతి node topK compute (bottom-up merge).
3. **Updates:** live mutate చేస్తే readers race. Solution: **copy-on-write** - కొత్త trie build, atomic pointer swap (readers పాత దాన్ని finish చేస్తారు, కొత్తవి కొత్త దాన్ని చూస్తాయి).

Top-k precompute (build): leaf నుండి పైకి - ఒక్కో node topK = children topK లు + self merge → top k. Min-heap of size k.

Go Implementation – concurrent trie + top-k

Read-optimized: immutable serving trie + copy-on-write swap. Build time top-k precompute తో query $O(\text{prefix length})$.

```
package typeahead

import (
    "container/heap"
    "sort"
    "sync/atomic"
)

type Suggestion struct {
    Term string
    Score int
}

type node struct {
    children map[rune]*node
    topK     []Suggestion // ఈ node subtree లోని top-k (BUILD time precomputed)
}

// Trie – atomic.Pointer వల్ల readers lock లేకుండా చదువుతారు;
// update = కొత్త root build → atomic swap (copy-on-write).
type Trie struct {
    root atomic.Pointer[node]
    k     int
}

func New(k int) *Trie {
    t := &Trie{k: k}
    t.root.Store(&node{children: map[rune]*node{}})
    return t
}

// Build – (term→score) map నుండి కొత్త trie build చేసి atomic swap.
// ఇది offline/periodic గా జరుగుతుంది; readers ఏమాత్రం block కాదు.
func (t *Trie) Build(terms map[string]int) {
    root := &node{children: map[rune]*node{}}
    // 1) అన్ని terms ని insert (leaf topK లో అదే term ఉంచు)
```

```

    for term, score := range terms {
        cur := root
        for _, r := range term {
            nx, ok := cur.children[r]
            if !ok {
                nx = &node{children: map[rune]*node{}}
                cur.children[r] = nx
            }
            cur = nx
        }
        cur.topK = []Suggestion{{Term: term, Score: score}}
    }
    // 2) bottom-up top-k precompute (children topK లు merge)
    computeTopK(root, t.k)
    // 3) atomic swap - ఇక్కడనుండి కొత్త readers కొత్త trie చూస్తారు
    t.root.Store(root)
}

// computeTopK - post-order: ఒక్కో node topK = self + children topK లు merge (top-k).
func computeTopK(n *node, k int) []Suggestion {
    // children లోని అన్ని suggestions పోగు చేయి
    pool := append([]Suggestion(nil), n.topK...)
    for _, ch := range n.children {
        pool = append(pool, computeTopK(ch, k)...)
    }
    // top-k by score (min-heap of size k → O(n log k))
    n.topK = topKByScore(pool, k)
    return n.topK
}

// Suggest - LOCK-FREE read. prefix దాకా walk → ఆ node topK (O(1)).
func (t *Trie) Suggest(prefix string) []Suggestion {
    cur := t.root.Load() // atomic read: consistent snapshot
    for _, r := range prefix {
        nx, ok := cur.children[r]
        if !ok {
            return nil // ఈ prefix తో ఏ term లేదు
        }
        cur = nx
    }
    return cur.topK // ముందే compute అయింది → instant
}

// ---- top-k helper (min-heap of size k) ----
type minHeap []Suggestion

func (h minHeap) Len() int { return len(h) }
func (h minHeap) Less(i, j int) bool { return h[i].Score < h[j].Score }
func (h minHeap) Swap(i, j int) { h[i], h[j] = h[j], h[i] }
func (h *minHeap) Push(x any) { *h = append(*h, x.(Suggestion)) }
func (h *minHeap) Pop() any {
    old := *h
    n := len(old)
    v := old[n-1]
    *h = old[:n-1]
    return v
}

func topKByScore(all []Suggestion, k int) []Suggestion {

```

```

h := &minHeap{}
heap.Init(h)
for _, s := range all {
    if h.Len() < k {
        heap.Push(h, s)
    } else if (*h)[0].Score < s.Score {
        heap.Pop(h) // చిన్నదాన్ని తీసేయి
        heap.Push(h, s) // పెద్దదాన్ని పెట్టు
    }
}
out := make([]Suggestion, h.Len())
for i := len(out) - 1; i >= 0; i-- { // descending score
    out[i] = heap.Pop(h).(Suggestion)
}
sort.SliceStable(out, func(i, j int) bool { return out[i].Score >
out[j].Score })
return out
}

```

Go ఎందుకు perfect: serving trie **immutable** $\Rightarrow$ `atomic.Pointer` తో readers **lock-free** (millions of reads/sec, zero contention). Update = COW build + atomic `Store` swap - readers ఏమాత్రం block కావు (RCU-style). `container/heap` native top-k. Node-level topK precompute $\Rightarrow$ query = $O(\text{prefix length})$, subtree scan లేదు.

Bottlenecks & Trade-offs

- **Memory (100M terms):** single trie GB scale $\Rightarrow$ **shard by prefix[0]** (a-tree, b-tree... వేర్వేరు nodes). Trade-off: cross-shard query rare.
- **Freshness vs cost:** live insert = reader race $\Rightarrow$ COW rebuild (batch, hourly). Trade-off: trending term కొన్ని hours delay. Real-time needs $\rightarrow$ separate "hot terms" layer.
- **Personalization:** global trie + per-user re-rank (recent searches). Global trie shared, personalize at edge.
- **Cache:** hot prefixes ("a", "the") $\Rightarrow$ CDN/edge cache (అవి rarely మారతాయి).
- **Typos/fuzzy:** exact prefix trie only. Fuzzy కి edit-distance/n-gram layer (ఖరీదు, separate).

Key Points

- **Trie + node-level top-k precompute** $\Rightarrow$ query = $O(\text{prefix length})$, subtree scan లేదు.
- **Immutable serving trie + atomic pointer swap (COW)** $\Rightarrow$ lock-free concurrent reads (Go `atomic.Pointer`).
- **Build/serve separation:** offline aggregate $\rightarrow$ trie snapshot $\rightarrow$ push to read-only serving nodes.
- **Shard by prefix**, cache hot prefixes at edge.
- Freshness trade-off: batch rebuild (hours) vs real-time hot-term layer.

10. Video Streaming (YouTube / Netflix)

Real-life Analogy: సినిమా theatre chain + వంటగది. ఒక్కో raw video (కొత్త సినిమా reel) వచ్చాక దాన్ని వేర్వేరు qualities (240p/720p/1080p/4K) లోకి **transcode** చేస్తారు (వంట = ఒకే dish వేర్వేరు serving sizes). తర్వాత దగ్గర్లోని theatres (CDN edge) కి పంపుతారు - ప్రేక్షకుడు తన ఊర్లోని theatre (nearest edge) లో చూస్తాడు. Network నెమ్మదిగా ఉంటే చిన్న quality (adaptive bitrate) - buffering రాకుండా.

Requirements (Functional + Non-functional)

Functional:

- Video **upload** + **transcode** (multiple resolutions/bitrates)
- **Streaming playback** (adaptive bitrate - ABR)
- Thumbnails, metadata, search
- Resumable uploads (పెద్ద files)

Non-functional:

- **Massive read scale** - millions concurrent viewers (read >> write huge)
- **Low startup latency** + **no buffering** (ABR)
- **Durable storage** (videos ఎప్పటికీ lost కాకూడదు)
- Transcode throughput (upload spike handle)

Estimation (QPS, storage)

```
Uploads      : 500 hrs/min uploaded → 720K hrs/day (YouTube-scale)
Storage/hr   : 1 raw hr ~1-4 GB; transcoded (అన్ని renditions) ~2-3× → కొన్ని GB/hr
              720K hrs/day × ~5 GB = ~3.6 PB/day (blob store, tiered/cold archival)
Views        : 5B views/day → majority CDN నుండి (origin చాలా తక్కువ hit)
Bandwidth    : peak Tbps scale → CDN తప్పనిసరి (origin కి direct serve అసాధ్యం)
Transcode    : compute-heavy → job queue + autoscaled worker fleet
```

Design driver: playback bandwidth Tbps ⇒ **CDN** (origin కి 1-2% మాత్రమే వస్తుంది).
Transcode compute-heavy ⇒ **async job pipeline + autoscaling workers**. Blob storage (S3/GCS) durable.

API Design

```
POST /api/v1/uploads      → { uploadId, presignedUrls[] } (multipart/resumable)
PUT <presignedUrl>        (chunk upload directly to blob store)
POST /api/v1/uploads/{id}/complete → transcode job enqueue → { videoId,
status:"processing" }

GET /api/v1/videos/{id}   → metadata + manifest url
```

```
GET /manifest/{id}.m3u8      (HLS) or /{id}.mpd (DASH) → rendition ladder
GET /segments/{id}/720p/00042.ts → media segment (CDN served)
```

HLS/DASH manifest = rendition ladder (240p...4K) list + segment URLs. Player network బట్టి rendition switch చేస్తుంది (ABR) = client-driven.

Data Model

```
videos:      videoId, uploaderId, title, status(processing/ready/failed), duration,
              createdAt
renditions:  videoId, resolution, bitrate, codec, manifestPath, segmentCount
transcode_jobs: jobId, videoId, inputPath, state, attempts, workerId, priority
blob store:  raw/{videoId}, renditions/{videoId}/{res}/segment_*.ts, manifests
metadata DB: relational (videos, users) + search index (title/tags)
```

High-level Architecture

```
UPLOAD/TRANSCODE (write, async):
  Client → Upload API (Go) → Blob store (raw) [presigned, direct-to-blob]
      |
      |→ enqueue Transcode Job (Kafka/queue)
      |
      |→ Transcode Workers (Go orchestrators + ffmpeg) ← autoscaled
      |    240p / 480p / 720p / 1080p / 4K (parallel)
      |
      |→ Blob store (segments + HLS/DASH manifest) → mark video
      |
      |→ "ready"

PLAYBACK (read, massive):
  Player → CDN edge (nearest) —cache hit→ segment (99% ఇక్కడే)
      |
      |→ miss → Origin (blob) → fill CDN → serve
  Player ABR: bandwidth measure → rendition up/down switch per segment
```

Deep Dive – Transcode job pipeline

Transcode = pipeline యొక్క గుండె (compute-heavy, failure-prone, must be reliable).

Stages:

1. **Split** - raw video ని segments/GOPs గా విడగొట్టు (parallel transcode కి).
2. **Fan-out transcode** - ఒక్కో rendition (240p...4K) × ఒక్కో segment = independent job. Massively parallel (worker pool + autoscale).
3. **Package** - transcoded segments → HLS (.ts + .m3u8) / DASH (.mp4 + .mpd) manifest.
4. **Publish** - blob store కి upload, video status "ready", CDN prefetch (popular).

Reliability:

- **Idempotent jobs** - worker crash అయితే job re-run; already-done segment skip (blob లో ఉందా check).

- **At-least-once** queue - job lost కాకూడదు. Duplicate transcode wasteful కానీ correct (deterministic output).
- **Priority** - shorts/popular creators కి fast lane; batch/re-encode low priority.

Go Implementation – transcode job pipeline (orchestrator)

Go = orchestrator (ffmpeg actual encode చేస్తుంది; Go concurrency, retries, idempotency manage చేస్తుంది). Worker pool + per-rendition fan-out + idempotency.

```
package transcode

import (
    "context"
    "fmt"
    "os/exec"
    "sync"
)

type Rendition struct {
    Name      string // "720p"
    Width     int
    Bitrate   string // "2500k"
}

// standard ladder – ABR కి renditions.
var Ladder = []Rendition{
    {"240p", 426, "400k"},
    {"480p", 854, "1000k"},
    {"720p", 1280, "2500k"},
    {"1080p", 1920, "4500k"},
}

type Job struct {
    VideoID   string
    InputPath string // blob store లో raw
    Attempt   int
}

type BlobStore interface {
    Exists(ctx context.Context, key string) (bool, error) // idempotency check
    Put(ctx context.Context, key, localPath string) error
}

type Worker struct {
    blob      BlobStore
    parallel  int // ఒక video కి ఎన్ని renditions ఏకకాలంలో
}

// Transcode – ఒక video కి అన్ని renditions ని parallel గా process (idempotent).
func (w *Worker) Transcode(ctx context.Context, job Job) error {
    sem := make(chan struct{}, w.parallel) // $3 semaphore: bounded parallel
    encodes
    var wg sync.WaitGroup
    errCh := make(chan error, len(Ladder))

    for _, r := range Ladder {
        wg.Add(1)
```

```

        go func(r Rendition) {
            defer wg.Done()
            sem <- struct{}{} // acquire (max parallel encodes)
            defer func() { <-sem }() // release

            outKey := fmt.Sprintf("renditions/%s/%s/index.m3u8",
job.VideoID, r.Name)

            // IDEMPOTENCY: ఈ rendition ఇదివరకే అయిందా? (worker crash
retry safe)

            if ok, _ := w.blob.Exists(ctx, outKey); ok {
                return // skip – already transcoded (deterministic
output)
            }

            localOut, err := w.encode(ctx, job.InputPath, r) // ffmpeg
invoke

            if err != nil {
                errCh <- fmt.Errorf("%s: %w", r.Name, err)
                return
            }
            if err := w.blob.Put(ctx, outKey, localOut); err != nil {
                errCh <- err
            }
        }(r)
    }

    wg.Wait()
    close(errCh)
    // ఏ ఒక్క rendition fail అయినా job retry (partial progress idempotency వల్ల
wasted కాదు)
    for e := range errCh {
        if e != nil {
            return e // queue job ని re-deliver చేస్తుంది; done renditions
skip అవుతాయి
        }
    }

    // అన్ని renditions ready → master manifest రాయ + status update
    return w.writeMasterManifest(ctx, job.VideoID)
}

// encode – Go orchestrator ffmpeg ని invoke చేస్తుంది (context తో cancellable).
func (w *Worker) encode(ctx context.Context, input string, r Rendition) (string,
error) {
    out := fmt.Sprintf("/tmp/%s_%s.m3u8", r.Name, r.Name)
    // HLS segments generate: scale + bitrate + segment
    cmd := exec.CommandContext(ctx, "ffmpeg",
        "-i", input,
        "-vf", fmt.Sprintf("scale=%d:-2", r.Width),
        "-b:v", r.Bitrate,
        "-hls_time", "6", // 6-sec segments (ABR switch granularity)
        "-hls_playlist_type", "vod",
        "-hls_segment_filename", fmt.Sprintf("/tmp/%s_%03d.ts", r.Name),
        out,
    )
    if err := cmd.Run(); err != nil {
        return "", fmt.Errorf("ffmpeg %s: %w", r.Name, err)
    }
    return out, nil
}

```

```

}

func (w *Worker) writeMasterManifest(ctx context.Context, videoID string) error {
    // master.m3u8: అన్ని rendition playlists list (player ABR ఎంచుకోడానికి)
    return nil
}

// ---- Job consumer: queue నుండి jobs తీసి worker pool తో process ----
func RunConsumer(ctx context.Context, w *Worker, jobs <-chan Job, workers int) {
    var wg sync.WaitGroup
    for i := 0; i < workers; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            for job := range jobs {
                if err := w.Transcode(ctx, job); err != nil {
                    // requeue with backoff (attempt++); max దాటితే
                    DLQ
                    _ = err
                }
            }
        }()
    }
    wg.Wait()
}

```

Go ఎందుకు perfect ఇక్కడ: Go actual encode చేయదు (ffmpeg C library చేస్తుంది), కానీ **orchestration** కి perfect - `exec.CommandContext` (cancellable ffmpeg), **semaphore** (§3, bounded parallel encodes - CPU overwhelm కాకుండా), worker pool (queue drain), **idempotent skip** (crash-safe retry). Go binary చిన్నది ⇒ transcode workers ని k8s లో fast autoscale (upload spike కి).

Bottlenecks & Trade-offs

- **Transcode compute cost:** అతి ఖరీదు. Trade-off: pre-transcode అన్ని renditions (storage ఎక్కువ, playback ready) vs on-demand (compute at first view, storage తక్కువ). Popular videos = all renditions; long-tail = fewer.
- **CDN cost vs origin load:** 99% CDN hit కావాలి. Cold/long-tail videos CDN miss ⇒ origin. Trade-off: CDN storage cost vs origin bandwidth.
- **ABR quality vs buffering:** aggressive high-bitrate = buffering risk; conservative = low quality. Player buffer-based ABR (buffer తగ్గితే bitrate down).
- **Upload reliability:** పెద్ద files ⇒ resumable/multipart, presigned direct-to-blob (API bypass).
- **Hot new video (viral):** millions ఒకసారి ⇒ CDN prefetch/pre-warm + origin shielding (tiered CDN).

Key Points

- **Async transcode pipeline:** upload → split → parallel per-rendition encode → package (HLS/DASH) → publish.
- **Idempotent jobs** (blob exists check) ⇒ crash-safe at-least-once retry, no wasted re-encode.

- **CDN serves 99% playback**; origin shielded. Playback bandwidth Tbps $\Rightarrow$ CDN తప్పనిసరి.
- **ABR** = client-driven rendition switch (HLS/DASH manifest); 6-sec segments.
- Go = **orchestrator** (ffmpeg does encode): semaphore-bounded parallelism, `exec.CommandContext`, autoscaled workers.

11. Ride-sharing / Geo (Uber / Lyft)

Real-life Analogy: ఊరిని grid boxes గా విభజించడం. ప్రతి driver ఏ box లో ఉన్నాడో ఒక board మీద గుర్తుపెడతారు. Rider "నాకు cab కావాలి" అంటే - rider box + చుట్టుపక్కల boxes లోని drivers ని మాత్రమే చూస్తారు (మొత్తం ఊరు కాదు). Geohash = ఆ box పేరు. దగ్గరి boxes పేర్లు దాదాపు ఒకేలా ఉంటాయి (prefix match) - అందుకే వెతకడం fast.

Requirements (Functional + Non-functional)

Functional:

- Drivers **location updates** (ప్రతి కొన్ని seconds)
- Rider request $\Rightarrow$ **nearby drivers** find + **match**
- Trip lifecycle (request $\rightarrow$ match $\rightarrow$ pickup $\rightarrow$ drop $\rightarrow$ pay)
- ETA, live tracking

Non-functional:

- **High write throughput** - millions drivers $\times$ location update ప్రతి 4s
- **Low latency matching** - < seconds (rider waiting)
- **Geospatial proximity query** - "5km radius drivers" fast
- Consistency: driver ఒకసారి ఒక ride కి మాత్రమే (no double-book)

Estimation (QPS, storage)

Active drivers: 5M concurrent
 Location update: ప్రతి 4 sec $\rightarrow 5M/4 = 1.25M$ writes/sec (అతి high write!)
 $\rightarrow$ in-memory geo index (Redis/custom), DB కి కాదు ప్రతి update
 Ride requests : 500K/min peak $\rightarrow \sim 8K$ matches/sec
 Proximity query: ప్రతి request కి nearby drivers $\rightarrow$ geohash bucket lookup O(nearby)
 Location state: 5M drivers $\times \sim 50$ bytes = 250 MB $\rightarrow$ in-memory సులభం (sharded by region)

Design driver: 1.25M location writes/sec $\Rightarrow$ ఇవి DB కి వెళ్ళకూడదు (in-memory geo index, latest-wins). Proximity query $\Rightarrow$ **geohash/quadtree** బకెట్. Matching $\Rightarrow$ per-region single-owner (double-book నివారణ).

API Design

```
POST /api/v1/drivers/location { driverId, lat, lng, heading } → 200 (high frequency)
POST /api/v1/rides/request   { riderId, pickup:{lat,lng}, dropoff }
                               → { rideId, status:"searching" }
WebSocket /ws/rides/{id}     → live: {"matched", driverId, eta} /
{"driver_location", lat,lng}
```

Internal: match service → nearby query → rank → offer → driver accept/reject

Data Model

```
driver_location (in-memory, sharded by region):
  driverId → { lat, lng, geohash, status(available/on_trip), lastUpdate }

geo index (in-memory):
  geohash bucket → set of driverIds (ఒక్కో ~1km² cell)

rides (DB, durable):
  rideId, riderId, driverId, state, pickup, dropoff, fare, timestamps

Redis: driver availability locks (matching atomicity)
```

Geohash: lat/lng → short string (ఉదా `tdr1y`). Precision = length (6 chars ≈ 1.2km cell). దగ్గరి points ⇒ common prefix. Neighboring cells కూడా చూడాలి (boundary; rider cell edge లో ఉంటే పక్క cell driver దగ్గరగా ఉండవచ్చు).

High-level Architecture

```
LOCATION (high write):
  Driver app → LB → Location Service (Go) → in-memory Geo Index (sharded by region)
                                                    (latest-wins; DB కి కాదు ప్రతి update;
                                                    periodic snapshot/trip-log మాత్రం DB కి)

MATCHING:
  Rider → Match Service (Go) → Geo Index: nearby drivers (rider geohash + neighbors)
                                     |
                                     → rank (distance, rating, ETA, surge)
    ↳ offer to top driver (Redis lock: double-book నివారణ)
      accept? → create ride (DB) → notify both (WebSocket)
      reject/timeout? → next driver

Sharding: geography ప్రకారం (city/region) → ఒక్కో shard తన drivers ని own చేస్తుంది
```

Deep Dive – Geospatial index + matching (atomicity)

Proximity query options:

పద్ధతి	ఎలా	Trade-off
Geohash	lat/lng → string; bucket = cell; prefix = nearby	Simple, boundary cells handle చేయాలి
Quadtree	plane ని recursively 4 గా విభజన; dense ⇒ deeper	Adaptive density; rebalance ఖరీదు
S2/H3	sphere → cells (Google/Uber production)	Accurate, library needed

Geohash boundary problem: rider cell edge దగ్గర ⇒ నిజంగా దగ్గరి driver పక్క cell లో ఉండవచ్చు.
Solution: rider cell + **8 neighbor cells** అన్నీ query.

Matching atomicity (కీలకో): ఒక driver ని ఇద్దరు riders కి ఏకకాలంలో offer చేస్తే double-book.
Solution: driver ని offer చేసేటప్పుడు **atomic lock** (Redis SET NX driver:x rideId EX 15). Driver reject/timeout ⇒ release. ఇది distributed mutual exclusion.

Go Implementation – geospatial index + matching

In-memory sharded geo index (per-region lock) + geohash neighbors + atomic driver reservation. Location updates lock-cheap, queries neighbor-aware.

```
package geo

import (
    "math"
    "sort"
    "sync"
)

type Driver struct {
    ID      string
    Lat     float64
    Lng     float64
    Geohash string
    OnTrip  bool
}

// GeoIndex – geohash bucket → drivers. Region ప్రకారం shard అవుతుంది.
// RWMutex: reads (queries) ఎక్కువ, writes (location updates) కూడా ఎక్కువ →
// production లో per-bucket లేదా sharded lock (contention తగ్గించడానికి).
type GeoIndex struct {
    mu      sync.RWMutex
    buckets map[string]map[string]*Driver // geohash → (driverID → driver)
    drivers map[string]*Driver           // driverID → driver (fast update)
    precision int                     // geohash length (cell size)
}

func NewGeoIndex(precision int) *GeoIndex {
    return &GeoIndex{
        buckets: make(map[string]map[string]*Driver),
        drivers: make(map[string]*Driver),
        precision: precision,
    }
}

// UpdateLocation – high-frequency. Driver bucket మారితే move చేస్తుంది.
```

```

func (g *GeoIndex) UpdateLocation(id string, lat, lng float64) {
    newHash := Encode(lat, lng, g.precision)

    g.mu.Lock()
    defer g.mu.Unlock()

    d, exists := g.drivers[id]
    if exists && d.Geohash == newHash {
        d.Lat, d.Lng = lat, lng // అదే cell → in-place update (cheap)
        return
    }
    if exists { // cell మారింది → పాత bucket నుండి remove
        delete(g.buckets[d.Geohash], id)
        if len(g.buckets[d.Geohash]) == 0 {
            delete(g.buckets, d.Geohash)
        }
    } else {
        d = &Driver{ID: id}
        g.drivers[id] = d
    }
    d.Lat, d.Lng, d.Geohash = lat, lng, newHash
    if g.buckets[newHash] == nil {
        g.buckets[newHash] = make(map[string]*Driver)
    }
    g.buckets[newHash][id] = d
}

// Nearby – rider cell + 8 neighbors లోని available drivers, distance sorted.
func (g *GeoIndex) Nearby(lat, lng float64, limit int) []*Driver {
    center := Encode(lat, lng, g.precision)
    cells := append(Neighbors(center), center) // boundary problem fix: 8 + self

    g.mu.RLock()
    var found []*Driver
    for _, cell := range cells {
        for _, d := range g.buckets[cell] {
            if !d.OnTrip {
                found = append(found, d)
            }
        }
    }
    g.mu.RUnlock()

    // నిజమైన haversine distance తో sort (geohash cell approximate)
    sort.Slice(found, func(i, j int) bool {
        return haversine(lat, lng, found[i].Lat, found[i].Lng) <
            haversine(lat, lng, found[j].Lat, found[j].Lng)
    })
    if len(found) > limit {
        found = found[:limit]
    }
    return found
}

// ---- Matching with atomic reservation (double-book నివారణ) ----

type Reservation interface {
    // TryReserve – driver ని ఈ ride కి atomic గా lock (Redis SET NX EX).
    // ఇద్దరు riders ఏకకాలంలో try చేస్తే ఒక్కరే గెలుస్తారు.

```

```

    TryReserve(driverID, rideID string) (bool, error)
    Release(driverID string) error
}

type Matcher struct {
    index *GeoIndex
    resv Reservation
}

// Match - nearby drivers ని rank చేసి, ఒక్కొక్కరికి offer (atomic reserve).
func (m *Matcher) Match(rideID string, lat, lng float64) (*Driver, error) {
    candidates := m.index.Nearby(lat, lng, 10)
    for _, d := range candidates {
        ok, err := m.resv.TryReserve(d.ID, rideID) // ATOMIC - double-book
నివారణ
        if err != nil {
            continue
        }
        if ok {
            // driver కి offer పంపు (WebSocket). accept? → confirm;
            reject/timeout? → Release + next
            return d, nil
        }
        // ఈ driver ఇప్పటికే reserved → తర్వాతివాడు
    }
    return nil, nil // ఈ radius లో ఎవరూ లేరు → radius పెంచు లేదా surge
}

// ---- geohash + haversine helpers ----
const geohashBase32 = "0123456789bcdefghjkmnpqrstuvwxyz"

func Encode(lat, lng float64, precision int) string {
    latR := [2]float64{-90, 90}
    lngR := [2]float64{-180, 180}
    var hash []byte
    even := true
    bit, ch := 0, 0
    for len(hash) < precision {
        var mid float64
        if even {
            mid = (lngR[0] + lngR[1]) / 2
            if lng > mid {
                ch |= 1 << (4 - bit)
                lngR[0] = mid
            } else {
                lngR[1] = mid
            }
        } else {
            mid = (latR[0] + latR[1]) / 2
            if lat > mid {
                ch |= 1 << (4 - bit)
                latR[0] = mid
            } else {
                latR[1] = mid
            }
        }
        even = !even
        if bit < 4 {
            bit++

```

```

        } else {
            hash = append(hash, geohashBase32[ch])
            bit, ch = 0, 0
        }
    }
    return string(hash)
}

// Neighbors – skeleton: 8 surrounding cells (production: proper geohash adjacency).
func Neighbors(hash string) []string { return nil }

func haversine(lat1, lng1, lat2, lng2 float64) float64 {
    const R = 6371e3 // meters
    φ1, φ2 := lat1*math.Pi/180, lat2*math.Pi/180
    dφ := (lat2 - lat1) * math.Pi / 180
    dλ := (lng2 - lng1) * math.Pi / 180
    a := math.Sin(dφ/2)*math.Sin(dφ/2) +
        math.Cos(φ1)*math.Cos(φ2)*math.Sin(dλ/2)*math.Sin(dλ/2)
    return R * 2 * math.Atan2(math.Sqrt(a), math.Sqrt(1-a))
}

```

Go ఎందుకు perfect: 1.25M location updates/sec $\Rightarrow$ in-memory index + `RWMutex` (production: **per-region shard** $\Rightarrow$ lock contention split - ఒక్కో city వేరే `GeoIndex`, వేరే goroutine). Location update = latest-wins in-place (DB write కాదు). Matching = **atomic reserve** (Redis lock) $\Rightarrow$ Go goroutines concurrent match చేసినా double-book లేదు. `haversine` native math.

Bottlenecks & Trade-offs

- **Write hot region:** downtown = huge drivers $\Rightarrow$ single shard hot. Finer geohash + shard by region + per-bucket lock.
- **Geohash boundary:** cell edge $\Rightarrow$ 8 neighbors query (miss avoid). Trade-off: query range vs precision.
- **Location update cost:** ప్రతి 4s $\times$ 5M = huge. Trade-off: frequency (fresh vs load); adaptive (moving fast $\Rightarrow$ frequent, idle $\Rightarrow$ rare).
- **Matching fairness vs efficiency:** nearest driver (rider fast) vs global optimization (fleet efficiency, batch matching). Uber batches slightly for better global assignment.
- **Double-book:** atomic reservation తప్పనిసరి; lock TTL (driver no-response $\Rightarrow$ auto-release, deadlock నివారణ).

Key Points

- **Geohash/quadtree bucket** $\Rightarrow$ proximity query $O(\text{nearby cells})$, మొత్తం ఊరు scan లేదు. **8 neighbors** for boundary.
- **In-memory geo index, latest-wins;** location updates DB కి వెళ్ళవు (1.25M/sec).
- **Shard by region** $\Rightarrow$ lock contention split, hot-region isolation.
- **Atomic driver reservation** (Redis `SET NX EX`) $\Rightarrow$ double-book నివారణ + auto-release TTL.
- Go: `RWMutex` /sharded locks, goroutine-per-region, `haversine` distance ranking.

12. Distributed Job Scheduler (Cron at Scale)

Real-life Analogy: రైల్వే ట్రైమ్‌టేబుల్ + స్టేషన్ మాస్టర్. వేల trains (jobs) నిర్ణీత సమయాలకు బయలుదేరాలి. ఒక్క station master (leader) time చూసి "ఇప్పుడు ఏ trains బయలుదేరాలి" అని announce చేస్తాడు. Master సెలవు అయితే మరో master (leader election) బాధ్యత తీసుకుంటాడు - కానీ ఒకేసారి ఇద్దరు master ఉంటే గందరగోళం (ఒకే train రెండుసార్లు). అందుకే ఒక్కడే master (lease), platforms (shards) మధ్య trains పంచుకుంటారు.

Requirements (Functional + Non-functional)

Functional:

- Cron jobs schedule (one-time + recurring: "ప్రతి రోజు 2am")
- **Millions of jobs**, second-level precision
- Job execution tracking (success/fail/retry)
- Timezone-aware, backfill on downtime

Non-functional:

- **Reliability** - job miss కాకూడదు (at-least-once execution)
- **No duplicate execution** (లేదా idempotent - at-least-once $\Rightarrow$ duplicates possible)
- **Horizontal scale** - jobs across workers
- **Fault tolerance** - node down అయినా jobs run

Estimation (QPS, storage)

```
Jobs           : 10M scheduled jobs
Due/sec (peak): అనేకం ఒకే second కి (2am spike)  $\rightarrow$  due-index query fast కావాలి
Job record     : ~200 bytes  $\rightarrow 10M \times 200B = 2\text{ GB} \rightarrow$  DB సులభం
Sharding       : jobs ని N shards లో (hash jobId)  $\rightarrow$  ఒక్కో worker shard subset
Precision      : second-level  $\rightarrow$  tick loop / due-time index (sorted by nextRun)
```

Design driver: millions jobs, second precision $\Rightarrow$ **due-time index** (sorted). No-duplicate $\Rightarrow$ **leader election OR shard ownership + lease**. Reliability $\Rightarrow$ at-least-once + idempotent jobs.

API Design

```
POST /api/v1/jobs { name, cronExpr:"0 2 * * *", payload, timezone }  $\rightarrow$  { jobId }
DELETE /api/v1/jobs/{id}
GET /api/v1/jobs/{id}/runs  $\rightarrow$  execution history

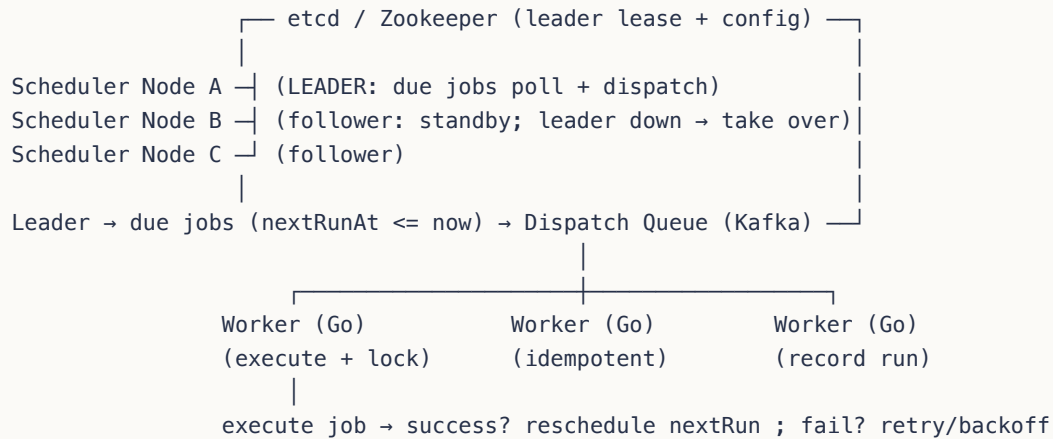
Internal: Scheduler (leader)  $\rightarrow$  due jobs  $\rightarrow$  dispatch  $\rightarrow$  Worker pool  $\rightarrow$  execute  $\rightarrow$  record + reschedule
```

Data Model

```
jobs:      jobId, name, cronExpr, payload, nextRunAt, status, shardId, timezone
job_runs:  runId, jobId, scheduledFor, startedAt, finishedAt,
state(running/success/failed), attempt
leases:    leaderKey → (nodeId, expiry)      [etcd/Zookeeper/DB]
locks:     "job:{id}:{scheduledFor}" → nodeId  (duplicate execution నివారణ)

Due index: jobs sorted by nextRunAt (DB index లేదా in-memory min-heap/priority queue)
```

High-level Architecture



Alternative (shared-nothing): leader కాకుండా - jobs ని **shard** (hash jobId → shard), ఒక్కో scheduler node కొన్ని shards **own** (via lease). Node down ⇒ దాని shards మరో node కి rebalance. Single leader bottleneck లేదు. (Production: ఈ shard+lease approach scale అవుతుంది.)

Deep Dive – Leader election + at-least-once + no-duplicate

Leader election (why): ఇద్దరు schedulers ఒకే job ని dispatch చేస్తే duplicate. ఒక్కడే "who dispatches" నిర్ణయించాలి.

- **etcd lease:** node ఒక lease (TTL key) పట్టుకుంటుంది. Renew చేస్తూ ఉంటే leader. Crash ⇒ lease expire ⇒ మరో node grab. (Raft-backed, split-brain safe.)
- **Fencing token:** leader ప్రతి action కి monotonic token జతచేస్తుంది. పాత leader (paused, lease expired but alive) stale token తో వస్తే reject.

At-least-once vs exactly-once:

- Network/crash వల్ల exactly-once దాదాపు అసాధ్యం (distributed). Practical target = **at-least-once + idempotent jobs**.
- **No-duplicate execution** best-effort: `job:{id}:{scheduledFor}` lock (ఒక్క worker మాత్రమే execute). కానీ worker lock తీసుకుని crash అయితే? Lease expiry ⇒ మరో worker retry ⇒ possible duplicate ⇒ **job idempotent** అయ్యుండాలి.

Scheduling precision: min-heap by nextRunAt. Leader `sleep until next due`, wake, dispatch, reschedule. Second-level precision.

Go Implementation – scheduler with etcd lease + worker

Leader election (etcd lease), due-job dispatch (min-heap), worker with per-run lock + idempotency. `context` for leadership loss.

```
package scheduler

import (
    "container/heap"
    "context"
    "time"

    clientv3 "go.etcd.io/etcd/client/v3"
    "go.etcd.io/etcd/client/v3/concurrency"
)

// ---- Leader election via etcd lease ----

type Node struct {
    cli    *clientv3.Client
    nodeID string
}

// RunAsLeader – leadership గెలిస్తే fn నడుస్తుంది. Leadership పోతే ctx cancel.
// etcd concurrency.Election = Raft-backed, split-brain safe.
func (n *Node) RunAsLeader(ctx context.Context, fn func(ctx context.Context)) error {
    sess, err := concurrency.NewSession(n.cli, concurrency.WithTTL(10)) // 10s
    lease

    if err != nil {
        return err
    }
    defer sess.Close()

    el := concurrency.NewElection(sess, "/scheduler/leader")
    // Campaign – leader అయ్యేదాకా block (queue లో wait)
    if err := el.Campaign(ctx, n.nodeID); err != nil {
        return err
    }

    // మనం ఇప్పుడు leader. కానీ lease పోతే (network partition) leadership కోల్పోతాం.
    leaderCtx, cancel := context.WithCancel(ctx)
    defer cancel()
    go func() {
        select {
        case <-sess.Done(): // lease expired → leadership lost → fn ని ఆపు
            cancel()
        case <-ctx.Done():
        }
    }()

    fn(leaderCtx) // leader logic (due-job dispatch loop)
    return nil
}

// ---- Due-job dispatch (leader only) ----

type Job struct {
    ID      string
    // ... other fields ...
}
```

```

    CronExpr    string
    NextRunAt   time.Time
    Payload     []byte
}

// jobHeap - min-heap by NextRunAt (earliest due job పైస).
type jobHeap []*Job

func (h jobHeap) Len() int           { return len(h) }
func (h jobHeap) Less(i, j int) bool { return h[i].NextRunAt.Before(h[j].NextRunAt) }
func (h jobHeap) Swap(i, j int)      { h[i], h[j] = h[j], h[i] }
func (h *jobHeap) Push(x any)        { *h = append(*h, x.(*Job)) }
func (h *jobHeap) Pop() any {
    old := *h
    j := old[len(old)-1]
    *h = old[:len(old)-1]
    return j
}

type Dispatcher struct {
    jobs    *jobHeap
    dispatch func(ctx context.Context, j *Job) error // Kafka కి push
    next     func(cronExpr string, after time.Time) time.Time
}

// DispatchLoop - leader loop: next due job చూసా sleep, due అయితే dispatch +
// reschedule.
// leadership పోతే (ctx cancel) వెంటనే ఆగుతుంది (duplicate dispatch నివారణ).
func (d *Dispatcher) DispatchLoop(ctx context.Context) {
    for {
        if d.jobs.Len() == 0 {
            select {
            case <-time.After(time.Second):
            case <-ctx.Done():
                return
            }
            continue
        }
        top := (*d.jobs)[0]
        wait := time.Until(top.NextRunAt)

        select {
        case <-ctx.Done():
            return // LEADERSHIP LOST → dispatch అప్పు (వేరే leader
            తీసుకుంటాడు)
        case <-time.After(wait): // ఈ job due అయింది
            j := heap.Pop(d.jobs).(*Job)
            // dispatch queue కి (Kafka). Worker execute చేస్తాడు.
            // Producer idempotency: key = jobID + scheduledFor
            (duplicate dispatch dedup)
            _ = d.dispatch(ctx, j)

            // recurring అయితే reschedule (కొత్త nextRun తో మళ్ళీ heap లో)
            if j.CronExpr != "" {
                j.NextRunAt = d.next(j.CronExpr, time.Now())
                heap.Push(d.jobs, j)
            }
        }
    }
}

```

```

    }
}

// ---- Worker: execute with per-run lock + idempotency ----

type RunLock interface {
    // Acquire - "job:{id}:{scheduledFor}" ని lock (SET NX). ఒక్క worker మాత్రమే
    execute.
    Acquire(ctx context.Context, key string, ttl time.Duration) (bool, error)
}

type Worker struct {
    lock RunLock
    exec func(ctx context.Context, j *Job) error // actual job (idempotent
    అయ్యుండాల్సి)
}

// Handle - queue నుండి వచ్చిన job ని execute. Lock + idempotency.
func (w *Worker) Handle(ctx context.Context, j *Job, scheduledFor time.Time) error {
    key := "run:" + j.ID + ":" + scheduledFor.Format(time.RFC3339)

    // per-run lock: duplicate delivery (at-least-once queue) వస్తే ఒక్కరే execute
    got, err := w.lock.Acquire(ctx, key, 5*time.Minute)
    if err != nil {
        return err
    }
    if !got {
        return nil // మరో worker ఇప్పటికే execute చేస్తున్నాడు → skip
    }

    // job execute. exec IDEMPOTENT అయ్యుండాల్సి - lock holder crash అయితే
    // మరో worker retry చేయవచ్చు (at-least-once ⇒ duplicate possible).
    return w.exec(ctx, j)
}

```

Go ఎందుకు perfect: context cancellation = leadership loss ని instant గా propagate చేస్తుంది - leader lease పోగానే `ctx.Done()` ⇒ dispatch loop ఆగుతుంది (rogue duplicate dispatch నివారణ). container/heap = second-precision due scheduling (sleep-until-next). etcd concurrency library = production-grade Raft leader election. Worker pool goroutines + per-run lock = at-least-once, best-effort no-duplicate.

Bottlenecks & Trade-offs

- **Single leader bottleneck:** ఒక్క leader millions jobs dispatch. Solution: **shard jobs** (leader per shard, లేదా shared-nothing shard ownership) ⇒ horizontal scale.
- **Thundering herd (2am spike):** వేల jobs ఒకే second. Solution: **jitter** (spread within window), rate-limit dispatch, priority.
- **At-least-once ⇒ duplicates:** exactly-once అసాధ్యం (distributed) ⇒ **jobs idempotent** design (idempotency key). ఇది non-negotiable.
- **Clock skew:** nodes clocks differ ⇒ early/late dispatch. NTP sync + leader single clock source.

- **Missed jobs (downtime):** scheduler down అయిన కాలంలో due jobs? On recovery **backfill** (nextRunAt < now due jobs catch up), లేదా skip (config).
- **Leader failover gap:** old leader crash → new leader elect (lease TTL delay). ఆ gap లో due jobs కొంచెం late.

Key Points

- **Leader election (etcd lease, Raft)** ⇒ single dispatcher, split-brain safe; `context` cancel on leadership loss.
- **Min-heap by nextRunAt** ⇒ second-precision sleep-until-due dispatch.
- **At-least-once + idempotent jobs** (exactly-once అసాధ్యం); per-run lock = best-effort no-duplicate.
- **Shard jobs** for horizontal scale (single leader bottleneck నివారణ).
- **Jitter** for thundering herd; **backfill** for missed jobs; NTP for clock skew.
- Go: etcd `concurrency`, `container/heap`, `context` propagation.

13. Distributed Key-Value Store (Dynamo-style)

Real-life Analogy: గ్రంథాలయాల నెట్‌వర్క్. ఒక్క పుస్తకం (key) ఏ లైబ్రరీ (node) లో ఉందో consistent hashing chart చెబుతుంది. భద్రత కోసం ప్రతి పుస్తకం 3 లైబ్రరీలలో copies (replication). "పుస్తకం ఇవ్వు" అంటే ఏదో ఒక copy చాలు (R=1, fast) లేదా ఖచ్చితత్వం కోసం majority copies check (R=2, latest). ఒక లైబ్రరీ మూతపడితే మిగతావి ఉన్నాయి (availability). లైబ్రరీలు ఒకదానికొకటి "ఏ పుస్తకాలు ఉన్నాయి" gossip చేసుకుంటాయి.

Requirements (Functional + Non-functional)

Functional:

- `Put(key, value)`, `Get(key)`, `Delete(key)`
- **Replication** (N copies, durability)
- **Tunable consistency** (R/W quorum)

Non-functional:

- **High availability** (AP - node down అయినా serve; Dynamo philosophy)
- **Horizontal scale** - nodes add చేస్తే capacity పెరగాలి
- **Partition tolerant** (CAP: AP - network split అయినా available)
- Low latency, eventual consistency (tunable)

Estimation (QPS, storage)

Data	: 100 TB, ఒక్కో value ~1 KB → 100B keys
Nodes	: ఒక్కో node ~2 TB → 50 nodes (+ replication N=3 → 150 TB raw)
QPS	: 1M ops/sec → nodes మధ్య పంచుకుంటే ~20K/node

Replication: $N=3$ (durability); quorum $W+R > N$ (strong-ish) OR $W=1, R=1$ (fast, eventual)

Vnodes : ఒక్క physical node ~150 virtual nodes (balance)

Design driver: horizontal scale + availability $\Rightarrow$ **consistent hashing** (node add/remove $\sim 1/N$ keys move) + **replication** + **tunable quorum**. Membership $\Rightarrow$ **gossip** (decentralized, no single coordinator).

API Design

```
PUT /kv/{key}    body=value, header X-Consistency: W=2     $\rightarrow 200$  (W replicas ack అయ్యాక)
GET /kv/{key}    header X-Consistency: R=2                 $\rightarrow \{ \text{value, version}(\text{vector clock}) \}$ 
DELETE /kv/{key}                                 $\rightarrow$  tombstone (eventual)
```

Internal (coordinator $\rightarrow$ replicas): gRPC replicate(key, value, version)

Gossip: node $\leftrightarrow$ node membership + failure detection

Data Model

Ring: consistent hash (key $\rightarrow$ coordinator node $\rightarrow$ next $N-1$ nodes = replicas)

Storage/node: LSM-tree (write-heavy) – key $\rightarrow$ (value, vectorClock, tombstone)

Versioning: vector clock లేదా last-write-wins timestamp (conflict resolution)

Membership: gossip-maintained node list + heartbeat (alive/suspect/dead)

Hinted handoff: replica down అయితే temp node data ఉంచి, recover అయ్యాక తిరిగి ఇస్తుంది

High-level Architecture

```
Client  $\rightarrow$  ఏదైనా node (coordinator)  $\rightarrow$  consistent hash ring
|
| key  $\rightarrow$  preference list: [N1, N2, N3] (N replicas)
|  $\rightarrow$  N1 (coordinator)  $\rightarrow$ 
|  $\rightarrow$  N2 (replica)  $\rightarrow$  W acks కావాలి (write quorum)
|  $\rightarrow$  N3 (replica)
|
Read: R replicas అడుగు  $\rightarrow$  latest version return; stale replicas ని repair (read-repair)

Gossip: nodes ప్రతి కొన్ని sec heartbeat + membership exchange (decentralized, no master)
Ring change: node join/leave  $\rightarrow \sim 1/N$  keys migrate (consistent hashing వల్ల)
```

Deep Dive – Consistent hashing + quorum (R/W) + conflict resolution

Consistent hashing (§3 toolkit): key $\rightarrow$ ring position $\rightarrow$ clockwise next node = coordinator; తర్వాత $N-1$ nodes = replicas (**preference list**). Node add/remove $\Rightarrow \sim 1/N$ keys మాత్రమే migrate ($\text{mod-}N$ అయితే దాదాపు అన్నీ).

Quorum (tunable consistency): N replicas. W = write ఎన్ని ack కావాలి. R = read ఎన్ని అడగాలి.

- **$W + R > N$** $\Rightarrow$ read set $\cap$ write set $\neq \emptyset \Rightarrow$ read latest చూస్తుంది (**strong-ish consistency**). ఉదా $N=3, W=2, R=2$.
- **$W=1, R=1$** $\Rightarrow$ fast కానీ stale read possible (**eventual**).
- **$W=N$** $\Rightarrow$ slow write, fast read; **$R=N$** $\Rightarrow$ vice versa.
- Trade-off: latency vs consistency, ప్రతి request tune చేయవచ్చు.

Conflict resolution: network partition లో ఒకే key రెండు nodes లో వేర్వేరుగా update అయితే?

- **Last-Write-Wins (LWW):** timestamp పెద్దది గెలుస్తుంది (simple కానీ data loss).
- **Vector clocks:** causality track; concurrent writes $\Rightarrow$ conflict detect $\Rightarrow$ client resolves (Dynamo shopping cart).

Failure handling:

- **Hinted handoff:** replica down $\Rightarrow$ temp మరో node data హోల్డ్ చేస్తుంది; recover అయ్యాక handoff. Write availability కాపాడుతుంది.
- **Read repair:** read లో stale replica కనిపిస్తే latest తో update.
- **Anti-entropy (Merkle trees):** background లో replicas sync (divergence detect).

Go Implementation – consistent-hash ring + node with quorum

Ring (§3 extended: preference list of N) + coordinator quorum write/read + read-repair. Go concurrency = replicas ని parallel గా contact చేసి quorum wait.

```
package kvstore

import (
    "context"
    "errors"
    "hash/crc32"
    "sort"
    "strconv"
    "sync"
    "time"
)

// ---- Ring with preference list (N replicas per key) ----

type Ring struct {
    mu      sync.RWMutex
    replicas int    // virtual nodes per physical node
    keys    []uint32 // sorted ring positions
    hashMap map[uint32]string
    nodes   map[string]bool
}

func NewRing(vnodes int) *Ring {
    return &Ring{replicas: vnodes, hashMap: map[uint32]string{}, nodes:
map[string]bool{}}
}

func (r *Ring) hash(s string) uint32 { return crc32.ChecksumIEEE([]byte(s)) }

func (r *Ring) Add(node string) {
```

```

    r.mu.Lock()
    defer r.mu.Unlock()
    r.nodes[node] = true
    for i := 0; i < r.replicas; i++ {
        h := r.hash(node + "#" + strconv.Itoa(i))
        r.keys = append(r.keys, h)
        r.hashMap[h] = node
    }
    sort.Slice(r.keys, func(i, j int) bool { return r.keys[i] < r.keys[j] })
}

// PreferenceList – key & N distinct physical nodes (coordinator + replicas).
// ring మీద clockwise, duplicate physical nodes skip చేస్తూ.
func (r *Ring) PreferenceList(key string, n int) []string {
    r.mu.RLock()
    defer r.mu.RUnlock()
    if len(r.keys) == 0 {
        return nil
    }
    h := r.hash(key)
    idx := sort.Search(len(r.keys), func(i int) bool { return r.keys[i] >= h })

    var out []string
    seen := map[string]bool{}
    for i := 0; i < len(r.keys) && len(out) < n; i++ {
        node := r.hashMap[r.keys[(idx+i)%len(r.keys)]]
        if !seen[node] {
            seen[node] = true
            out = append(out, node)
        }
    }
    return out
}

// ---- Versioned value ----

type VersionedValue struct {
    Value      []byte
    Timestamp  int64 // LWW (production: vector clock)
    Tombstone  bool
}

type Replica interface {
    Store(ctx context.Context, key string, v VersionedValue) error
    Fetch(ctx context.Context, key string) (VersionedValue, error)
}

// ---- Coordinator: quorum write/read ----

type Coordinator struct {
    ring      *Ring
    replicas  map[string]Replica // nodeID → RPC client
    N, W, R  int                // N replicas, W write quorum, R read quorum
}

var ErrQuorumNotMet = errors.New("quorum not met")

// Put – N replicas & parallel write; W acks వస్తే success (quorum).
func (c *Coordinator) Put(ctx context.Context, key string, value []byte) error {

```

```

nodes := c.ring.PreferenceList(key, c.N)
v := VersionedValue{Value: value, Timestamp: time.Now().UnixNano()}

acks := make(chan error, len(nodes))
for _, node := range nodes {
    go func(node string) { // అన్ని replicas ని PARALLEL లో contact
        r, ok := c.replicas[node]
        if !ok {
            acks <- errors.New("no replica: " + node) // → hinted
handoff candidate
            return
        }
        acks <- r.Store(ctx, key, v)
    }(node)
}

// W successful acks దాకా wait (మిగతావి background లో complete అవుతాయి)
success := 0
for i := 0; i < len(nodes); i++ {
    if err := <-acks; err == nil {
        success++
        if success >= c.W {
            return nil // write quorum met → client కి success
        }
    }
}

return ErrQuorumNotMet // W acks రాలేదు (hinted handoff తో availability
పెంచవచ్చు)
}

// Get - R replicas అడుగు; latest version return + stale replicas repair.
func (c *Coordinator) Get(ctx context.Context, key string) (VersionedValue, error) {
    nodes := c.ring.PreferenceList(key, c.N)

    type resp struct {
        node string
        v    VersionedValue
        err  error
    }
    results := make(chan resp, len(nodes))
    for _, node := range nodes {
        go func(node string) {
            r, ok := c.replicas[node]
            if !ok {
                results <- resp{node, VersionedValue{},
errors.New("no replica")}
                return
            }
            v, err := r.Fetch(ctx, key)
            results <- resp{node, v, err}
        }(node)
    }

    var latest VersionedValue
    var collected []resp
    got := 0
    for i := 0; i < len(nodes); i++ {
        res := <-results
        if res.err == nil {

```


గెలుస్తుంది

```
        collected = append(collected, res)
        if res.v.Timestamp > latest.Timestamp { // LWW: newest

                latest = res.v
        }
        got++
        if got >= c.R {
                break // read quorum met
        }
    }
    if got < c.R {
        return VersionedValue{}, ErrQuorumNotMet
    }

    // READ REPAIR: stale replicas ని latest తో async update (eventual convergence)
    go c.readRepair(context.Background(), key, latest, collected)
    return latest, nil
}

func (c *Coordinator) readRepair(ctx context.Context, key string, latest
VersionedValue, seen []resp) {
    for _, r := range seen {
        if r.v.Timestamp < latest.Timestamp { // ఈ replica stale → repair
            if rep, ok := c.replicas[r.node]; ok {
                _ = rep.Store(ctx, key, latest)
            }
        }
    }
}
}
```

Go ఎందుకు perfect: quorum = N replicas ని **parallel** contact చేసి W (లేదా R) acks కి wait - Go goroutines + buffered channel దీనికి textbook fit. Slow replica మిగతా వాటిని block చేయదు (channel select). Read-repair background goroutine (`go c.readRepair`) - read latency కి add కాకుండా eventual convergence. gRPC (Go native) replica-to-replica RPC.

Bottlenecks & Trade-offs

- **Consistency vs latency (quorum):** $W+R>N$ = strong-ish కానీ slow (ఎక్కువ replicas wait). $W=R=1$ = fast కానీ stale. Per-request tunable.
- **Hot key:** ఒక్క key కి huge QPS $\Rightarrow$ ఆ preference list nodes hot. Solution: key-level caching, replication factor పెంచడం (read).
- **Conflict resolution:** LWW simple కానీ concurrent write ఒకటి పోతుంది; vector clocks correct కానీ complex (client resolve). Trade-off.
- **Membership at scale:** gossip = decentralized కానీ convergence delay (కొన్ని sec). పెద్ద cluster $\Rightarrow$ gossip fanout tune.
- **Ring imbalance:** vnodes తక్కువ $\Rightarrow$ uneven load. 100-200 vnodes/node balance.
- **Node join rebalancing:** $\sim 1/N$ keys migrate $\Rightarrow$ network/disk spike. Throttle migration.

Key Points

- **Consistent hashing + preference list (N replicas)** $\Rightarrow$ scale + durability; node change $\sim 1/N$ keys move.
- **Tunable quorum:** $W+R>N$ = strong-ish, $W=R=1$ = fast/eventual. Parallel replica contact (Go goroutines).
- **Conflict resolution:** LWW (simple) vs vector clocks (correct, client-resolved).
- **Read-repair + hinted handoff + anti-entropy** = eventual convergence, write availability.
- **Gossip** = decentralized membership (no master); AP system (CAP).
- Go: goroutines+channels for quorum, background read-repair, gRPC replication.

14. Distributed Cache (Memcached / Redis-style)

Real-life Analogy: వంటగదిలో చేతికందే అర (spice rack) vs స్టోర్ రూమ్. తరచూ వాడే మసాలాలు (hot data) చేతికందేలా అరలో (cache/RAM) - వెంటనే అందుతాయి. అరలో స్థలం పరిమితం కాబట్టి, చాలా రోజులు వాడని దాన్ని (least recently used) బయటికి తీసి store room (DB) కి పంపుతావు. కొత్తది కావాలంటే store room కెళ్ళి తెచ్చి అరలో పెడతావు (cache-aside).

Requirements (Functional + Non-functional)

Functional:

- `Get(key)` , `Set(key, value, ttl)` , `Delete(key)`
- **LRU eviction** (memory పరిమితం)
- **Distributed** (data across nodes, capacity aggregate)
- TTL expiry

Non-functional:

- **Ultra-low latency** - sub-ms (RAM lookup)
- **High throughput** - millions ops/sec
- **High concurrency** - వేల goroutines ఏకకాలంలో
- Memory-efficient eviction; hot-key handling

Estimation (QPS, storage)

Cache size : 500 GB hot data $\rightarrow$ ఒక్కో node 64 GB $\rightarrow$ ~ 8 nodes
QPS : 5M ops/sec $\rightarrow$ 8 nodes $\rightarrow$ $\sim 600K$ /node $\rightarrow$ single lock అయితే bottleneck!
 $\rightarrow$ sharded lock (node లోపల N shards) $\rightarrow$ contention split
Value size : ~ 1 KB avg $\rightarrow$ 500 GB / 1 KB = 500M entries
Eviction : LRU, TTL; memory pressure $\rightarrow$ evict cold
Hit ratio : target 95%+ (miss = DB hit, ఖరీదు)

Design driver: millions ops/sec + high concurrency $\Rightarrow$ **sharded lock** (single mutex bottleneck నివారణ). Memory పరిమితం $\Rightarrow$ **LRU eviction** ($O(1)$). Distributed $\Rightarrow$ **consistent hashing** (client-side, §3).

API Design

```
GET    key          → value | MISS
SET    key value ttl → OK
DELETE key          → OK
```

Client-side routing: $\text{hash}(\text{key}) \rightarrow \text{node}$ (consistent hashing). Cache nodes shared-nothing

(ఒకదానితో ఒకటి మాట్లాడవు; client ఏ node కి వెళ్ళాలో నిర్ణయిస్తుంది – memcached model).

Cache-aside (application pattern):

```
v = cache.Get(k); if miss { v = db.Load(k); cache.Set(k, v, ttl) }; return v
```

Data Model

Per node: sharded LRU

```
shard[i] = { map[key]*entry + doubly-linked-list (LRU order) + mutex }
entry = { key, value, expiry, prev/next (LRU list pointers) }
```

Routing: consistent hash ring (client-side) $\rightarrow$ key $\rightarrow$ node

Eviction: capacity దాటితే LRU tail remove; TTL expired lazy/active cleanup

High-level Architecture

App (Go) $\rightarrow$ client-side consistent hash $\rightarrow$ ఏ cache node?

```
├─ Cache Node 1 (Go: sharded LRU)
├─ Cache Node 2 (Go: sharded LRU)
└─ Cache Node 3 (Go: sharded LRU)
    └─ shared-nothing (nodes మాట్లాడవు)
```

miss $\rightarrow$ App reads DB $\rightarrow$ App writes cache (cache-aside)

Node internal: key $\rightarrow$ hash $\rightarrow$ shard[i] $\rightarrow$ (map + LRU list) under shard[i].mutex
256 shards $\rightarrow$ lock contention $256\times$ తగ్గింది

Deep Dive – Sharded LRU with per-shard mutex

సమస్య: single `map + mutex` cache = ఒక్క global lock $\Rightarrow$ 5M ops/sec అన్నీ serialize $\Rightarrow$ bottleneck.
Go లో ఒక్క mutex millions ops handle చేయలేదు (contention).

Solution – sharding: cache ని N shards (ఉదా 256) గా విభజించు. `shard = hash(key) % 256`. ఒక్కో shard తన **సొంత mutex + map + LRU list**. వేర్వేరు keys వేర్వేరు shards $\Rightarrow$ locks contend కావు $\Rightarrow$ concurrency $256\times$.

LRU O(1): hash map (key → node) + **doubly-linked list** (recency order). Access ⇒ node ని list front కి move (O(1)). Evict ⇒ list tail remove (O(1) LRU). Go `container/list` వాడవచ్చు, కానీ custom pointers faster.

TTL: lazy (Get లో expired అయితే skip + delete) + active (background sweeper goroutine, sampled).

Go Implementation – sharded LRU with mutex per shard

Production cache internals. Sharded to kill lock contention; each shard = LRU (map + intrusive doubly-linked list). This is the hot path.

```
package cache

import (
    "container/list"
    "hash/fnv"
    "sync"
    "time"
)

// entry – LRU list element లో నిల్వ.
type entry struct {
    key    string
    value  []byte
    expiry int64 // unix nanos; 0 = no TTL
}

// shard – ఒక LRU cache slice. సొంత mutex ⇒ మిగతా shards తో lock contend కాదు.
type shard struct {
    mu      sync.Mutex
    ll      *list.List           // front = MRU, back = LRU
    items   map[string]*list.Element // key → list element (O(1) lookup)
    capacity int                  // ఈ shard లో గరిష్ట entries
}

func newShard(cap int) *shard {
    return &shard{ll: list.New(), items: make(map[string]*list.Element),
        capacity: cap}
}

func (s *shard) get(key string, now int64) ([]byte, bool) {
    s.mu.Lock()
    defer s.mu.Unlock()

    el, ok := s.items[key]
    if !ok {
        return nil, false // MISS
    }
    e := el.Value.(*entry)
    if e.expiry != 0 && e.expiry <= now { // lazy TTL: expired → evict
        s.removeElement(el)
        return nil, false
    }
    s.ll.MoveToFront(el) // accessed → most-recently-used
    return e.value, true
}
```

```

}

func (s *shard) set(key string, value []byte, ttl time.Duration, now int64) {
    s.mu.Lock()
    defer s.mu.Unlock()

    var exp int64
    if ttl > 0 {
        exp = now + ttl.Nanoseconds()
    }
    if el, ok := s.items[key]; ok { // update existing → front కి
        e := el.Value.(*entry)
        e.value, e.expiry = value, exp
        s.ll.MoveToFront(el)
        return
    }
    // కొత్త entry → front లో push
    el := s.ll.PushFront(&entry{key: key, value: value, expiry: exp})
    s.items[key] = el

    // capacity దాటితే LRU (list back) evict
    if s.ll.Len() > s.capacity {
        s.removeElement(s.ll.Back())
    }
}

func (s *shard) del(key string) {
    s.mu.Lock()
    defer s.mu.Unlock()
    if el, ok := s.items[key]; ok {
        s.removeElement(el)
    }
}

// removeElement – list + map రెండింటి నుండి తొలగించు (lock ఇప్పటికే పట్టుకున్నాం).
func (s *shard) removeElement(el *list.Element) {
    s.ll.Remove(el)
    delete(s.items, el.Value.(*entry).key)
}

// ---- ShardedCache: N shards, key → shard by hash ----

type ShardedCache struct {
    shards []*shard
    shardMask uint32 // N-1 (N = power of 2 → mask = fast modulo)
}

// New – shardCount MUST be power of 2 (mask trick). totalCap అన్ని shards మధ్య పంచు.
func New(shardCount, totalCapacity int) *ShardedCache {
    c := &ShardedCache{
        shards: make([]*shard, shardCount),
        shardMask: uint32(shardCount - 1),
    }
    per := totalCapacity / shardCount
    for i := range c.shards {
        c.shards[i] = newShard(per)
    }
    return c
}

```

```

// shardFor - key → shard. fnv hash + mask (contention split).
func (c *ShardedCache) shardFor(key string) *shard {
    h := fnv.New32a()
    _, _ = h.Write([]byte(key))
    return c.shards[h.Sum32()&c.shardMask]
}

func (c *ShardedCache) Get(key string) ([]byte, bool) {
    return c.shardFor(key).get(key, time.Now().UnixNano())
}
func (c *ShardedCache) Set(key string, value []byte, ttl time.Duration) {
    c.shardFor(key).set(key, value, ttl, time.Now().UnixNano())
}
func (c *ShardedCache) Delete(key string) { c.shardFor(key).del(key) }

// ---- Active TTL sweeper (background) - expired entries ని lazily కాక proactively
// తీసేయి ----
func (c *ShardedCache) StartSweeper(interval time.Duration, stop <-chan struct{}) {
    ticker := time.NewTicker(interval)
    go func() {
        defer ticker.Stop()
        for {
            select {
            case <-ticker.C:
                now := time.Now().UnixNano()
                for _, s := range c.shards { // ఒక్కో shard ని విడిగా lock
                    (అన్నీ ఒకేసారి కాదు)
                    s.mu.Lock()
                    for el := s.ll.Back(); el != nil; { // back =
                        LRU/పాతవి ముందు
                        e := el.Value.(*entry)
                        prev := el.Prev()
                        if e.expiry != 0 && e.expiry <= now {
                            s.removeElement(el)
                        }
                        el = prev
                    }
                    s.mu.Unlock()
                }
            case <-stop:
                return
            }
        }
    }()
}

```

Go ఎందుకు perfect: sharded mutex = Go concurrency idiom - 256 shards ⇒ ఒక్కో shard lock మిగతా వాటిని block చేయదు ⇒ 5M ops/sec scale. `container/list` = $O(1)$ LRU (map+list). Power-of-2 shards + `&mask` = branch-free fast routing. Sweeper = ఒక్కో shard విడిగా lock (అన్నీ ఒకేసారి freeze కాదు). Client-side consistent hashing (§3 toolkit) node routing.

Bottlenecks & Trade-offs

- **Lock contention:** single mutex = bottleneck $\Rightarrow$ **sharding** (256 shards). Trade-off: shards ఎక్కువ = memory overhead, hot key ఇంకా ఒక shard లో.
- **Hot key:** ఒక్క key huge QPS $\Rightarrow$ ఒక shard hot (sharding help చేయదు!). Solution: client-side local cache, key replication, request coalescing.
- **Thundering herd / cache stampede:** hot key expire $\Rightarrow$ అందరూ ఒకేసారి DB hit. Solution: **singleflight** (Go golang.org/x/sync/singleflight - ఒక్క DB call, మిగతా callers share), లేదా probabilistic early refresh.
- **Eviction policy:** LRU simple కానీ scan-resistant కాదు (bulk read cold data ని hot evict చేస్తుంది). LFU/W-TinyLFU better కానీ complex.
- **Consistency:** cache-aside $\Rightarrow$ DB update అయినా cache stale. TTL + explicit invalidation. Trade-off: freshness vs hit ratio.
- **Cold start:** node restart $\Rightarrow$ empty cache $\Rightarrow$ DB hammered. Gradual warm-up, replication.

Key Points

- **Sharded LRU (per-shard mutex)** $\Rightarrow$ lock contention split (256 $\times$), millions ops/sec.
- **LRU O(1)** = hash map + doubly-linked list (`container/list`); MoveToFront on access, evict tail.
- **TTL:** lazy (on Get) + active (sweeper goroutine).
- **Cache stampede** $\Rightarrow$ singleflight/coalescing; **hot key** $\Rightarrow$ client local cache/replication.
- **Cache-aside** pattern; consistency = TTL + invalidation (freshness vs hit ratio).
- **Client-side consistent hashing** for node routing (shared-nothing, memcached model).
- Go: sharded mutex, power-of-2 mask routing, `container/list` .

15. Payment / Ledger System

Real-life Analogy: బ్యాంకు పాస్‌బుక్ (double-entry). ప్రతి లావాదేవీకి రెండు entries - ఒక ఖాతా నుండి తీసేస్తారు (debit), మరో ఖాతాలో వేస్తారు (credit). రెండూ కలిపి ఎప్పుడూ zero (money create అవదు, transfer మాత్రమే). బ్యాంకు ఒకే transfer ని రెండుసార్లు process చేయకూడదు (idempotency) - నీ ₹100 రెండుసార్లు కట్ కాకూడదు. లెక్క ఎప్పుడూ సరిపోవాలి - "డబ్బు ఎక్కడికీ మాయం కాకూడదు, పుట్టకూడదు."

Requirements (Functional + Non-functional)

Functional:

- Account balances, **money transfer** (A $\rightarrow$ B)
- **Transaction history** (audit trail, immutable ledger)
- Multi-step payments (saga: reserve $\rightarrow$ charge $\rightarrow$ confirm)

Non-functional:

- **Correctness above all** - money lost/created కాకూడదు (strong consistency)
- **Idempotency** - retry/duplicate $\Rightarrow$ ఒక్కసారే charge (double-charge అపరాధం)
- **Exactly-once effect** - network retry అయినా balance ఒక్కసారే మారాలి
- **Durability** - committed transaction ఎప్పటికీ lost కాకూడదు
- Auditability (regulatory)

Estimation (QPS, storage)

Transactions: 10M/day → ~115/sec average, peak ~5K/sec (sale events)
 → payment QPS moderate; correctness > throughput (unlike feed/cache)
 Ledger entries: double-entry → 2x rows/txn → 20M rows/day → append-only, immutable
 Storage/entry: ~200 bytes → 20M × 200B = 4 GB/day → SQL (ACID), partition by time
 Consistency : STRONG (SQL transactions, serializable/repeatable-read) – eventual ಽಢ!

Design driver: correctness/consistency ⇒ **SQL + ACID transactions** (NoSQL eventual consistency). Idempotency ⇒ **idempotency key + unique constraint**. Multi-service ⇒ **saga** (distributed transaction). Audit ⇒ **immutable double-entry ledger**.

API Design

```
POST /api/v1/transfers
Header: Idempotency-Key: <client-uuid>           ← exactly-once కీలకం
Body: { "from":"acc_1", "to":"acc_2", "amount":10000, "currency":"INR" }
→ 201 { transferId, status:"completed" }
→ (అదే key మళ్ళీ) → 200 same result (re-execute కాదు!)
```

```
GET /api/v1/accounts/{id}/balance → { balance, asOf }
GET /api/v1/accounts/{id}/ledger → immutable entries (audit)
```

Idempotency-Key = payment API యొక్క గుండె. Client retry చేసినా (timeout, network) same key $\Rightarrow$ server మళ్ళీ charge చేయదు, మొదటి result తిరిగి ఇస్తుంది.

Data Model

```

accounts:      accId, balance, currency, version (optimistic lock)
ledger_entries: entryId, txnId, accId, direction(DEBIT/CREDIT), amount, createdAt
                (APPEND-ONLY, immutable - ఎప్పుడూ UPDATE/DELETE కాదు; audit)
transfers:     transferId, idempotencyKey(UNIQUE), from, to, amount, status, result
idempotency:    idempotencyKey → (status, response) [UNIQUE constraint = dedup]

INVARIANT: ఒక్కో txnId కి  $\Sigma(\text{DEBIT}) = \Sigma(\text{CREDIT})$  (double-entry; money conserved)

```

High-level Architecture

Client → Payment API (Go) → [Idempotency check: key ఇదివరకే ఉందా?]
 |
 └─ ఉంది → stored result తిరిగి ఇవ్వ (NO re-charge)


```

└─ కొత్తది → కింద process
▼
SQL Transaction (ACID, serializable):
BEGIN
  lock from-account (SELECT FOR UPDATE / optimistic version)
  check balance >= amount
  debit from (balance -= amount)  ┌ atomic
  credit to  (balance += amount) ┤ (both or neither)
  ledger: 2 immutable entries (DEBIT + CREDIT)
  store idempotency result
COMMIT
|
Multi-service payment: SAGA (reserve → charge PSP → confirm; fail →
compensate/refund)

```

Deep Dive – Idempotency + double-entry + exactly-once + saga

Idempotency (exactly-once effect):

- Client **Idempotency-Key** (UUID) పంపుతాడు. Server ఆ key ని **unique constraint** తో store చేస్తుంది.
- మొదటిసారి: process + result store. మళ్ళీ same key: **stored result తిరిగి ఇవ్వు** (re-process కాదు).
- Race (రెండు requests same key ఏకకాలంలో): unique constraint ⇒ ఒక్కటే insert గెలుస్తుంది; రెండోది conflict ⇒ wait/return first.
- ఇది **at-least-once delivery** ని **exactly-once effect** గా మారుస్తుంది.

Double-entry (correctness invariant):

- ప్రతి transfer = 2 ledger entries (source DEBIT + destination CREDIT), same txnId, **$\Sigma \text{debit} == \Sigma \text{credit}$** .
- Ledger **append-only/immutable** - correction ⇒ కొత్త reversing entry (పాతది edit కాదు). Audit trail intact.
- Balance = derived (Σ entries) లేదా cached column (reconciled).

Atomicity – single DB: SQL transaction (`BEGIN...COMMIT`) both updates atomic. `SELECT FOR UPDATE` (pessimistic) లేదా `version` column (optimistic lock) ⇒ concurrent transfer race నివారణ.

Saga – multi-service (distributed): transfer అనేక services span చేస్తే (wallet, PSP, ledger) single DB transaction అసాధ్యం. **Saga** = sequence of local transactions + **compensating actions** (fail అయితే rollback). ఉదా: reserve funds → charge PSP → confirm. Charge fail ⇒ **release reservation** (compensate). Eventual consistency కానీ correct.

Go Implementation – idempotent transfer handler

Payment handler: idempotency check → atomic double-entry SQL transaction → store result.
Correctness-first, retry-safe.

```

package payment

import (

```

```

    "context"
    "database/sql"
    "encoding/json"
    "errors"
    "net/http"
)

var (
    ErrInsufficientFunds = errors.New("insufficient funds")
    ErrDuplicate          = errors.New("duplicate idempotency key in-flight")
)

type TransferRequest struct {
    IdempotencyKey string
    From, To       string
    Amount         int64 // MINOR units (paise/cents) – float ఎప్పుడూ money కి కాదు!
    Currency       string
}

type TransferResult struct {
    TransferID string `json:"transferId"`
    Status     string `json:"status"`
}

type Service struct {
    db *sql.DB
}

// Transfer – idempotent + atomic double-entry. Retry-safe (exactly-once effect).
func (s *Service) Transfer(ctx context.Context, req TransferRequest) (TransferResult, error) {
    // STEP 1: idempotency fast-path – ఈ key ఇదివరకే complete అయిందా?
    if res, ok, err := s.lookupIdempotent(ctx, req.IdempotencyKey); err != nil {
        return TransferResult{}, err
    } else if ok {
        return res, nil // NO re-charge – stored result తిరిగి ఇవ్వు
    }

    // STEP 2: atomic transaction. Serializable = strongest (money correctness).
    tx, err := s.db.BeginTx(ctx, &sql.TxOptions{Isolation:
sql.LevelSerializable})
    if err != nil {
        return TransferResult{}, err
    }
    defer func() { _ = tx.Rollback() }() // commit కాకపోతే auto-rollback (safety)

    // 2a: idempotency row ని insert (UNIQUE constraint). Race లో ఒక్కటే గెలుస్తుంది.
    transferID, err := s.claimIdempotencyKey(ctx, tx, req)
    if err != nil {
        return TransferResult{}, err // duplicate in-flight → retry/wait
    }

    // 2b: source account ని lock + balance check (SELECT ... FOR UPDATE)
    var balance int64
    err = tx.QueryRowContext(ctx,
        `SELECT balance FROM accounts WHERE acc_id=$1 FOR UPDATE`, req.From,
    ).Scan(&balance)
    if err != nil {
        return TransferResult{}, err
    }
}

```

```

    }
    if balance < req.Amount {
        // insufficient → transfer FAILED no record (idempotent: retry కూడా
fail చూస్తుంది)
        _ = s.recordFailed(ctx, tx, transferID, ErrInsufficientFunds)
        if cErr := tx.Commit(); cErr != nil {
            return TransferResult{}, cErr
        }
        return TransferResult{}, ErrInsufficientFunds
    }

    // 2c: DEBIT source + CREDIT destination – both or neither (atomic)
    if _, err = tx.ExecContext(ctx,
        `UPDATE accounts SET balance = balance - $1 WHERE acc_id=$2`,
req.Amount, req.From); err != nil {
        return TransferResult{}, err
    }
    if _, err = tx.ExecContext(ctx,
        `UPDATE accounts SET balance = balance + $1 WHERE acc_id=$2`,
req.Amount, req.To); err != nil {
        return TransferResult{}, err
    }

    // 2d: DOUBLE-ENTRY ledger – 2 immutable rows, Σdebit == Σcredit
    if _, err = tx.ExecContext(ctx,
        `INSERT INTO ledger_entries (txn_id, acc_id, direction, amount)
VALUES ($1,$2,'DEBIT',$3), ($1,$4,'CREDIT',$3)`,
transferID, req.From, req.Amount, req.To); err != nil {
        return TransferResult{}, err
    }

    // 2e: idempotency result ని store (same tx లో – key+result atomic)
    result := TransferResult{TransferID: transferID, Status: "completed"}
    if err = s.storeResult(ctx, tx, req.IdempotencyKey, result); err != nil {
        return TransferResult{}, err
    }

    // STEP 3: COMMIT – ఇక్కడ వరకు ఏదైనా fail అయితే మొత్తం rollback (money safe)
    if err = tx.Commit(); err != nil {
        return TransferResult{}, err // COMMIT fail → client retry
(idempotency వల్ల safe)
    }
    return result, nil
}

// ---- HTTP handler ----
func (s *Service) HandleTransfer(w http.ResponseWriter, r *http.Request) {
    var body struct {
        From, To, Currency string
        Amount              int64
    }
    _ = json.NewDecoder(r.Body).Decode(&body)

    key := r.Header.Get("Idempotency-Key")
    if key == "" {
        http.Error(w, "Idempotency-Key required", http.StatusBadRequest)
        return
    }
}

```

```

    res, err := s.Transfer(r.Context(), TransferRequest{
        IdempotencyKey: key, From: body.From, To: body.To,
        Amount: body.Amount, Currency: body.Currency,
    })
    switch {
    case errors.Is(err, ErrInsufficientFunds):
        http.Error(w, "insufficient funds", http.StatusPaymentRequired)
    case err != nil:
        http.Error(w, "transfer failed", http.StatusInternalServerError)
    default:
        w.WriteHeader(http.StatusCreated)
        _ = json.NewEncoder(w).Encode(res)
    }
}

// ---- helpers (skeleton; claimIdempotencyKey UNIQUE constraint మీద ఆధారపడుతుంది) --
--
func (s *Service) lookupIdempotent(ctx context.Context, key string) (TransferResult,
bool, error) {
    return TransferResult{}, false, nil
}
func (s *Service) claimIdempotencyKey(ctx context.Context, tx *sql.Tx, req
TransferRequest) (string, error) {
    // INSERT INTO idempotency(key,...) - UNIQUE(key). conflict -> ErrDuplicate.
    return "txn_generated_id", nil
}
func (s *Service) recordFailed(ctx context.Context, tx *sql.Tx, id string, cause
error) error { return nil }
func (s *Service) storeResult(ctx context.Context, tx *sql.Tx, key string, r
TransferResult) error {
    return nil
}
}

```

Go ఎందుకు perfect: database/sql native transactions (BeginTx isolation levels) - serializable కి money correctness. defer tx.Rollback() = commit కాకపోతే auto-safe (Go idiom, leak లేదు). context deadline = hung DB call cancel. **Amount int64 (minor units)** - float64 money కి ఎప్పుడూ కాదు (rounding errors). Idempotency = unique constraint + Go error handling ⇒ retry-safe exactly-once.

Bottlenecks & Trade-offs

- **Correctness vs throughput:** serializable isolation = slow (locks) కానీ payment కి non-negotiable. Feed/cache లా eventual consistency ఇక్కడ ఆమోదయోగ్యం కాదు.
- **Hot account:** popular merchant account ⇒ FOR UPDATE lock contention. Solution: sharded sub-accounts + periodic aggregation, లేదా optimistic retry.
- **Idempotency store:** key lookup ప్రతి request. Same tx లో claim ⇒ atomic కానీ DB load. TTL (keys forever ఉంచకూడదు).
- **Distributed (saga) complexity:** compensations partial failure ⇒ complex. Compensations కూడా idempotent + retryable అయ్యిందాలి. Trade-off: 2PC (blocking, strong) vs saga (available, eventual).

- **Exactly-once myth:** true exactly-once *delivery* అసాధ్యం (distributed); exactly-once *effect* = idempotency తో సాధించేది.
- **Audit/immutability:** ledger append-only $\Rightarrow$ storage పెరుగుతుంది; time-partition + archival.

Key Points

- **Idempotency-Key + unique constraint** $\Rightarrow$ at-least-once delivery $\rightarrow$ exactly-once *effect* (double-charge నివారణ).
- **Double-entry, append-only ledger** $\Rightarrow \Sigma \text{debit} == \Sigma \text{credit}$ invariant, immutable audit trail.
- **ACID transaction (serializable)** $\Rightarrow$ atomic debit+credit; `SELECT FOR UPDATE` /version for race.
- **Money = int64 minor units, float64 ఎప్పుడూ కాదు.**
- **Saga + compensations** for multi-service (2PC blocking vs saga eventual).
- Correctness > throughput (payment; feed/cache లా eventual కాదు).
- Go: `database/sql` transactions, `defer Rollback`, `context` deadlines.

16. Collaborative Editing (Google Docs / Real-time Editor)

Real-life Analogy: ఒకే whiteboard మీద చాలామంది ఒకేసారి రాయడం. ఇద్దరు ఒకే షీట్ రాస్తే అక్షరాలు కలగలిసిపోతాయి - ఎవరి అక్షరం ఎక్కడ వెళ్ళాలో "rules" కావాలి. **OT** = ఇద్దరి మధ్య నిల్చున్న translator, ప్రతి move ని మిగతావాళ్ళ moves కి తగ్గట్టు re-adjust చేస్తాడు. **CRDT** = ప్రతి అక్షరానికి ఒక unique, మారని position ID ఇవ్వడం $\Rightarrow$ ఏ order లో వచ్చినా final text అందరికీ ఒక్కటే. "Merge conflict" అనేదే లేకుండా అందరూ converge అవుతారు.

Requirements (Functional + Non-functional)

Functional:

- ఒకే document ని అనేక users **ఏకకాలంలో** edit చేయగలగాలి
- మార్పులు **real-time** గా (<100ms) అందరికీ కనిపించాలి
- ఇతరుల **cursors / selections (presence)** కనిపించాలి
- **Offline edit** $\rightarrow$ online అయ్యాక auto-merge (conflict లేకుండా)
- **Version history + undo/redo**

Non-functional:

- **Convergence** (strong eventual consistency) - అన్ని replicas చివరికి ఒకే state
- **Low latency** real-time sync; **high availability**
- Per-doc concurrency సాధారణంగా చిన్నది (~10-100 active) కానీ total docs billions
- **Intention preservation** - user ఏం అనుకున్నాడో అదే జరగాలి (edits ఒకదాన్ని ఒకటి పాడుచేయకూడదు)

Estimation (QPS, storage)

100M DAU, active editing లో ~5 keystrokes/sec/user
ఒక్కో op ~50-100 bytes (insert char + position id)
Peak concurrent editors: ~1M → ~5M ops/sec (batch/debounce తో తగ్గించచ్చు)

Doc storage: snapshot + op log (compaction తో). ఒక doc ~10-100 KB.
CRDT metadata overhead: tombstones + position ids ⇒ raw text కంటే 2-5× (GC అవసరం)
Presence: ephemeral (persist చేయం), WebSocket మీద మాత్రమే

API Design

WebSocket (per-doc room, real-time bidirectional):
→ JOIN {docId, sinceVersion} (reconnect: missed ops పంపు)
→ OP {docId, op} (insert/delete op broadcast)
→ CURSOR {docId, userId, pos} (presence, ephemeral)
← OP {op, fromUser} (ఇతరుల ops)
← ACK {opId} (server received)

REST (snapshot + history):
GET /docs/{id} → {content, version} (join ముందు baseline)
GET /docs/{id}/history → [versions] (time-travel / undo)
POST /docs → {docId}

Data Model

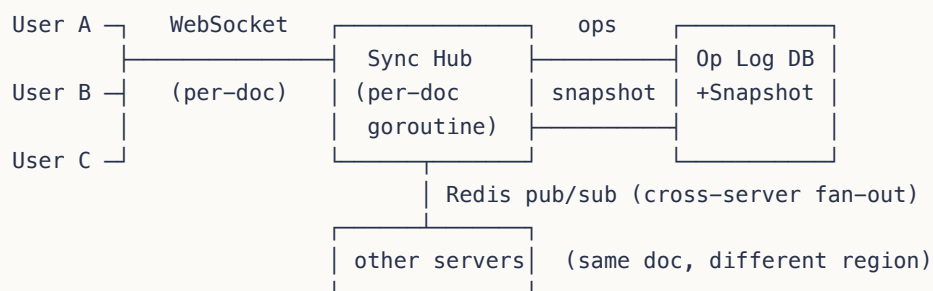
Document = ordered sequence of characters (each char = CRDT element).

CRDT element (fractional-index / RGA style):
{ Position, Value rune, Deleted bool(tombstone) }
Position = dense, totally-ordered key ⇒ రెండు positions మధ్య ఎప్పుడూ కొత్తది.

OT alternative:
op log: [{type: ins/del, pos, char, version}] + central server ordering.

Storage:
- Op log (append-only) + periodic snapshot (compaction).
- Presence: in-memory only (Redis pub/sub for cross-server fan-out).

High-level Architecture



Client-side CRDT ⇒ offline edit; reconnect అయితే ops exchange → converge.

Deep Dive – OT vs CRDT (conflict resolution యొక్క గుండె)

రెండు users ఒకే position లో ఏకకాలంలో type చేస్తే ఏం జరుగుతుంది? ఇదే core problem.

అంశం	OT (Operational Transformation)	CRDT (Conflict-free Replicated Data Type)
ఆలోచన	Op ని concurrent op కి తగ్గట్టు transform (index shift)	ప్రతి char కి unique immutable position $\Rightarrow$ merge automatic
Server పాత్ర	సాధారణంగా central server ordering ఇవ్వాలి	Server optional (P2P కూడా పని చేస్తుంది)
Complexity	Transform functions prove చేయడం కష్టం (TP1/TP2)	Data structure లోనే correctness; transform అవసరం లేదు
Metadata	తక్కువ (op = pos+char)	ఎక్కువ (position ids + tombstones)
ఎవరు వాడతారు	Google Docs (Jupiter/Wave), Etherpad	Figma, Yjs, Automerge, Riak

OT ఎలా: User A op `insert("X", pos=5)`, అదే సమయంలో User B op `insert("Y", pos=2)`. B op ముందు apply అయితే, A op ఇప్పుడు pos=5 కాదు, pos=6 (B ఒక char ముందు చేర్చాడు). Transform function `T(opA, opB)` ఈ shift adjust చేస్తుంది. అన్ని clients ఒకే transformed sequence apply చేస్తే converge. కానీ n-way transform + offline = combinatorial explosion $\Rightarrow$ central server తో linearize చేస్తారు.

CRDT ఎలా (నేను దీన్ని ఎంచుకుంటా): ప్రతి char కి **fractional position** (దట్టమైన key) ఇస్తాం. రెండు adjacent chars మధ్య insert = వాటి positions మధ్య కొత్త position generate చేయడం. రెండు concurrent inserts అదే చోట జరిగినా వేర్వేరు positions (site-id tie-break), అందరూ **sort by position** చేస్తే ఒకే order. `insert` commutative + idempotent $\Rightarrow$ ఏ order/ఎన్నిసార్లు apply చేసినా converge. **Delete = tombstone** (లేదా fractional-index లో నేరుగా remove - neighbor ని ID తో reference చేయం కాబట్టి safe).

ఎందుకు CRDT: central server లేకుండా offline-first + convergence proof data structure లోనే $\Rightarrow$ senior systems favorite. Trade-off: metadata/tombstone growth ($\Rightarrow$ GC).

Presence (cursors): ఇది CRDT కాదు - ephemeral. ప్రతి user cursor = **LWW** (last-write-wins), WebSocket మీద broadcast, disconnect అయితే expire. Document data తో కలపం.

Go Implementation – fractional-index sequence CRDT + sync hub

`between(lo, hi)` రెండు position keys మధ్య కొత్త dense key ఇస్తుంది - ఇదే CRDT గుండె. `integrate` sorted insert (commutative + idempotent). Central ordering అవసరం లేదు.

```
package crdt

import (
    "bytes"
    "sort"
    "strings"
)
```

```

// Position = dense, totally-ordered key. రెండు positions మధ్య ఎప్పుడూ కొత్తది
// generate చేయొచ్చు (fractional indexing). Site = concurrent-insert tie-break.
type Position struct {
    Path []byte // base-256 "fraction"; lexicographic compare
    Site uint64
}

func (a Position) Less(b Position) bool {
    if c := bytes.Compare(a.Path, b.Path); c != 0 {
        return c < 0
    }
    return a.Site < b.Site // సమాన path => deterministic tie-break => convergence
}

func samePos(a, b Position) bool {
    return a.Site == b.Site && bytes.Equal(a.Path, b.Path)
}

// between - lo, hi paths మధ్య ఒక path (strictly between). lo=nil => start, hi=nil =>
// end.
// రెండు distinct positions కి పని చేస్తుంది (common case). ఒకే gap లో concurrent
// inserts -> Site tie-break; production LSEQ/Logoot site ని path లో embed చేస్తుంది.
func between(lo, hi []byte) []byte {
    var out []byte
    for i := 0; ; i++ {
        var l byte = 0
        if i < len(lo) {
            l = lo[i]
        }
        var h byte = 255
        if i < len(hi) {
            h = hi[i]
        }
        if l+1 < h { // మధ్యలో room ఉంది
            out = append(out, l+(h-l)/2)
            return out
        }
        out = append(out, l) // room లేదు -> లోపలికి descend (denser)
    }
}

// Char = ఒక్క character. Position absolute key (neighbor reference కాదు) =>
// concurrent delete జరిగినా మన insert position valid.
type Char struct {
    Pos Position
    Value rune
}

type OpType int

const (
    Insert OpType = iota
    Delete
)

type Op struct {
    Type OpType
    Char Char
}

```



```

}

// Doc = Position ప్రకారం sorted characters. Replica-per-user.
type Doc struct {
    site uint64
    chars []Char
}

func NewDoc(site uint64) *Doc { return &Doc{site: site} }

// LocalInsert – visual index i వద్ద rune insert; broadcast చేయాల్సిన Op return.
func (d *Doc) LocalInsert(i int, v rune) Op {
    var lo, hi []byte
    if i > 0 {
        lo = d.chars[i-1].Pos.Path
    }
    if i < len(d.chars) {
        hi = d.chars[i].Pos.Path
    }
    ch := Char{Pos: Position{Path: between(lo, hi), Site: d.site}, Value: v}
    d.integrate(ch)
    return Op{Type: Insert, Char: ch}
}

// LocalDelete – visual index i వద్ద char delete; Op return.
func (d *Doc) LocalDelete(i int) Op {
    ch := d.chars[i]
    d.remove(ch.Pos)
    return Op{Type: Delete, Char: ch}
}

// Apply – remote op apply. Commutative + idempotent ⇒ ఏ order/duplicate అయినా converge.
func (d *Doc) Apply(op Op) {
    switch op.Type {
    case Insert:
        d.integrate(op.Char)
    case Delete:
        d.remove(op.Char.Pos)
    }
}

// integrate – Pos ప్రకారం sorted position లో insert (binary search). Duplicate ⇒ skip.
func (d *Doc) integrate(ch Char) {
    idx := sort.Search(len(d.chars), func(k int) bool {
        return !d.chars[k].Pos.Less(ch.Pos) // first pos >= ch.Pos
    })
    if idx < len(d.chars) && samePos(d.chars[idx].Pos, ch.Pos) {
        return // ఇప్పటికే ఉంది → idempotent
    }
    d.chars = append(d.chars, Char{})
    copy(d.chars[idx+1:], d.chars[idx:])
    d.chars[idx] = ch
}

func (d *Doc) remove(p Position) {
    idx := sort.Search(len(d.chars), func(k int) bool {
        return !d.chars[k].Pos.Less(p)
    })
}

```

```

        if idx < len(d.chars) && samePos(d.chars[idx].Pos, p) {
            d.chars = append(d.chars[:idx], d.chars[idx+1:]...)
        }
    }

    func (d *Doc) String() string {
        var b strings.Builder
        for _, c := range d.chars {
            b.WriteRune(c.Value)
        }
        return b.String()
    }
}

```

Sync hub (per-doc WebSocket fan-out): Chat §6 hub pattern నే reuse - ఒక్కో doc కి ఒక "room", op వస్తే మిగతా clients కి broadcast. Ops idempotent + commutative కాబట్టి ordering/dedup గురించి hub worry అవ్వల్సిన అవసరం లేదు.

```

type Client struct{ send chan Op }

// DocHub - ఒక document కి subscribers. Single goroutine owns state (no locks).
type DocHub struct {
    subscribe   chan *Client
    unsubscribe chan *Client
    broadcast   chan Op
    clients     map[*Client]bool
}

func NewDocHub() *DocHub {
    return &DocHub{
        subscribe:   make(chan *Client),
        unsubscribe: make(chan *Client),
        broadcast:   make(chan Op, 256),
        clients:     map[*Client]bool{},
    }
}

func (h *DocHub) Run() {
    for {
        select {
        case c := <-h.subscribe:
            h.clients[c] = true
        case c := <-h.unsubscribe:
            delete(h.clients, c)
            close(c.send)
        case op := <-h.broadcast:
            for c := range h.clients {
                select {
                case c.send <- op: // fan-out
                default: // slow client → drop (backpressure);
                }
            }
        }
    }
}

reconnect లో resync

```

Go ఎందుకు perfect: per-doc hub = **single goroutine owns state** $\Rightarrow$ mutex అవసరం లేదు (share memory by communicating). CRDT `integrate` binary-search sorted insert - lock-free per-replica (ఒక్కో client తన Doc). `select { case send: default: drop }` = slow client backpressure. Millions of docs = millions of cheap goroutines (Go stack few KB).

Bottlenecks & Trade-offs

- **CRDT metadata growth:** tombstones + position ids raw text కంటే చాలా ఎక్కువ $\Rightarrow$ **compaction/GC** (deleted tombstones ని causal stability తర్వాత purge) లేదా periodic snapshot.
- **Interleaving anomaly:** రెండు users ఏకకాలంలో వేర్వేరు words type చేస్తే characters interleave అవ్వచ్చు (CRDT known issue). Word/block-level granularity తగ్గిస్తుంది.
- **OT vs CRDT:** OT metadata తక్కువ కానీ central server + transform proofs కష్టం; CRDT offline-first + provable కానీ memory ఎక్కువ. Google Docs = OT (bandwidth), Figma/Yjs = CRDT.
- **Large docs:** పెద్ద document కి ప్రతి keystroke op = network chatty $\Rightarrow$ **batch/debounce** (50-100ms). Snapshot + op-log-since-snapshot join.
- **Presence scale:** cursors high-frequency $\Rightarrow$ throttle + Redis pub/sub cross-server; document state తో కలపకూడదు.
- **Undo/redo:** distributed లో "నా చివరి op" undo = inverse op generate + broadcast (global undo కాదు).

Key Points

- **CRDT** = ప్రతి char కి unique immutable **position** $\Rightarrow$ merge commutative + idempotent $\Rightarrow$ **convergence** (conflict-free). Central ordering అవసరం లేదు.
- **OT** = op ని concurrent op కి **transform** (index shift); central server linearize; Google Docs వాడేది.
- **Fractional indexing:** రెండు positions మధ్య ఎప్పుడూ కొత్త dense key $\Rightarrow$ neighbor ని ID తో reference చేయం $\Rightarrow$ concurrent delete safe.
- **Presence (cursors)** = ephemeral **LWW**, document CRDT తో కలపొద్దు.
- Trade-off: CRDT memory/tombstone growth $\Rightarrow$ GC/compaction; batch keystrokes; interleaving granularity.
- Go: per-doc **hub goroutine** (lock-free), `sort.Search` sorted insert, `select-default` backpressure.

17. Web Crawler (Search Engine Spider)

Real-life Analogy: సాలీడు web మీద దారం పట్టుకుని ప్రయాణం. ఒక్కో page లోని links పట్టుకుని కొత్త pages కి వెళ్తుంది (crawl). కానీ మంచి librarian లా - ఒకే book మళ్ళీ మళ్ళీ చదవదు (**visited set**), ఒకే publisher (domain) ని ఒకేసారి వందల requests తో విసిగించదు (**politeness**), "please don't enter" board (robots.txt) ని గౌరవిస్తుంది. లక్ష్యం: billions of pages ని efficient గా, polite గా discover చేయడం.

Requirements (Functional + Non-functional)

Functional:

- **Seed URLs** నుండి మొదలుపెట్టి pages download చేయడం
- Page లోని **links extract** చేసి recursively crawl
- **Dedup** - ఒకే URL/content మళ్ళీ crawl చేయకూడదు
- **robots.txt** + `crawl-delay` గౌరవించడం (politeness)
- Content ని **store** (indexing కి), **re-crawl** for freshness

Non-functional:

- **Scale:** billions of pages; **distributed** workers
- **Politeness:** ఒక domain ని overload చేయకూడదు (per-host rate limit)
- **Fault-tolerant:** worker/crash అయితే resume (durable frontier)
- **Trap-avoidance:** infinite URL spaces, spider traps, duplicate content
- **Extensible:** వేర్వేరు content types / parsers

Estimation (QPS, storage)

1B pages / month $\rightarrow 1B / (30 \times 86400) \approx 386$ pages/sec sustained; peak $\times 3 \approx 1200$ /sec
Avg page ~ 100 KB $\rightarrow 1B \times 100KB = 100$ TB/month raw (compressed $\sim 20-30$ TB)

Seen-set (dedup):

Exact set: $1B \text{ URLs} \times \sim 50B = 50 \text{ GB}$ (RAM/node కి పెద్దది)

Bloom filter: $1B \times \sim 10 \text{ bits (1\% FP)} \approx 1.25 \text{ GB} \rightarrow \text{RAM లో fit } \checkmark$

Frontier: billions of URLs $\rightarrow$ durable queue (Redis/Kafka), RAM కాదు

DNS: ప్రతి fetch కి resolve $\Rightarrow$ DNS cache లేకపోతే bottleneck

API Design

User-facing API కాదు (internal system). Core interfaces:

Enqueue(url, depth) $\rightarrow$ frontier లో add (dedup తర్వాత)
worker: Fetch $\rightarrow$ Parse $\rightarrow$ Extract $\rightarrow$ Enqueue(children)

Admin/control:

POST /seeds {urls} $\rightarrow$ seed inject

GET /stats	→ crawled, frontier size, QPS, errors
GET /robots/{host}	→ cached robots policy (debug)

Data Model

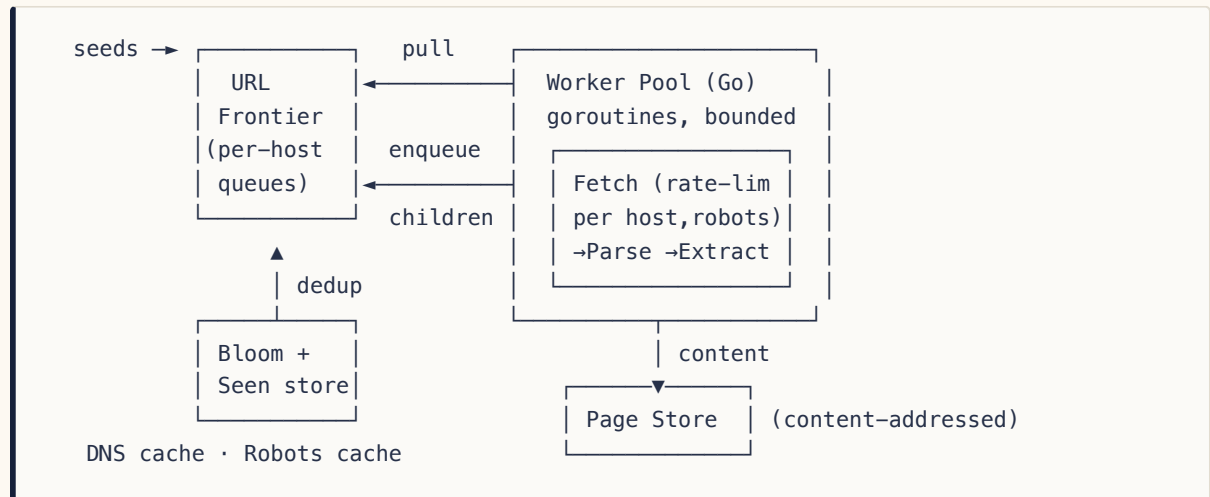
URL Frontier (priority queue, per-host):
 host → queue of {url, depth, priority} (politeness: per-host isolation)

Seen set (dedup):
 bloom filter (fast, probabilistic) + exact store (URL → crawl metadata)

Page store (content-addressed, \$19 style):
 contentHash → raw bytes ; url → {contentHash, lastCrawled, etag, status}

Robots cache:
 host → {rules, crawlDelay, fetchedAt} (TTL, re-fetch periodically)

High-level Architecture



Deep Dive – Politeness + Frontier + Dedup (crawler యొక్క గుండె)

1. Politeness (అతి ముఖ్యం): ఒక website ని second కి వందల requests కొడితే అది down అవుతుంది (unintentional DoS) + మనని ban చేస్తుంది. కాబట్టి:

- **Per-host rate limit** (ఉదా: max 1 req/sec/host, లేదా robots.txt crawl-delay)
- ఒక host ని ఒకే worker handle చేసేలా partition (host hash → worker) ⇒ per-host state simple
- **robots.txt** fetch + cache + గౌరవించడం (disallowed paths skip)

2. Frontier design: ఇది priority queue - ఏ URL ముందు crawl చేయాలి?

- **BFS (default):** queue (FIFO) ⇒ broad coverage, links సహజంగా వేర్వేరు hosts కి వెళ్తాయి ⇒ politeness సులభం. Crawlers దీన్ని వాడతాయి.
- **DFS:** stack ⇒ ఒకే site లో లోతుగా ఇరుక్కుపోతుంది (spider trap risk), politeness కష్టం.
- **Priority:** PageRank/importance ఆధారంగా (important pages ముందు). Two-level: (host queue) + (per-host FIFO).

3. Dedup: ఒకే URL billions సార్లు కనిపిస్తుంది (ప్రతి page లో home link).

- **Bloom filter:** "ఇది ఇదివరకే చూశామా?" $O(1)$, 1.25GB కి 1B URLs. **False positive** (అరుదుగా కొత్త page skip) ఆమోదయోగ్యం; **false negative ఎప్పుడూ కాదు** (duplicate crawl అవదు).
- **URL normalization:** HTTP vs http, trailing /, fragment #, query param order $\Rightarrow$ ఒకే canonical form (లేకపోతే duplicate crawl).
- **Content dedup:** వేర్వేరు URLs ఒకే content (mirrors) $\Rightarrow$ content hash (§19).

4. Trap avoidance: infinite calendars (?date=...), session ids in URL $\Rightarrow$ **max depth**, max URLs/host, URL length limit.

Go Implementation – concurrent crawler (bounded pool + sync.Map + per-host rate limit)

Go crawler కి perfect fit: ఒక్కో fetch goroutine, **bounded worker pool** (politeness + resource control), sync.Map visited (concurrent dedup), golang.org/x/time/rate per-host limiter.

```
package crawler

import (
    "context"
    "io"
    "net/http"
    "net/url"
    "strings"
    "sync"
    "time"

    "golang.org/x/net/html"
    "golang.org/x/time/rate"
)

type task struct {
    url    string
    depth int
}

type Crawler struct {
    client    *http.Client
    workers   int
    maxDepth int
    perHost   rate.Limit // req/sec/host (politeness)
    burst     int

    seen    sync.Map // url(string)  $\rightarrow$  struct{} (concurrent dedup seen-set)
    hostLim sync.Map // host(string)  $\rightarrow$  *rate.Limiter (per-host politeness)
    tasks   chan task
    pending sync.WaitGroup // in-flight tasks (frontier empty detection)
    wg      sync.WaitGroup // worker lifetimes
}

func New(workers, maxDepth int, perHost rate.Limit) *Crawler {
    return &Crawler{
        client:    &http.Client{Timeout: 10 * time.Second},
        workers:   workers,
    }
}
```

```

        maxDepth: maxDepth,
        perHost: perHost,
        burst: 1,
    }
}

// Run – seeds నుండి crawl; frontier empty అయ్యాక return.
func (c *Crawler) Run(ctx context.Context, seeds ...string) {
    c.tasks = make(chan task, 4096)
    for i := 0; i < c.workers; i++ { // bounded pool
        c.wg.Add(1)
        go c.worker(ctx)
    }
    for _, s := range seeds {
        c.enqueue(task{url: s, depth: 0})
    }
    go func() { c.pending.Wait(); close(c.tasks) }() // frontier empty → workers
exit
    c.wg.Wait()
}

// enqueue – dedup (LoadOrStore, race-safe) తర్వాత frontier లో add.
func (c *Crawler) enqueue(t task) {
    if _, loaded := c.seen.LoadOrStore(t.url, struct{}{}); loaded {
        return // ఇదివరకే చూశాం (production: bloom filter + exact store)
    }
    c.pending.Add(1)
    // non-blocking dispatch: worker frontier లోకి enqueue చేసినా deadlock లేదు.
    // (production frontier = durable Redis/Kafka, in-memory channel కాదు.)
    go func() { c.tasks <- t }()
}

func (c *Crawler) worker(ctx context.Context) {
    defer c.wg.Done()
    for t := range c.tasks {
        c.process(ctx, t)
        c.pending.Done()
    }
}

func (c *Crawler) limiter(host string) *rate.Limiter {
    if l, ok := c.hostLim.Load(host); ok {
        return l.(*rate.Limiter)
    }
    l, _ := c.hostLim.LoadOrStore(host, rate.NewLimiter(c.perHost, c.burst))
    return l.(*rate.Limiter)
}

func (c *Crawler) process(ctx context.Context, t task) {
    u, err := url.Parse(t.url)
    if err != nil || (u.Scheme != "http" && u.Scheme != "https") {
        return
    }
    // politeness: per-host rate limit (robots.txt crawl-delay ఇక్కడ apply చేయాలి)
    if err := c.limiter(u.Host).Wait(ctx); err != nil {
        return // ctx cancelled
    }
    req, _ := http.NewRequestWithContext(ctx, http.MethodGet, t.url, nil)
    req.Header.Set("User-Agent", "GoCrawler/1.0")
}

```

```

    resp, err := c.client.Do(req)
    if err != nil {
        return
    }
    defer resp.Body.Close()
    if resp.StatusCode != http.StatusOK ||
        !strings.HasPrefix(resp.Header.Get("Content-Type"), "text/html") {
        return
    }
    // store content ఇక్కడ (content-addressed, §19)
    if t.depth >= c.maxDepth {
        return
    }
    for _, link := range extractLinks(resp.Body, u) {
        c.enqueue(task{url: link, depth: t.depth + 1})
    }
}

// extractLinks - HTML tokenize, <a href> extract, relative→absolute resolve,
normalize.
func extractLinks(r io.Reader, base *url.URL) []string {
    var links []string
    z := html.NewTokenizer(r)
    for {
        switch z.Next() {
        case html.ErrorToken:
            return links // EOF సహా
        case html.StartTagToken, html.SelfClosingTagToken:
            tok := z.Token()
            if tok.Data != "a" {
                continue
            }
            for _, a := range tok.Attr {
                if a.Key != "href" {
                    continue
                }
                ref, err := url.Parse(a.Val)
                if err != nil {
                    continue
                }
                abs := base.ResolveReference(ref) // relative →
absolute
                abs.Fragment = "" // #section drop
(normalize)

                if abs.Scheme == "http" || abs.Scheme == "https" {
                    links = append(links, abs.String())
                }
            }
        }
    }
}

```


Go ఎందుకు perfect: bounded worker pool (fixed N goroutines) = politeness + resource control (unbounded goroutines self-DoS). `sync.Map` = read-heavy concurrent dedup, `LoadOrStore` race-safe "మొదటిసారి?".
`golang.org/x/time/rate` per-host limiter + `sync.Map` = ఒక్కో host కి lazy politeness.
`context` = whole-crawl cancellation/timeout. `pending.Wait()` = frontier empty detection (elegant termination).

Bottlenecks & Trade-offs

- **DNS bottleneck:** ప్రతి fetch కి DNS resolve $\Rightarrow$ **DNS cache** (లేకపోతే throughput DNS కి bound). Async resolver.
- **Frontier durability:** in-memory channel crash లో lost $\Rightarrow$ production లో **Redis/Kafka durable frontier** (resume). పై code in-memory (demo).
- **Politeness vs throughput:** strict per-host limit $\Rightarrow$ ఒక giant site slow; కానీ ban risk. Balance: many hosts parallel, per-host serial.
- **Bloom false positive:** అరుదుగా real page skip (trade-off). False negative లేదు కాబట్టి duplicate crawl అవదు. Exact store backstop.
- **Spider traps:** infinite URL spaces $\Rightarrow$ max depth + per-host URL cap + pattern detection.
- **Duplicate content:** mirrors/canonical $\Rightarrow$ content-hash dedup (URL dedup సరిపోదు).
- **Freshness vs cost:** re-crawl ఎంత తరచుగా? change-rate estimate (news తరచుగా, archive అరుదుగా).

Key Points

- **Bounded worker pool** (fetchers) = politeness + resource control; unbounded = self-DoS.
- **Per-host rate limit + robots.txt/crawl-delay** = ఒక site ని overload చేయకూడదు (+ ban నివారణ).
- **Bloom filter dedup** (1B URLs ~1.25GB) - false positive OK, false negative ఎప్పుడూ కాదు. + **URL normalization**.
- **BFS (queue)** = broad coverage + natural politeness; DFS = trap risk. Priority = importance-first.
- **Durable frontier** (Redis/Kafka) = crash resume; **DNS cache** = throughput.
- Go: `sync.Map` visited, `x/time/rate` per-host, bounded goroutine pool, `context` cancellation, `pending.Wait()` termination.

18. Distributed Message Queue (Design Kafka)

Real-life Analogy: వరుస numbers ఉన్న notice board. Topic = ఒక notice board; partition = board మీద ఒక్క column. కొత్త notice ఎప్పుడూ **కింద** add అవుతుంది (append-only), దానికి వరుస number (**offset**). Readers (consumers) "నేను చివరిగా ఏ number చదివాను" గుర్తుంచుకుంటారు, అక్కడి నుండి కొనసాగుతారు. Board చదివాక కూడా కొంతకాలం notices ఉంచుతుంది (**retention**) ⇒ కొత్త reader పాతవి కూడా చదవగలడు, ఒకే notice ని అనేకమంది independent గా చదవగలరు.

Requirements (Functional + Non-functional)

Functional:

- **Publish** messages to a **topic**; topic ని **partitions** గా విభజన
- **Consumers** offset ఆధారంగా చదవడం; **consumer groups** (partition → ఒక్క consumer/group ⇒ parallelism)
- **Ordering** guarantee (partition లోపల మాత్రమే)
- **Retention** (time/size based); పాత messages delete/compact
- **Replication** (durability - broker down అయినా data ఉండాలి)

Non-functional:

- **High throughput** (millions msg/sec) - sequential disk writes + zero-copy
- **Durability** - ack తర్వాత message lost అవకూడదు
- **Horizontal scale** - brokers/partitions add చేయడం
- **Delivery semantics**: at-least-once, at-most-once, (near) **exactly-once**

Estimation (QPS, storage)

1M msg/sec, avg 1 KB → 1 GB/sec = 86 TB/day
Retention 7 days → ~600 TB (replication ×3 ⇒ ~1.8 PB)

Per-partition throughput ~10-50 MB/sec ⇒ 1 GB/sec కి ~20-100 partitions/topic
Partitions ఎక్కువ = parallelism ఎక్కువ కానీ metadata/leader-election overhead ఎక్కువ

Consumer: ఒక్క partition ఒక్క consumer/group ⇒ max parallelism = partition count
Offset store: consumer group → partition → committed offset (చిన్నది)

API Design

Producer:

Produce(topic, key, value) → (partition, offset)
key hash → partition (అదే key ⇒ అదే partition ⇒ per-key ordering)
acks: 0 (fire-forget) | 1 (leader) | all (ISR - durable)

Consumer:

Subscribe(topic, groupId)

Poll(maxRecords) → []Record (offset సుండి pull; long-poll)
 Commit(offset) (processed దాకా mark)
 Seek(partition, offset) (replay - వెనక్కి/ముందుకి)

Fetch (low-level): Fetch(topic, partition, fromOffset, maxBytes) → []Record

Data Model

Topic → [Partition 0, Partition 1, ... N-1]

Partition = append-only, ordered log (offset = index):

segment files (1GB each) → [record, record, ...]

Record: {offset, key, value, timestamp, crc}

Sparse index: offset → file byte position (binary search + scan)

Replication:

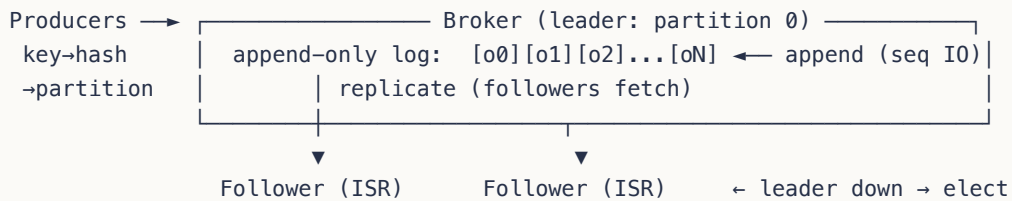
each partition: 1 leader + M followers (ISR = in-sync replicas)

high-water mark = అన్ని ISR లో replicate అయిన max offset (consumers దీని దాకా)

Consumer offsets: (group, topic, partition) → committed offset

(Kafka లో __consumer_offsets అనే internal topic)

High-level Architecture



Consumer Group A: C1←part0 C2←part1 (ఒక్కే partition ఒక్క consumer)

Consumer Group B: C1←part0,part1 (independent offsets, same data)

Controller (KRaft/ZooKeeper): metadata + leader election

Deep Dive – Log + Replication + Delivery Semantics (గుండె)

1. ఎందుకు append-only log ఇంత fast:

- **Sequential disk writes** = random కంటే 100-1000× fast (disk seek లేదు); SSD మీద కూడా sequential మేలు.
- **OS page cache** - writes/reads cache లో; app heap కాదు.
- **Zero-copy** (`sendfile`) - disk → socket నేరుగా, user-space copy లేదు ⇒ consumer read cheap.
- **Segments**: log ని 1GB files గా roll; retention = పాత segment file **delete** (individual message కాదు - $O(1)$).

2. Replication + durability: leader partition writes handle చేస్తుంది, followers **fetch** చేసి replicate చేస్తారు. **ISR** (in-sync replicas) = leader తో caught-up followers.

- **acks=all** ⇒ producer అన్ని ISR replicate అయ్యాక ack ⇒ durable (leader down అయినా data ఉంది).
- **High-water mark (HW):** consumers HW దాకా మాత్రమే చూస్తారు (un-replicated messages చూడరు) ⇒ leader fail అయినా consumed data lost అవుతుంది.
- **Leader election:** leader down ⇒ controller ISR నుండి కొత్త leader ఎన్నుకుంటుంది. **Unclean election** (ISR ఖాళీ ⇒ non-ISR leader) = availability కోసం data loss risk (trade-off).

3. Delivery semantics (interview favorite):

Semantic	ఎలా	Trade-off
At-most-once	commit ముందు process ⇒ crash ⇒ message lost	fast, loss OK అయితే
At-least-once	process తర్వాత commit ⇒ crash ⇒ reprocess (duplicate)	default; idempotent consumer కావాలి
Exactly-once	idempotent producer (producerId+seq dedup) + transactions (atomic write + offset commit)	costly కానీ correct

Interview line: "Exactly-once *delivery* distributed లో అసాధ్యం; Kafka ఇచ్చేది exactly-once *processing* = idempotent producer + transactional offset commit, లేదా consumer వైపు offset-based dedup."

4. Ordering: partition లోపల మాత్రమే guaranteed (single log). Global order అక్కరలేదు ⇒ scale. అదే key అదే partition ⇒ per-key order (ఉదా: ఒక user events ordered).

Go Implementation – append-only partitioned log + offset consumers

Kafka గుండె = per-partition append-only log + offset-based pull consumer. `sync.Cond` తో long-poll (కొత్త data కి consumer wait), `base` offset తో retention (front-trim).

```
package mq

import (
    "hash/fnv"
    "sync"
)

type Record struct {
    Offset int64
    Key    []byte
    Value  []byte
    TsUnix int64
}

// Partition = append-only, ordered log. Offset = position. ఒక partition = ordering unit.
type Partition struct {
    mu    sync.RWMutex
    recs []Record // production: segment files + sparse index + page cache
```

```

    base int64    // recs[0].Offset (retention front-trim వల్ల >0)
    next int64    // కేటాయించాల్సిన తదుపరి offset
    cond *sync.Cond
}

func NewPartition() *Partition {
    p := &Partition{}
    p.cond = sync.NewCond(&p.mu)
    return p
}

// Append – record ని log చివర add, offset return (leader మీద). Sequential write.
func (p *Partition) Append(key, val []byte, ts int64) int64 {
    p.mu.Lock()
    defer p.mu.Unlock()
    off := p.next
    p.recs = append(p.recs, Record{Offset: off, Key: key, Value: val, TsUnix:
ts})
    p.next++
    p.cond.Broadcast() // wait చేస్తున్న consumers ని wake (long-poll)
    return off
}

// Read – from offset నుండి గరిష్టంగా max records (offset-based fetch).
func (p *Partition) Read(from int64, max int) []Record {
    p.mu.RLock()
    defer p.mu.RUnlock()
    if from < p.base {
        from = p.base // పాత offset retention లో పోయింది → అందుబాటులో ఉన్నదాని
నుండి
    }
    if from >= p.next {
        return nil
    }
    end := from + int64(max)
    if end > p.next {
        end = p.next
    }
    out := make([]Record, end-from)
    copy(out, p.recs[from-p.base:end-p.base])
    return out
}

// TrimBefore – retention: minOff కంటే పాత records drop (segment delete స్ఫూర్తి).
func (p *Partition) TrimBefore(minOff int64) {
    p.mu.Lock()
    defer p.mu.Unlock()
    if minOff <= p.base || minOff > p.next {
        return
    }
    p.recs = p.recs[minOff-p.base:]
    p.base = minOff
}

// Consumer – ఒక partition ని offset ఆధారంగా చదువుతుంది. Offset **consumer వద్ద**
// (pull model, broker state కాదు) ⇒ replay సులభం (Seek వెనక్కి).
type Consumer struct {
    p      *Partition
    offset int64
}

```

```

}

func NewConsumer(p *Partition, from int64) *Consumer {
    return &Consumer{p: p, offset: from}
}

// Poll – long-poll: కొత్త data లేకపోతే wait; వచ్చిన records return + offset advance.
func (c *Consumer) Poll(max int) []Record {
    c.p.mu.Lock()
    for c.offset >= c.p.next { // కొత్తది లేదు → block
        c.p.cond.Wait()
    }
    c.p.mu.Unlock()

    recs := c.p.Read(c.offset, max)
    if n := len(recs); n > 0 {
        // process తర్వాత advance = at-least-once (crash → reprocess)
        c.offset = recs[n-1].Offset + 1
    }
    return recs
}

func (c *Consumer) Seek(off int64) { c.offset = off } // replay / skip

// Topic = partitions; key hash → partition (per-key ordering).
type Topic struct {
    parts []*Partition
}

func NewTopic(n int) *Topic {
    t := &Topic{parts: make([]*Partition, n)}
    for i := range t.parts {
        t.parts[i] = NewPartition()
    }
    return t
}

// Produce – key ఉంటే hash→partition (అదే key అదే partition); లేకపోతే partition 0.
func (t *Topic) Produce(key, val []byte, ts int64) (int, int64) {
    part := 0
    if key != nil {
        h := fnv.New32a()
        h.Write(key)
        part = int(h.Sum32() % uint32(len(t.parts)))
    }
    return part, t.parts[part].Append(key, val, ts)
}

func (t *Topic) Partition(i int) *Partition { return t.parts[i] }

// Assign – consumer group: p partitions ని n consumers కి పంచడం (range assignment).
// Rebalance: consumer join/leave అయితే మళ్ళీ assign.
func Assign(partitions, consumers int) map[int][]int {
    out := map[int][]int{}
    for p := 0; p < partitions; p++ {
        out[p%consumers] = append(out[p%consumers], p)
    }
}

```

Go ఎందుకు perfect: append = slice append (sequential); `sync.RWMutex` = multi-consumer concurrent reads + single writer. `sync.Cond` = long-poll (busy-loop కాదు) - కొత్త record వస్తే `Broadcast` తో consumers wake. Offset consumer వద్ద $\Rightarrow$ broker దాదాపు stateless, replay trivial. `hash/fnv` partitioner = per-key ordering. Real Kafka: segment files + `sendfile` zero-copy + Raft (KRaft) replication - ఇదే model, disk-backed.

Bottlenecks & Trade-offs

- **Hot partition:** skewed key (ఒక celebrity user) $\Rightarrow$ ఒక partition overload. Solution: better key, sub-partitioning.
- **acks=all latency vs durability:** all ISR wait = slow కానీ durable; acks=1 fast కానీ leader-crash లో loss. Trade-off per use-case.
- **ISR shrink:** slow follower ISR నుండి పడిపోతే durability తగ్గుతుంది; unclean election = availability కోసం data loss.
- **Rebalance storms:** consumer group churn $\Rightarrow$ frequent rebalance $\Rightarrow$ processing pause. Static membership / cooperative rebalance తగ్గిస్తుంది.
- **Too many partitions:** parallelism ఎక్కువ కానీ controller metadata + leader election + open files overhead. Balance.
- **Ordering vs parallelism:** global order కావాలంటే 1 partition (no parallelism); per-key order + scale = key partitioning.
- **Exactly-once cost:** transactions = 2-phase overhead $\Rightarrow$ throughput తగ్గుతుంది.

Key Points

- **Append-only log** (sequential IO + page cache + zero-copy) = extreme throughput; **segments** $\Rightarrow$ retention = file delete $O(1)$.
- **Partition = ordering + parallelism unit;** ordering partition లోపల మాత్రమే; key hash $\rightarrow$ partition = per-key order.
- **Offset consumer వద్ద** (pull) $\Rightarrow$ replay/multi-consumer trivial; **consumer group** = partition/consumer parallelism.
- **Replication (leader + ISR) + acks=all + high-water mark** = durability; unclean election = availability $\leftrightarrow$ loss trade-off.
- **At-least-once** (default, process $\rightarrow$ commit) + idempotency = **exactly-once effect**; true exactly-once delivery అసాధ్యం.
- Go: slice-append log, `sync.Cond` long-poll, `RWMutex` multi-consumer, `fnv` partitioner.

19. Distributed File Storage (Dropbox / S3 / Object Storage)

Real-life Analogy: contents fingerprint అధారంగా label చేసిన warehouse. Box మీద "ఎవరిది" అని కాదు, దాని contents యొక్క **fingerprint (hash)** label చేస్తాం. ఇద్దరు ఒకేలాంటి item తెస్తే **ఒక్కటే box** ఉంచి ఇద్దరికీ అదే claim ticket ఇస్తాం (**dedup**). ఒక file = order లో ఉన్న tickets (chunk hashes) జాబితా. Item మారితే ఏ tickets మారాయో అవి మాత్రమే కొత్తగా store (**delta sync**) $\Rightarrow$ bandwidth ఆదా.

Requirements (Functional + Non-functional)

Functional:

- పెద్ద files (GBs) **upload/download**; files ని **chunks** గా split
- ఒకేలాంటి chunks **dedup** (across files, across users)
- **Metadata** (file $\rightarrow$ chunk list, versions, tree)
- **Sync** across devices - **delta** (మారిన chunks మాత్రమే)
- **Sharing + presigned URLs; versioning**

Non-functional:

- **Durability** (11 nines) - replication / erasure coding
- **Scale** - exabytes, billions of objects
- **Bandwidth efficiency** - delta sync (whole file కాదు)
- **Availability** + metadata **consistency**

Estimation (QPS, storage)

500M users $\times$ avg 10 GB = 5 EB (exabytes)
Chunk size 4 MB $\rightarrow$ 5EB / 4MB $\approx$ 1.25 trillion chunks
Dedup $\sim$ 30% savings (identical files/chunks: OS files, shared docs)

Metadata: chunk hash 32B (SHA-256) + refcount; file manifest = [hashes]
1.25T chunks $\times$ $\sim$ 64B metadata $\approx$ 80 TB metadata $\rightarrow$ sharded DB

Durability: replication $\times$ 3 (5EB $\rightarrow$ 15EB) vs erasure coding ($\sim$ 1.4 $\times$, 5EB $\rightarrow$ 7EB)
 $\rightarrow$ erasure coding storage-efficient కానీ repair CPU/network ఎక్కువ
Bandwidth: delta sync $\Rightarrow$ upload = మారిన chunks మాత్రమే (edit కి full file కాదు)

API Design

File ops:
PUT /files/{path} (chunked upload) $\rightarrow$ {fileId, version}
GET /files/{path} $\rightarrow$ stream
GET /files/{path}/manifest $\rightarrow$ {chunks: [hash], size, version}

Chunk-level (dedup + delta):
HEAD /chunks/{hash} $\rightarrow$ 200 (ఉంది, skip) | 404 (కావాలి)


```
POST /chunks/missing {hashes} → [missing hashes] (batch dedup check)
PUT /chunks/{hash} (content-addressed upload)
```

Presigned (app server offload):

```
GET /presign?path=..&op=put&exp=.. → time-limited signed URL
(client నేరుగా blob store కి, app server bandwidth ఆదా)
```

Data Model

Chunk store (content-addressed, immutable):

SHA-256(content) → blob (అదే content ⇒ అదే key ⇒ ఒక్కసారే store)

File manifest (metadata DB - sharded by userId):

fileId → { name, size, version, mtime, chunks: [hash, hash, ...] }
(manifest = references మాత్రమే, content కాదు ⇒ చిన్నది)

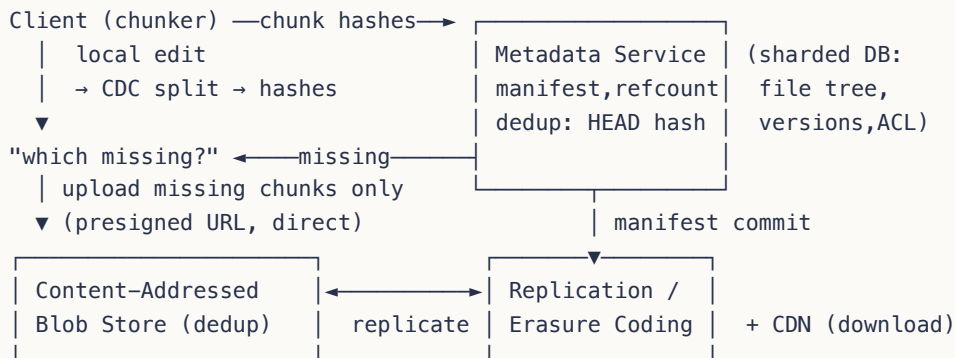
Chunk refcount (GC కి):

hash → count (0 అయితే unreferenced → garbage collect)

Version history: fileId → [version → manifest] (time-travel)

ACL / sharing: fileId → {owner, sharedWith[]}

High-level Architecture



Deep Dive – Chunking + Content-Addressing + Delta Sync (గుండె)

1. ఎందుకు chunk చేయాలి: (a) పెద్ద file resumable upload (chunk fail → అదే chunk retry), (b) dedup granularity, (c) delta sync (మారిన chunks మాత్రమే), (d) parallel upload.

2. Fixed vs Content-Defined Chunking (CDC) – ముఖ్య insight:

- **Fixed-size (4MB blocks):** simple. కానీ file మొదట్లో ఒక్క byte insert అయితే అన్ని subsequent chunks **shift** ⇒ అన్ని hashes మారతాయి ⇒ dedup పోతుంది (whole file re-upload)!
- **Content-Defined (rolling hash):** chunk boundary ని **content** నిర్ణయిస్తుంది (rolling hash ఒక pattern hit అయినప్పుడు cut). Insert జరిగినా boundary అదే content వద్ద ⇒ **ఒక్క local chunk మాత్రమే** మారుతుంది ⇒ dedup survives. (borg/restic/rsync దీన్ని వాడతాయి; Dropbox 4MB fixed.)

3. Content-addressed storage: chunk id = SHA-256(content) .

- **Immutable** ⇒ forever cache, CDN-friendly.
- **Dedup automatic** - అదే content ⇒ అదే hash ⇒ ఒక్కసారే store (across ALL users/files).
- **Integrity** - download అయ్యాక hash verify ⇒ corruption detect.

4. Delta sync: client chunk hashes compute చేసి, "server వద్ద ఏవి లేవు?" అని అడుగుతుంది (HEAD /batch) ⇒ **missing chunks మాత్రమే** upload. 1GB file లో 1MB మార్పు ⇒ ~1-2 chunks upload (1GB కాదు).

5. Dedup vs privacy (senior nuance): global cross-user dedup = storage ఆదా కానీ **side-channel** (ఒక chunk "ఇదివరకే ఉంది" అంటే ఆ content వేరేవాళ్ళ దగ్గర ఉందని తెలుస్తుంది - existence leak). Solution: per-user dedup, లేదా **convergent encryption** (content hash = key ⇒ dedup + encrypted).

Go Implementation – content-defined chunker + content-addressed store (dedup + delta)

Buzhash rolling hash తో content-defined boundaries, SHA-256 content-addressing, `map[Hash][[]byte]` dedup store + refcount GC. `MissingChunks` = delta sync గుండా.

```
package cas

import (
    "crypto/sha256"
    "encoding/hex"
    "fmt"
    "math/rand"
    "sync"
)

// ---- Content-Defined Chunking (Buzhash rolling hash) ----
// Fixed chunks: file మొదట్లో byte insert ⇒ అన్ని chunks shift ⇒ dedup పోతుంది.
// Content-defined boundary (rolling hash) ⇒ local edit ⇒ local chunk మాత్రమే మారుతుంది.

const (
    minChunk = 2 << 10 // 2 KB (అతి చిన్న chunks నివారణ)
    maxChunk = 64 << 10 // 64 KB (అతి పెద్ద నివారణ)
    window   = 48       // rolling window (bytes)
    // mask: hash & mask == 0 ⇒ boundary. avg chunk ≈ 2^13 = 8 KB.
    mask     = (1 << 13) - 1
)

var bz [256]uint32

func init() {
    r := rand.New(rand.NewSource(1)) // deterministic seed ⇒ అన్ని nodes ఒకే
    boundaries
    for i := range bz {
        bz[i] = r.Uint32()
    }
}

func rotl(x uint32, k uint) uint32 { return x<<k | x>>(32-k) }

// Split – data ని content-defined chunks గా విభజన.
```

```

func Split(data []byte) [][]byte {
    var chunks [][]byte
    var h uint32
    start := 0
    for i := 0; i < len(data); i++ {
        h = rotl(h, 1) ^ bz[data[i]] // rolling: క్రిత byte కలుపు
        if i-start >= window {
            h ^= rotl(bz[data[i-window]], uint(window%32)) // window దాటిన
byte తీసి
        }
        size := i - start + 1
        if (size >= minChunk && h&mask == 0) || size >= maxChunk {
            chunks = append(chunks, data[start:i+1])
            start = i + 1
            h = 0
        }
    }
    if start < len(data) {
        chunks = append(chunks, data[start:])
    }
    return chunks
}

// ---- Content-Addressed Store (dedup + refcount GC) ----
type Hash string

func hashOf(b []byte) Hash {
    s := sha256.Sum256(b)
    return Hash(hex.EncodeToString(s[:]))
}

type Store struct {
    mu      sync.RWMutex
    blobs map[Hash][]byte // content-addressed: hash → blob (immutable)
    refcnt map[Hash]int // GC: 0 → unreferenced
}

func NewStore() *Store {
    return &Store{blobs: map[Hash][]byte{}, refcnt: map[Hash]int{}}
}

// Put – blob store + hash return. అదే content ఉంటే మళ్ళీ store చేయదు (dedup).
func (s *Store) Put(b []byte) Hash {
    h := hashOf(b)
    s.mu.Lock()
    defer s.mu.Unlock()
    if _, ok := s.blobs[h]; !ok {
        cp := make([]byte, len(b))
        copy(cp, b) // caller buffer reuse చేయొచ్చు → copy
        s.blobs[h] = cp // మొదటిసారి మాత్రమే (dedup)
    }
    s.refcnt[h]++
    return h
}

func (s *Store) Get(h Hash) ([]byte, bool) {
    s.mu.RLock()
    defer s.mu.RUnlock()
    b, ok := s.blobs[h]

```

```

        return b, ok
    }

    // MissingChunks – delta sync: client hashes పంపితే, లేని వాటిని return
    // ⇒ client అవి **మాత్రమే** upload (bandwidth అదా).
    func (s *Store) MissingChunks(hashes []Hash) []Hash {
        s.mu.RLock()
        defer s.mu.RUnlock()
        var missing []Hash
        for _, h := range hashes {
            if _, ok := s.blobs[h]; !ok {
                missing = append(missing, h)
            }
        }
        return missing
    }

    // release – refcount 0 (file delete తర్వాత) unreferenced chunk తొలగించు (GC).
    func (s *Store) release(h Hash) {
        s.mu.Lock()
        defer s.mu.Unlock()
        if s.refcnt[h]--; s.refcnt[h] <= 0 {
            delete(s.blobs, h)
            delete(s.refcnt, h)
        }
    }

    // ---- Manifest (file → ordered chunk hashes) ----
    type Manifest struct {
        Name   string
        Size   int64
        Chunks []Hash
    }

    // Upload – file ని chunk → కొత్త chunks మాత్రమే store (dedup) → manifest.
    // Return: manifest + నిజంగా store అయిన bytes (dedup ఎంత ఆదా చేసిందో చూపుతుంది).
    func Upload(s *Store, name string, data []byte) (Manifest, int) {
        m := Manifest{Name: name, Size: int64(len(data))}
        storedBytes := 0
        for _, ch := range Split(data) {
            _, existed := s.Get(hashOf(ch))
            h := s.Put(ch)
            if !existed {
                storedBytes += len(ch) // ఈ chunk కొత్తది (dedup కాలేదు)
            }
            m.Chunks = append(m.Chunks, h)
        }
        return m, storedBytes
    }

    // Download – manifest chunks ని order లో join చేసి file reconstruct + integrity verify.
    func Download(s *Store, m Manifest) ([]byte, error) {
        out := make([]byte, 0, m.Size)
        for _, h := range m.Chunks {
            b, ok := s.Get(h)
            if !ok {
                return nil, fmt.Errorf("missing chunk %s", h)
            }
            if hashOf(b) != h { // content-addressed ⇒ integrity check

```

```

        return nil, fmt.Errorf("corrupt chunk %s", h)
    }
    out = append(out, b...)
}
return out, nil
}

```

Presigned URL (app server offload): app server ఒక HMAC-signed, time-limited URL ఇస్తుంది; client నేరుగా blob store కి upload/download చేస్తాడు (app server bandwidth ఆదా, secret client కి వెళ్ళదు):

```

package cas

import (
    "crypto/hmac"
    "crypto/sha256"
    "encoding/hex"
    "fmt"
    "time"
)

// Presign - path + expiry ని HMAC తో sign; verify: exp future? + HMAC match?
// => blob store నేరుగా serve చేస్తుంది (ప్రతి request కి auth service అవసరం లేదు).
func Presign(secret []byte, path string, exp time.Time) string {
    mac := hmac.New(sha256.New, secret)
    fmt.Fprintf(mac, "%s\n%d", path, exp.Unix())
    sig := hex.EncodeToString(mac.Sum(nil))
    return fmt.Sprintf("/blob/%s?exp=%d&sig=%s", path, exp.Unix(), sig)
}

```

Go ఎందుకు perfect: `crypto/sha256` content-addressing, `map[Hash][]byte` + `sync.RWMutex` dedup store (read-heavy: చాలా HEAD checks, తక్కువ writes). Buzhash rolling hash = streaming chunking. `crypto/hmac` presigned URLs = stateless auth. Production: blobs = S3/disk + erasure coding, metadata = sharded Postgres/Cassandra, కానీ **model ఇదే**.

Bottlenecks & Trade-offs

- **Metadata scale:** billions of files => metadata DB **shard by userId**; manifest చిన్నది (references) కాబట్టి RAM-cacheable.
- **Fixed vs CDC chunking:** fixed simple కానీ insert-shift dedup పోతుంది; CDC dedup మేలు కానీ CPU (rolling hash) + variable chunk sizes.
- **Hot chunks:** popular file (viral video) chunk => read hotspot => **CDN + replication** (content-addressed => cache forever, trivial).
- **Dedup vs privacy:** global dedup storage ఆదా కానీ existence side-channel => per-user dedup / **convergent encryption**.
- **GC races:** chunk delete అవుతుండగా మరో file అదే chunk reference చేస్తే? refcount race => **mark-sweep** (concurrent) లేదా delete ముందు grace period.

- **Durability - replication (×3) vs erasure coding (~1.4×):** EC storage-efficient కానీ node fail repair = CPU + network heavy (k chunks చదివి reconstruct).
- **Metadata↔blob consistency:** blob ముందు commit, తర్వాత manifest ⇒ manifest ఎప్పుడూ committed chunks నే reference చేస్తుంది (orphan chunks GC తీసేస్తుంది); reverse అయితే dangling reference.
- **Small files:** chunk/manifest overhead చిన్న files కి పెద్ద ratio ⇒ small files ని pack (combine).

Key Points

- **Chunking** = resumable upload + dedup granularity + delta sync + parallelism.
- **Content-Defined Chunking (rolling hash)** > fixed: insert-shift లో dedup survives (local chunk మాత్రమే మారుతుంది).
- **Content-addressed (SHA-256)** = automatic dedup (అదే content ⇒ ఒక్కసారి store) + immutability + integrity verify.
- **Delta sync** = missing chunks మాత్రమే upload ⇒ 1GB file లో చిన్న edit = KBs transfer.
- **Presigned URLs** = client నేరుగా blob store కి (app server offload); **erasure coding** = storage-efficient durability.
- Trade-offs: dedup↔privacy (convergent encryption), fixed↔CDC, replication↔EC, GC refcount races.
- Go: `sha256` content-addressing, Buzhash rolling chunker, `map+RWMutex` dedup store, `hmac` presigned URLs.

Part 3 – Reference

20. Common Patterns Across Designs (Recap)

ఇప్పటిదాకా 16 case studies చేశాం. వాటిల్లో **అవే patterns** మళ్ళీ మళ్ళీ వచ్చాయి. ఇవి interview లో ఏ system కైనా reach చేసే "tools" - ఒకసారి పట్టుకుంటే ఏ design అయినా వీటి combination గా కనిపిస్తుంది.

COMMON PATTERNS • ప్రతి DESIGN LO వచ్చేవి

DESIGNS అన్నిటిలోనూ తిరిగి వచ్చే భాగాలు

Load balancer	traffic పంచడం + health check	Cache	read-heavy path ని తేలిక చేయడం
Queue	producer/consumer ని విడదీయడం	Sharded store	data ని విభజించడం
Worker pool	background పని – Go lo సహజం	Observability	ఇది లేకపోతే debug అసాధ్యం

Interview vyuham

ఈ అరు భాగాలతో మొదటి architecture గీయండి – ఏ problem కైనా ఇది ఒక మంచి మొదలు.
తర్వాత problem-నిర్దిష్టమైన భాగాన్ని (geo index, trie, ledger) చేర్చడం.
ఇది "ఖాళీ board" భయాన్ని తీసేస్తుంది.

Real-life Analogy

వంటవాడి 5 మసాలా దినుసులు. ఏ కూర చేసినా - పప్పు, కూర, పులుసు - అవే base masalas వేర్వేరు నిష్పత్తులలో. System design లో ఈ 5 patterns అలాంటివి: idempotency, dedup, backpressure, sharding, caching. కొత్త "recipe" (system) వచ్చినా, ఇవే దినుసులు.

Pattern 1 – Idempotency (retry-safe operations)

సమస్య: network unreliable $\Rightarrow$ retries; duplicate $\Rightarrow$ double-effect (double charge, double notification).

Solution: operation కి unique key; already-processed అయితే re-execute కాదు, stored result తిరిగి ఇవ్వు.

ఎక్కడ చూశాం	ఎలా
§8 Notification	idempotency key + dedup store (SET NX)
§12 Job Scheduler	per-run lock + idempotent job body

ఎక్కడ చూశాం	ఎలా
§15 Payment	idempotency key + UNIQUE constraint (exactly-once effect)

Interview line: "At-least-once delivery + idempotent operations = exactly-once *effect*. True exactly-once *delivery* distributed లో అసాధ్యం."

Pattern 2 – Deduplication

సమస్య: same event/message multiple times (retries, at-least-once queue, duplicate publish).

Solution: dedup store (key → seen), TTL. Redis `SET NX`, bloom filter (probabilistic, memory-efficient).

ఎక్కడ చూశాం	ఎలా
§8 Notification	idempotencyKey Redis SET NX, TTL 48h
§6 Chat	clientMsgId (duplicate send dedup)
§15 Payment	idempotency key unique index

Pattern 3 – Backpressure (protect from overload)

సమస్య: producer fast, consumer slow ⇒ memory పెరిగి crash (unbounded queue).

Solution: bounded buffers; full అయితే - block (slow producer), drop (shed load), లేదా reject (429).

ఎక్కడ చూశాం	ఎలా
§6 Chat hub	buffered <code>send</code> chan + <code>default: drop</code> (slow client)
§5 Rate limiter	429 reject (protect downstream)
§3 Worker pool	bounded workers (DB overwhelm నివారణ)
§10 Transcode	semaphore (bounded parallel encodes)

Go idiom: buffered channel = built-in backpressure. `select { case ch <- v: default: /* drop/shed */ }` = non-blocking send.

Pattern 4 – Sharding / Partitioning

సమస్య: ఒక్క node/DB/lock మొత్తం load handle చేయలేదు (storage, QPS, contention).

Solution: data/load ని N shards గా విభజన. Shard key ఎంపిక కీలకం (even distribution, hot-key నివారణ).

ఎక్కడ చూశాం	Shard key / ఎలా
§14 Cache	key hash → 256 shards (lock contention split)
§13 KV store	consistent hashing (key → node)
§11 Uber	geography/region → shard (hot-region isolation)
§6 Chat	convId → partition (ordering + parallelism)
§4 URL	shortCode hash → DB shard

Consistent hashing (§3) = shard rebalancing pain తగ్గిస్తుంది (node change ~1/N keys, mod-N కాదు).

Pattern 5 – Caching

సమస్య: read-heavy; DB ప్రతి read కి = slow + expensive.

Solution: hot data RAM లో (cache-aside, read-through, write-through). Hit ratio 95%+ target.

ఎక్కడ చూశాం	ఎలా
§4 URL	Redis shortCode→longUrl (cache-first redirect)
§7 Feed	precomputed timeline cache (Redis sorted set)
§9 Typeahead	in-memory trie + hot prefix edge cache
§14 Cache	THE cache (sharded LRU)

Cache pitfalls: stampede (§14 singleflight), stale (TTL + invalidation), hot key (local cache/replication).

Bonus patterns (across designs)

Pattern	ఎక్కడ	సారాంశం
Fan-out	§7 Feed, §6 Chat	ఒక event → N recipients (write vs read)
Queue as buffer	§8 Notification, §10 Video	spike absorb, decouple producer/consumer
Leader election	§12 Scheduler	single coordinator (etcd/Raft lease)
Quorum	§13 KV store	R/W tunable consistency ($W+R>N$)
Read-repair / anti-entropy	§13 KV store	eventual convergence of replicas
Saga + compensation	§15 Payment	multi-service distributed transaction

Pattern	ఎక్కడ	సారాంశం
Circuit breaker	§8 Notification	failing dependency ⇒ fast-fail (retry storm నివారణ)
CDN / edge	§10 Video, §4 URL	serve close to user, origin shield

Go concurrency ↔ pattern mapping (cheat)

System pattern	Go mechanism
Fan-out	worker pool (channel + WaitGroup) §3
Single-owner state	channel hub goroutine §3 (Chat §6)
Bounded concurrency	semaphore (buffered channel) §3
Shard routing	consistent hash ring §3 / hash+mask §14
Backpressure	buffered channel + <code>select default</code>
Cancellation/timeout	<code>context.Context</code> (§10, §12, §13, §15)
Lock-free reads	<code>atomic.Pointer</code> COW (Typeahead §9)
Fine-grained locking	sharded <code>sync.Mutex</code> (Cache §14)
Quorum wait	goroutines + buffered channel (KV §13)

Key Points

- 16 systems = 5 core patterns (idempotency, dedup, backpressure, sharding, caching) + bonus (fan-out, queue, leader, quorum, saga) recombined.
- **Idempotency + dedup** = correctness under retries (exactly-once effect).
- **Backpressure + sharding + caching** = scale + protection.
- ప్రతి pattern కి ఒక Go mechanism ఉంది - ఈ mapping పట్టుకుంటే code instant గా వస్తుంది.
- కొత్త system వచ్చినా: "ఏ patterns? ఏ Go primitives?" - అదే framework.

21. System-Design-in-Go Interview Checklist + Cheat Sheets

ఇది చివరి section - interview కి ముందు 10 నిమిషాలు చదివే "revision card". Framework, memory tips, common mistakes, numbers - అన్నీ ఒకచోట.

Real-life Analogy

పరీక్షకు ముందు formula sheet. అన్ని case studies చదివాక కూడా, పరీక్ష హాల్లోకి వెళ్ళే ముందు ఒక్క page చూస్తావు - అన్ని sutras ఒకచోట. ఇది ఆ page.

The Interview Framework (revision)

1. REQUIREMENTS → Functional + Non-functional. Scope narrow. Read vs write heavy?
2. ESTIMATION → DAU → QPS → Peak($\times 3$) → Storage → Bandwidth → Cache size.
3. API DESIGN → REST/gRPC endpoints. Idempotency key? Pagination?
4. DATA MODEL → SQL(relations/ACID) vs NoSQL(scale/flex). Access pattern.
5. HIGH-LEVEL → Client → LB → Service → Cache → DB → Queue. Draw boxes.
6. DEEP DIVE → 1-2 hardest: bottleneck, sharding, hot key, failure mode.
7. WRAP UP → Node down? Region down? Retry storm? Monitoring? What next?

Choosing the right approach (decision cheat)

ప్రశ్న	ఎంపిక
Read >> Write?	Cache + read replicas (§4, §7)
Write >> Read? / high write?	LSM store, in-memory index, sharding (§11, §13)
Strong consistency (money)?	SQL + ACID + serializable (§15)
High availability > consistency?	NoSQL, quorum $W=R=1$, eventual (§13)
Persistent connections?	WebSocket + goroutines (§6)
Fan-out ($1 \rightarrow N$)?	worker pool; write vs read (celebrity §7)
Proximity/geo query?	geohash/quadtree (§11)
Rate control?	token bucket / sliding window (§5)
Coordinate one leader?	etcd/Raft lease (§12)
Duplicate-safe?	idempotency key + unique constraint (§8, §15)
Node scale in/out smoothly?	consistent hashing (§13, §14)

SQL vs NoSQL (quick pick)

వాడు SQL అయితే...	వాడు NoSQL అయితే...
Relations, joins, transactions (payment)	Massive scale, flexible schema (feed, KV)
Strong consistency (ACID)	High write throughput, eventual OK
Complex queries	Simple key-value / doc access

వాడు SQL అయితే...	వాడు NoSQL అయితే...
§15 Payment	§4 URL, §7 Feed, §13 KV

Go-specific interview points (say these!)

- "ఒక్కో connection/request కి goroutine cheap (~KB stack) ⇒ millions concurrent (§6)."
- "Channel = fan-out + backpressure clean గా (worker pool §7, hub §6)."
- "Single-owner goroutine ⇒ shared state కి lock అవసరం లేదు (channel hub §6)."
- "Sharded mutex ⇒ lock contention split (cache §14)."
- " `atomic.Pointer` COW ⇒ lock-free reads (typeahead §9)."
- " `context` ⇒ cancellation/deadline goroutine tree అంతా (§10, §12, §13, §15)."
- "Static binary ⇒ k8s fast autoscale (transcode §10)."
- "Money = int64 minor units, float64 ఎప్పుడూ కాదు (§15)."

Memory Tips (గుర్తుంచుకునే మంత్రాలు)

System	ఒక్క వాక్యం మంత్రం
§4 URL Shortener	"Counter + base62, cache-first redirect (read-heavy)"
§5 Rate Limiter	"Token bucket lazy refill; Redis+Lua for distributed"
§6 Chat	"2 pumps/conn, single-owner hub, backpressure drop"
§7 Feed	"Fan-out write for normal, read-merge for celebrities"
§8 Notification	"Dedup → prefs → render → classified retry → DLQ"
§9 Typeahead	"Trie + node top-k precompute, atomic COW swap"
§10 Video	"Async transcode, idempotent jobs, CDN 99% playback"
§11 Uber	"Geohash bucket + 8 neighbors, atomic driver reserve"
§12 Scheduler	"etcd lease leader, min-heap due, idempotent at-least-once"
§13 KV Store	"Consistent hash + quorum W+R>N, read-repair"
§14 Cache	"Sharded LRU (256 mutexes), map+list O(1)"
§15 Payment	"Idempotency key + double-entry + serializable ACID"

Master mantra: "Estimate → shard the load, cache the reads, queue the writes, make it idempotent, and pick your consistency."

Common Mistakes (ఏటిని చేయకు!)

తప్పు	సరైనది
Requirements clarify చేయకుండా boxes గీయడం	ముందు functional + non-functional + scale అడుగు
Estimation skip చేయడం	Numbers design ని drive చేయాలి (QPS $\Rightarrow$ cache/shard?)
అన్నిటికీ strong consistency	Feed/cache eventual OK; payment మాత్రమే strong
hash(key) % N sharding	Consistent hashing (node change $\sim 1/N$, not all)
ప్రతి goroutine కి unbounded spawn	Worker pool / semaphore (bounded)
Money ని float64	int64 minor units
Retry అన్నిటిని (permanent కూడా)	Classify: retryable vs permanent $\rightarrow$ DLQ
"Exactly-once delivery" claim	At-least-once + idempotent = exactly-once <i>effect</i>
Single leader/DB/lock bottleneck ignore	Shard, replicate, leader-per-shard
Celebrity fan-out on write (10M writes)	Hybrid: read-time merge for celebrities
Cache stampede ignore	Singleflight / coalescing
Slow consumer unbounded buffer	Backpressure: bounded + drop/shed
Silent (think aloud చేయకపోవడం)	ఆలోచన బయటికి చెప్పు (reasoning = seniority signal)
Global ordering అనవసరంగా	Per-entity ordering చాలు (per-conv seq §6)
Monitoring/failure modes మర్చిపోవడం	Wrap-up లో "node down? region down?" చెప్పు

Numbers Cheat Sheet

QPS:

$1\text{M/day} \approx 12/\text{sec}$ $1\text{B/day} \approx 12\text{K/sec}$ $1\text{ day} \approx 10^5\text{ sec}$
 Peak QPS = avg $\times 2-3$

Storage:

$1\text{M users} \times 1\text{KB} = 1\text{ GB}$ $1\text{B} \times 1\text{KB} = 1\text{ TB}$
 $\text{KB} \rightarrow \text{MB} \rightarrow \text{GB} \rightarrow \text{TB} = \times 1000\text{ each}$

Rule of thumb:

Cache hot set $\approx 20\%$ data (80-20)
 Read replicas for read-heavy; sharding for storage/write-heavy
 Replication $N=3$ (durability); quorum $W+R>N$ (strong-ish)

Latency Cheat Sheet

L1 cache	$\sim 1\text{ ns}$		Mutex lock/unlock	$\sim 25\text{ ns}$
RAM reference	$\sim 100\text{ ns}$		SSD random read	$\sim 100\text{ }\mu\text{s}$
Same-DC RTT	$\sim 0.5\text{ ms}$		HDD seek	$\sim 10\text{ ms}$

Cross-region RTT ~50-150 ms |

గుర్తు: RAM $\approx$ disk $\times$ 1000 fast; same-DC $\approx$ cross-region $\times$ 100 fast
→ cache RAM లో, replicas same region లో, CDN edge దగ్గర

Consistency spectrum (ఎప్పుడు ఏది)

STRONG ← → EVENTUAL

Payment(\$15)	Chat order(\$6)	KV quorum(\$13)	Feed(\$7)	Cache(\$14)
serializable	per-conv seq	W+R>N tunable	eventual	eventual/TTL

Rule: correctness-critical (money) = strong; scale/availability = eventual.
CAP: partition లో C vs A ఎంచుకో. Payment=CP, Dynamo-KV=AP.

Final revision flow (interview రోజు)

1. Framework 7 steps గుర్తు తెచ్చుకో
2. ఈ 12 mantras (Memory Tips table) skim చేయి
3. Go points (goroutine/channel/context/int64-money) ready గా పెట్టు
4. Numbers + latency cheat ఒకసారి చూడు
5. Common mistakes list చూసి "వీటిని చేయను" అనుకో

Key Points

- **7-step framework** ప్రతి problem కీ; **estimation design ని drive చేస్తుంది.**
- **Master mantra:** estimate → shard load, cache reads, queue writes, idempotent, pick consistency.
- Go strengths (goroutine/channel/context/sharded-mutex/atomic) ని ప్రతి design లో articulate చేయి.
- **Common mistakes** avoid = seniority signal; **think aloud** ఎప్పుడూ.
- Correctness-critical = strong (SQL/ACID); scale = eventual (NoSQL/quorum/cache).
- ఈ doc = `HLD_Go_Telugu.md` building blocks మీద built applied case studies. రెండూ కలిపి = interview-ready.

ముగింపు: OOP → LLD → System Design (HLD) → ఈ **applied Go case studies** - ఇది ప్రయాణపు చివరి మెట్టు. ప్రతి system ని ఒక real-life analogy తో గుర్తుంచుకో, ప్రతి hard component ని Go code తో reason చేయి. "ఒకసారి చదివితే మర్చిపోకూడదు" - అదే ఈ doc లక్ష్యం. All the best! 🚀